



UNIVERSIDAD DE MÁLAGA



E.T.S. INGENIERÍA
INFORMÁTICA
UNIVERSIDAD DE MÁLAGA

Grado en Ingeniería del Software

Biblioteca extensible de optimización metaheurística en Rust

Extensible metaheuristic optimization library in Rust

Realizado por

David Muñoz del Valle

Tutorizado por

Gabriel Jesús Luque Polo

Departamento

Departamento de Lenguajes y Ciencias de la Computación

MÁLAGA, Junio 2026



UNIVERSIDAD
DE MÁLAGA



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA
GRADO EN INGENIERÍA DEL SOFTWARE

Biblioteca extensible de optimización metaheurística en Rust

Extensible metaheuristic optimization library in Rust

Realizado por

David Muñoz del Valle

Tutorizado por

Gabriel Jesús Luque Polo

Departamento

Departamento de Lenguajes y Ciencias de la Computación

UNIVERSIDAD DE MÁLAGA

MÁLAGA, JUNIO 2026

Fecha defensa: Julio de 2026

Resumen

El presente Trabajo de Fin de Grado tiene como objetivo principal el diseño, desarrollo e implementación de una biblioteca de optimización metaheurística de alto rendimiento, caracterizada por una arquitectura extensible, modular y completamente autocontenida bajo una filosofía de cero dependencias externas. Para cumplir con estas directrices de diseño, el sistema ha sido programado íntegramente en Rust, un lenguaje de programación moderno que garantiza la seguridad de memoria y la ausencia de condiciones de carreras sobre los datos sin comprometer la eficiencia temporal ni el consumo de recursos de la máquina.

Con el fin de materializar este propósito principal, el proyecto se articula en torno a una serie de objetivos específicos destinados a aportar valor tanto desde el prisma de la ingeniería de software como de la computación evolutiva. En primer lugar, se persigue la definición de abstracciones sólidas mediante contratos de interfaz flexibles que desacoplen por completo la representación del dominio de los problemas de la lógica intrínseca de las metaheurísticas y sus operadores. En segundo lugar, se establece el objetivo de dar soporte nativo y transparente a escenarios de optimización tanto mono-objetivo como multi-objetivo, superando las rigideces estructurales comunes en plataformas tradicionales. Por último, la biblioteca busca proveer un motor de experimentación robusto y paralelizado, dotado de mecanismos de tolerancia a fallos y sistemas de monitorización en tiempo real, que facilite el ajuste de hiperparámetros y la realización de análisis comparativos frente a las principales herramientas de referencia de la literatura científica.

Palabras clave: Rust, Metaheurísticas, Optimización, Biblioteca

Abstract

The main objective of this Bachelor's Thesis is the design, development, and implementation of a high-performance metaheuristic optimization library, characterized by an extensible, modular, and fully self-contained architecture under a zero-external-dependencies philosophy. To comply with these design guidelines, the system has been programmed entirely in Rust, a modern programming language that guarantees memory safety and the absence of data races without compromising time efficiency or machine resource consumption.

In order to achieve this core purpose, the project is structured around a series of specific objectives aimed at providing value from the perspectives of both software engineering and evolutionary computation. First, it aims to define solid abstractions through flexible interface contracts that completely decouple the problem domain representation from the intrinsic logic of the metaheuristics and their operators. Second, it establishes the objective of providing native and transparent support for both single-objective and multi-objective optimization scenarios, overcoming the structural rigidities common in traditional frameworks. Finally, the library seeks to provide a robust and parallelized experimentation engine, equipped with fault-tolerance mechanisms and real-time monitoring systems, to facilitate hyperparameter tuning and comparative analysis against the leading reference tools in the scientific literature.

Keywords: Rust, Metaheuristics, Optimization, Library

Agradecimientos

Este es el único apartado en el que utilizaré la primera persona. No porque sea el único en el que esté permitido hacerlo, sino porque es el único donde realmente escribiré yo.

En este capítulo quiero resumir mi paso por la facultad, pero no me centraré en lo académico, sino en las personas que me han acompañado. Sé que inevitablemente me dejaré a muchos sin nombrar para no extenderme demasiado, pero quiero que sepáis que os llevo conmigo en el recuerdo de estos años.

Que viniera aquí no estaba entre mis planes, fue el sitio donde pude entrar con mi nota de selectividad. Nunca tuve muy claro lo que estudiar y de rebote acabé cursando Ingeniería Informática en Málaga. Llegué solo, nadie que conocía estudiaría en Málaga, sin embargo aquí, me esperaba muchísima gente increíble.

Los primeros días hice migas con los que todavía son mis amigos, un grupo maravilloso a los que les debo muchísimo, sin ellos no hubiera aguantado el primer cuatrimestre. El cuatrimestre que tan complicado se nos hizo con Cálculo impartida por **Yadira** o por Electrónica, de la cual mejor no doy más detalles.

Aquí conocí a **Carlos**, pocas personas me hacen reír tanto. También a **Tomás**, que aunque ya no nos veamos mucho, aprendí mucho de él. Luego a **Suito** que con su amabilidad, su sonrisa y ahora con su gorra nos cautiva a todos, **Dino**, la persona más bondadosa que existe, no exagero, nunca he visto un ápice de maldad en este chico. **Sofía**, nuestra China, luego **Salma**, la persona con más imaginación que conozco y la chica con mas calle de la ETSII. La **Marta**, la señora de los viajes, se ha tirado más tiempo en aviones que estudiando, es una crack, ahora te corre 20K y se la pela. **Jose**, se describe a si mismo como un pato galáctico pero es imposible ser más listo y más gafas, te habla en 100 idiomas y siempre está para todos. Por último **Roberto**, un rondeño de ojos bonitos. La vida nos puso en el mismo gimnasio y en la misma clase.

Avanzaron los cursos, perdí el contacto con muchos y en tercero decidí cambiarme de modalidad, tuve que adaptarme a una nueva clase, a un nuevo conjunto de personas. En este cambio me acompañó

Suito. Con él tenía una manía rara, nos gustaba tocarnos cariñosamente las piernas, por tanto no nos quedó otra que mezclarnos con un grupo para camuflarnos entre ellos y no parecer *raritos*. Este realmente no era un grupo, era el equipo JAJER.

Con nuestra incorporación el equipo se transformó en JAJERDA, ya que el nombre se forma de las iniciales de los miembros que lo componen. La primera J, de **Joge**, una bestia de la programación, enamorado del baloncesto, la A de **Artu**, el rey del grupo, siempre con una sonrisa, el cortisol le teme a él. La siguiente J le pertenece a **Juan Manuel**. Juanma tiene varias pasiones; derrapar, los bocadillos baratos, las Ondas de Elliot, el monte, la playa y la novia de Eduardo, ... yo sinceramente soy el fan número uno de Juanma. La E obviamente, de **Eduardo**. Un cordobés pila pila fuerte, **gym rat science based**, le gusta mucho el color negro y está potente. La última letra antes de las que nos corresponden a mi y a Suito(Alejandro), es la R, de ni más ni menos que de **Rubén**, el G.O.A.T del equipo, la mente pensante, mi compañero del Bar **Avilés**, el friki de los periféricos, el humano en el loop (vibecodea mucho), la humildad y la generosidad personificada. Este hombre nos pasará a todos por la izquierda, en unos meses lo veremos muy alto.

Durante este año me mudé, empecé a compartir piso con Sofía, la famosa china. Durante la convivencia la conocí mucho mejor y realmente me llevé una grata sorpresa, me sorprendió para bien. Sofía es increíble y realmente es capaz de lo que se proponga, tiene una capacidad de inimaginable de enfocarse y conseguir siempre lo que quiere.

Al año siguiente, en mi último curso, me volví a mudar y cambiaron de nuevo mis compañeros de piso. Estos eran Roberto, el cual ya conocéis y **Pablo**. La convivencia durante este curso fue inmejorable y esto es gracias a que ellos también lo eran. Pablo es un crack absoluto, es un futuro mayordomo de Montejaque y el fan número uno del Málaga FC. Servicial y siempre está para sus allegados. Sobre Roberto ya está todo dicho, el más cariñoso de toda ronda, le da miedo conseguir dopamina de una fuente que no sea mirar a una pared durante treinta minutos, futura medalla olímpica de tiro con honda y apasionado número uno de la tortilla francesa con patatas fritas, pero aun así, es el mejor.

En esta nueva edición del piso Software, se vivieron incendios, grandes partidos del Albacete, intensas jornadas de repostería y escenas espirituales.... Sobre esta última debo agradecer a **Rosa**, quien me enseñó sobre la importancia de estar relajado durante estas jornadas.

Este último curso me matriculé en la asignatura donde cursaría mis prácticas curriculares, donde

hice migas con **María** la gigante de los datos, con **Marta**, con **Matteo**, con **Pedro** y con **Álvaro**, la persona que más sabe de todo, literalmente, de todo. Todos son unos *cracks* en toda regla. Aunque con ellos fuera con los que más hablara por estar en la misma condición de becario, no me quiero ir sin agradecer a todo el equipo de **BHS Corrugated Spain**, que son todos geniales. Creo que no podría haber tenido más suerte con ellos. Echaré de menos el *Antojitos* y el bocadillo de Pata de los jueves .

Durante estos años siendo parte del equipo JAJERDA, participé en varios Hackathones, en una de estos tuve la oportunidad de conocer a un profesor del que me siempre había escuchado buenas palabras. Le pregunté sobre una duda que tenía y él estuvo ayudando de forma altruista como si el problema le afectara de primera mano, este profesor ahora es el tutor este TFG, muchas gracias **Gabriel**, no eres consciente lo agradecidos que estamos los alumnos por tenerte como docente.

Tras mis años en el grado también hubo personas que me ayudaron muchísimo, provenían de antes y siempre estuvieron allí. Obviamente hablo de mi familia y de mis amigos más cercanos. Ellos aunque suene muy cliché son de verdad los pilares de mi mismo, sin ellos yo me desplomaría, no sería capaz de nada. Con algunos he pasado todo lo que recuerdo de vida, **Agustín y Guiri**, todavía me acuerdo cuando de chicos jugábamos al fútbol y al *torito en alto* en el parque. Otros llegasteis un poco más tarde, como **Oncala**, que aunque hayamos pasado por altos y bajos, es de las personas en las que más confío.

Algunos más tarde con el balonmano como **Rámirez y Sergio**, mis *compas* de la Sierra, mis exhibicionistas y fiesteros. Otros durante el instituto, como **Barrera**, nuestro bombero y **Andrés**, la persona más loca que conozco, en el buen sentido.

A diferencia de los demás casos mi familia no llegó, yo llegué a ellos, mi madre y mi padre, los que siempre están, los que creen más en mí que yo mismo, los que me dan cobijo cuando lo necesito. Cuando estoy agobiado, cuando estoy enfermo, siempre me cuidaban. Cuando quería siempre podía volver a casa con ellos. Por su cariño, por su compañía, por sus batidos de naranja a las cinco, por llevarme al balonmano, por costearme esta carrera, por ser mis padres... A ellos se lo debo todo, gracias **Papá**, gracias **Mamá**. Muchas gracias por estar. Gracias también a **Luis**, que aunque gruñón, tienes muy buen corazón, el mejor hermano que se puede tener. Gracias a **Laura** y a **Mario**, a vosotros también os considero mis hermanos.

Falta todavía una persona por agradecer.

Esta persona constituye un caso un poco anticlimático. Antes de los dos nacer ya éramos amigos. La

vida nos mantuvo siempre cerca; en la misma clase en infantil, en la misma clase del colegio, en la misma clase del instituto, en el mismo grupo de amigos, en el mismo equipo de balonmano... Intentamos incluso irnos a estudiar a la misma ciudad para poder compartir piso y pasar juntos una etapa más.

Siempre contestaré con tu nombre cuando me pregunten sobre quien es mi mejor amigo, siempre me pusiste en primer lugar, me valoraste y me motivaste a ser mejor, a disfrutar y en los últimos momentos de tu vida, a programar.

Descansa en paz Moreno.

Resumen	1
Abstract	3
Agradecimientos	5
1 Introducción	15
1.1 Motivación	15
1.2 Objetivos	16
1.3 Tecnologías a utilizar	17
1.4 Estructura del documento	18
2 Metodología de trabajo	21
2.1 Modelo y ecosistema de desarrollo	21
2.2 Fases del proyecto	21
2.2.1 Investigación y capacitación	21
2.2.2 Definición de requisitos y planificación	22
2.2.3 Implementación técnica	22
2.3 Licencia de software	24
3 Optimización y Metaheurísticas	25
3.1 El Problema de Optimización	25
3.2 Métodos Exactos frente a Métodos Aproximados	26
3.3 Heurísticas y Metaheurísticas	27
3.4 Clasificación de las Metaheurísticas	28
3.4.1 Metaheurísticas basadas en trayectoria	28
3.4.2 Metaheurísticas basadas en población	29
3.5 Extensión a la Optimización Multi-Objetivo	30
3.5.1 Formulación Matemática del Problema y Dominancia	31
3.5.2 Algoritmo NSGA-II (Non-dominated Sorting Genetic Algorithm II)	31
4 Especificación y diseño	33

4.1	Arquitectura	33
4.2	Módulos	34
4.2.1	Algoritmos	34
4.2.2	Experimentos	35
4.2.3	Observadores	35
4.2.4	Operadores	36
4.2.5	Problemas	37
4.2.6	Conjunto de soluciones	38
4.2.7	Soluciones	38
4.2.8	Utilidades	39
5	Implementación y pruebas	41
5.1	Implementación	41
5.1.1	Diseño y abstracción de las Soluciones	41
5.1.2	La abstracción de los Problemas	44
5.1.3	Operadores	46
5.1.4	Algoritmos	49
5.1.5	Observadores y checkpoints	57
5.1.6	Utilidades y experimentos	61
5.2	Desarrollo de pruebas	67
5.2.1	Pruebas unitarias	67
5.2.2	Pruebas de integración	69
5.3	Desarrollo de ejemplos	70
6	Evaluación Comparativa y Análisis de Rendimiento	77
6.1	Bibliotecas de Referencia	78
6.1.1	jMetal y jMetalPy	78
6.1.2	Pagmo (C++)	79
6.1.3	DEAP (Distributed Evolutionary Algorithms in Python)	79
6.1.4	Mealpy	79
6.1.5	SciPy	79

6.2	Evaluación con la Función de Rastrigin	79
6.2.1	Análisis Crítico de Resultados	80
6.2.2	Análisis de Significancia Estadística	82
6.3	Evaluación con el Problema del Viajante (TSP)	84
6.3.1	Análisis Crítico de Resultados	84
6.3.2	Análisis de Significancia Estadística	87
6.4	Evaluación Discreta: Problema de la Mochila 0/1 (Knapsack)	89
6.4.1	Análisis Crítico de Resultados	89
6.4.2	Análisis de Significancia Estadística	92
6.5	Evaluación Multi-Objetivo: Función ZDT1 con NSGA-II	93
6.5.1	Análisis Crítico de Resultados	93
6.5.2	Análisis de Significancia Estadística	95
6.6	Función de Ackley con Evolución Diferencial	96
6.6.1	Análisis Crítico de Resultados	97
6.6.2	Análisis de Significancia Estadística	98
7	Conclusiones y líneas futuras	101
7.1	Limitaciones Identificadas	102
7.2	Líneas futuras de trabajo	103
7.3	Conclusiones sobre el desarrollo y el cambio de paradigma	104
	Apéndice A. Manual de instalación	107
A.1	Requisitos	107
A.2	Instalación desde el repositorio	107
A.3	Instalación como <i>crate</i> mediante Cargo	108
	Apéndice B. Manual de uso	109
B.1	Inicialización y definición del problema	109
B.2	Configuración del motor heurístico	110
B.3	Acoplamiento de Observadores	110
B.4	Ejecución y extracción de resultados	111
B.5	Ensamblaje final del programa	111

1.1	Resultados de una de las encuestas a desarrolladores de Stack Overflow 2025 [30]. .	17
2.1	Captura de pantalla del estado del tablero Kanban en Jira durante la ejecución del proyecto.	23
3.1	Representación en el espacio de objetivos de un problema con dos funciones a minimizar (f_1 y f_2). La solución A domina a la solución B , ya que es estrictamente mejor en f_1 e igual o mejor en f_2	31
4.1	Arquitectura de la biblioteca. Diagrama de clases, UML. Visual Paradigm 17.0 [39]. .	40
5.1	Arquitectura de comunicación asíncrona entre hilos mediante eventos.	59
5.2	Flujo compacto de gestión de persistencia y recuperación de estados en Linux.	60
6.1	Representación tridimensional de la función de Rastrigin en el dominio $x, y \in [-5.12, 5.12]$. Se observa su topología altamente multimodal, caracterizada por una cuadrícula regular de numerosos mínimos locales que rodean al óptimo global situado en el origen. Imagen generada mediante el entorno computacional Wolfram Cloud [40].	80
6.2	Resultados empíricos para la Función de Rastrigin ($D = 80$) con el algoritmo Hill Climbing (3500 evaluaciones, 50 ejecuciones).	81
6.3	Calidad de la optimización combinatoria (TSP-48) obtenida por un Algoritmo Genético bajo un límite estricto de tiempo.	85
6.4	Resultados empíricos para la Mochila (80 items). *Los algoritmos Exact DP y Greedy se han excluido intencionalmente de la gráfica temporal para no distorsionar la comparativa.	90
6.5	Resultados del experimento ZDT1 con NSGA-II (25 000 evaluaciones). *La ejecución de pagmo2 arrojó fallos de validación.	94
6.6	Resultados empíricos para la Función de Ackley ($D = 35$) con Evolución Diferencial (6400 evaluaciones, 30 ejecuciones).	97

6.1	Métricas estadísticas consolidadas de Rastrigin ($D = 80$, 3500 evaluaciones).	81
6.2	Matriz de comparaciones múltiples mediante el test de Nemenyi para la aptitud en Rastrigin.	83
6.3	Métricas estadísticas de las distancias para la instancia <code>tsp_48</code> (Tiempo de Muro = 5s, 28 semillas).	85
6.4	Resumen del rendimiento computacional y calidad de convergencia por biblioteca en el límite de 5 segundos. Se evidencia la paradoja de la optimización: una mayor tasa de evaluaciones no garantiza una mejor aptitud final.	87
6.5	Matriz de comparaciones múltiples mediante el test de Nemenyi para las distancias del TSP.	88
6.6	Comparativa de calidad y latencia en el Problema de la Mochila ($D=80$).	90
6.7	Matriz de comparaciones múltiples (test de Nemenyi) para el subexperimento de <i>Binary PSO</i>	92
6.8	Matriz de comparaciones múltiples (test de Nemenyi) para el subexperimento de <i>Simulated Annealing</i>	92
6.9	Métricas estadísticas para la instancia ZDT1 (NSGA-II) con 25000 evaluaciones. . .	94
6.10	Matriz de comparaciones múltiples mediante el test de Nemenyi para el Hipervolumen (ZDT1 con NSGA-II).	95
6.11	Métricas estadísticas agregadas para la instancia <code>ackley_dim_35</code> (6400 evaluaciones). .	97
6.12	Matriz de comparaciones múltiples (test de Nemenyi) para la aptitud final en la función de Ackley ($D = 35$).	99

1

Introducción

1.1 Motivación

La optimización metaheurística constituye una herramienta fundamental para la resolución de problemas complejos en aquellos ámbitos donde los métodos exactos resultan inviables, ya sea debido al elevado coste computacional o a la naturaleza no lineal y no convexa de los espacios de búsqueda. Este tipo de técnicas ha demostrado su utilidad en una amplia variedad de aplicaciones, como la planificación y asignación de recursos, la optimización de rutas, el diseño de sistemas, la inteligencia artificial, el aprendizaje automático y la ingeniería en general [5].

En distintos lenguajes de programación existen bibliotecas maduras que facilitan la utilización de técnicas metaheurísticas. Por ejemplo, *jMetal* es un framework desarrollado por investigadores de la Universidad de Málaga, ampliamente reconocido para la optimización multi-objetivo con metaheurísticas [20]. Su versión en Python, *jMetalPy*, ofrece un entorno extensible y orientado a objetos para experimentar con técnicas clásicas y avanzadas de optimización [11]. Asimismo, *MEALPY* (MEta-heuristic ALgorithms in PYthon) proporciona implementaciones de numerosos algoritmos inspirados en la naturaleza, tanto clásicos como híbridos, para abordar problemas complejos [42]. Sin embargo, dentro del ecosistema de Rust [35], el soporte para la optimización metaheurística es todavía limitado, lo que dificulta la adopción de estas metodologías en proyectos desarrollados en dicho lenguaje y pone de manifiesto la necesidad de disponer de herramientas nativas que se integren de forma natural con sus principios de diseño, especialmente en lo relativo al rendimiento, la seguridad de memoria y el control de recursos.

Este Trabajo de Fin de Grado surge con la motivación de dar respuesta a esta necesidad mediante el desarrollo de una biblioteca de optimización metaheurística modular y extensible. No obstante, abordar un proyecto de esta envergadura resulta especialmente desafiante en las etapas iniciales,

particularmente en ausencia de experiencia previa tanto en las técnicas de optimización como en el lenguaje de programación Rust. Afrontar un grado de fin de carrera implica recopilar y consolidar los conocimientos adquiridos a lo largo de la titulación y aplicarlos en un proyecto real, lo cual supone una tarea de notable complejidad. Por ello, la elección de un tema que despierte interés, curiosidad y motivación personal resulta un factor clave para el desarrollo del proyecto.

El diseño y desarrollo de algoritmos metaheurísticos conforma un campo especialmente atractivo, al combinar fundamentos de la teoría de la computación, la optimización y la inteligencia artificial. Su aplicación en dominios tan diversos como la logística, la ingeniería o la biología computacional pone de manifiesto su versatilidad y su impacto en la resolución de problemas reales.

Como se ha comentado, en el momento de inicio de este proyecto, no existían bibliotecas consolidadas orientadas al desarrollo de este tipo de soluciones en el lenguaje Rust, lo que constituyó una motivación adicional para su realización. Asimismo, el propio lenguaje Rust actuó como detonante del proyecto, al tratarse de una tecnología moderna y en auge, diseñada con un fuerte énfasis en la seguridad de memoria y el rendimiento, y especialmente adecuada tanto para el aprendizaje como para el desarrollo de software robusto y eficiente.

1.2 Objetivos

El objetivo principal de este proyecto es desarrollar una biblioteca en Rust que implemente algoritmos metaheurísticos para la optimización de problemas complejos. Debe ser capaz de resolver correctamente gran variedad de problemas de único objetivo y un conjunto de problemas multi-objetivo.

Idealmente, esta biblioteca debería ser fácil de usar, eficiente y extensible, permitiendo a los usuarios adaptar los algoritmos a sus necesidades específicas.

Con el propósito de demostrar el potencial, la viabilidad y el rendimiento de la herramienta, se implementará un conjunto seleccionado de problemas y algoritmos de referencia que servirán como base de pruebas. Sobre esta base, se llevarán a cabo validaciones experimentales y comparaciones exhaustivas, analizando críticamente los resultados obtenidos para evaluar el comportamiento real y la eficiencia de la biblioteca.

Para garantizar el cumplimiento de estas premisas, se realizará un desglose exhaustivo de los requisitos del sistema, donde se definirán formalmente las funcionalidades y restricciones que guiarán el desarrollo de la herramienta.

1.3 Tecnologías a utilizar

Como lenguaje de programación, se ha elegido **Rust**, debido a su enfoque en el rendimiento, por su capacidad de manejo eficiente de la concurrencia y diferentes hilos, lo que lo hace ideal para la implementación de algoritmos metaheurísticos complejos que a menudo requieren un procesamiento paralelo intensivo. Además, su creciente ecosistema y su tipo de sistema robusto nos permiten construir soluciones de optimización escalables y de alto rendimiento. También, Rust ha comenzado a consolidarse como uno de los lenguajes de programación utilizados en el desarrollo del *kernel* de Linux, dejando atrás su carácter experimental y siendo aprobado oficialmente para la implementación de determinados componentes del núcleo, lo que refuerza su madurez y fiabilidad para sistemas críticos [25].

Uno de los principales motivos por el que Rust es tan buena elección es por su gestor de paquetes y compilación, **Cargo**, que facilita la gestión de dependencias y la construcción de proyectos, permitiendo a los desarrolladores centrarse en la implementación de algoritmos en lugar de en la configuración del entorno.

Este gestor de paquetes se encuentra entre los más populares y admirados por los desarrolladores, como se puede observar en la estadística de la encuesta anual de Stack Overflow, donde Cargo destaca como uno de los gestores de paquetes más valorados para el desarrollo e infraestructura en la nube durante 2025 [30].

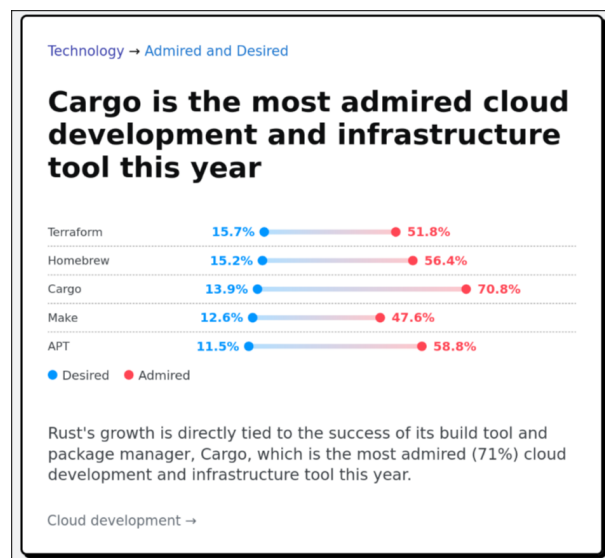


Figura 1.1. Resultados de una de las encuestas a desarrolladores de Stack Overflow 2025 [30].

En cuanto a las herramientas de desarrollo, se utilizó **Visual Studio Code** [28] como entorno de desarrollo integrado (IDE) debido a su flexibilidad, amplia gama de extensiones y soporte para Rust a través de la extensión *rust-analyzer* [3]. Durante el desarrollo más temprano, también se usó **Rust**

Rover [22], de **JetBrains** [21], que es un IDE específico para Rust. Esto fue debido a que en ese momento los conocimientos sobre Rust eran muy limitados y se necesitaba un entorno más guiado e invasivo para poder progresar.

La gestión y planificación temporal de las tareas se ha articulado mediante **Jira** [2], mientras que la composición y redacción de la memoria técnica se han elaborado en $\text{\LaTeX}$ [33]. En lo relativo a la ingeniería de software, el control de versiones se ha estructurado en torno al sistema descentralizado **Git** [36], empleando la plataforma **GitHub** [15] para el alojamiento del repositorio remoto.

La adopción de este ecosistema responde a la necesidad de garantizar la integridad y la trazabilidad del código fuente durante el ciclo de vida del desarrollo. Por un lado, la naturaleza distribuida de Git permite la experimentación segura y el aislamiento de nuevos algoritmos sin comprometer la estabilidad de la rama principal de la biblioteca. Por otro lado, GitHub aporta una infraestructura robusta para la persistencia de los datos, optimizando el seguimiento de las líneas de desarrollo e introduciendo un marco idóneo para la futura automatización de pruebas.

1.4 Estructura del documento

La presente memoria está organizada en siete capítulos y dos apéndices, estructurados de manera secuencial para guiar al lector desde la concepción teórica y metodológica del proyecto hasta su implementación técnica y validación empírica. El contenido de los mismos se distribuye de la siguiente manera:

En el **Capítulo 1** (el actual) se establece el contexto general del proyecto, detallando la motivación que origina el trabajo, los objetivos trazados para su consecución y el conjunto de tecnologías seleccionadas para el desarrollo.

En el **Capítulo 2** se describe la metodología de trabajo adoptada, desglosando el ecosistema de desarrollo, las fases del ciclo de vida del software (investigación, planificación e implementación técnica) y el modelo de licenciamiento del código fuente.

En el **Capítulo 3** se asienta el marco teórico del proyecto. En él se exponen los fundamentos de la optimización matemática, se contrastan los métodos exactos frente a los aproximados, y se profundiza en la taxonomía de las metaheurísticas (basadas en trayectoria y en población), concluyendo con una introducción a la optimización multiobjetivo y al algoritmo NSGA-II.

En el **Capítulo 4** se aborda la especificación y el diseño de la biblioteca. Se presenta la arquitectura general del sistema y se detalla el modelado de los distintos módulos que conforman su núcleo (algoritmos, operadores, problemas, estructuras de soluciones, observadores y utilidades).

En el **Capítulo 5** se profundiza en los detalles de la implementación técnica en el lenguaje Rust.

Se explica cómo se han resuelto los desafíos arquitectónicos mediante abstracciones de alto nivel, el funcionamiento interno de los motores algorítmicos y el sistema de recolección de métricas. Asimismo, se documenta la batería de pruebas unitarias y de integración desarrolladas para asegurar la robustez del código, junto con la validación experimental.

En el **Capítulo 6** se expone el núcleo analítico del trabajo: la evaluación comparativa y el análisis de rendimiento. A través de un riguroso banco de pruebas estandarizado (Rastrigin, TSP, Knapsack, ZDT1 y Ackley), se contrasta el rendimiento computacional y la calidad de convergencia de la biblioteca frente a herramientas líderes de la industria (jMetal, pagmo2, DEAP, entre otras), respaldando los resultados mediante contrastes de significancia estadística.

En el **Capítulo 7** se exponen las conclusiones finales extraídas tras el desarrollo y la experimentación. Se reflexiona sobre el cambio de paradigma que supone la adopción de Rust, se identifican las limitaciones de la versión actual y se proponen líneas de trabajo futuras para la evolución del proyecto.

En el **Apéndice A** contiene el manual de usuario, proporcionando los requisitos previos y las instrucciones necesarias para la instalación y compilación de la biblioteca desde su repositorio oficial.

Finalmente, el **Apéndice B** incluye un manual que explica cómo configurar un problema de optimización metaheurístico sencillo, con el objetivo de facilitar la inicialización y el uso de la biblioteca.

2

Metodología de trabajo

En este capítulo se describen los procedimientos, herramientas y fases seguidas para la consecución de los objetivos del proyecto.

2.1 Modelo y ecosistema de desarrollo

Para el desarrollo del proyecto se ha adoptado una metodología iterativa, inspirada en el marco de trabajo SCRUM, pero adaptada a las particularidades de un entorno de trabajo unipersonal. Esta adaptación permite conservar las ventajas de la planificación incremental y la revisión continua, ajustándolas a la naturaleza y alcance del proyecto.

Por otra parte, las herramientas de desarrollo empleadas en este proyecto corresponden a las descritas conceptualmente en la **Sección 1.3**. La selección de este ecosistema técnico se orientó a maximizar la eficiencia en la detección de errores y la navegación por el código fuente, consolidando a Visual Studio Code como el entorno de desarrollo principal debido a su versatilidad y al robusto soporte proporcionado por la extensión *rust-analyzer* [3].

2.2 Fases del proyecto

Un proyecto como en el que nos encontramos debe estructurarse por partes, en este caso en tres etapas diferenciadas:

2.2.1 Investigación y capacitación

Debido a la pronunciada curva de aprendizaje de Rust, esta fase inicial se centró en la asimilación de sus fundamentos sintácticos y las particularidades de su compilador. Para consolidar estos conceptos, se llevó a cabo la migración de un proyecto personal previo (una aplicación de escritorio), trasladando su arquitectura basada en Java y Javalin hacia un ecosistema nativo en Rust mediante el framework Tauri [37].

Paralelamente, se desarrolló un repositorio de pruebas a modo de entorno de experimentación, destinado a la implementación de pequeños módulos funcionales que permitieron dominar aún más el lenguaje. Esta etapa de formación técnica se complementó con un estudio exhaustivo sobre metaheurísticas, especialmente, sobre algoritmos genéticos, sentando las bases teóricas necesarias para el posterior diseño de la biblioteca.

Dentro del estudio técnico, cobró especial relevancia el análisis del repositorio y de la arquitectura de jMetal [20], un framework de optimización metaheurística multi-objetivo escrita en Java por investigadores de la Universidad de Málaga, ayudó a comprender los patrones de diseño y las estructuras de datos necesarias para implementar metaheurísticas de forma extensible.

2.2.2 Definición de requisitos y planificación

Durante esta etapa se realizó un trabajo de introspección, se analizaron los objetivos y se elaboró un documento donde se especificaron todos los requisitos del sistema.

A partir de estos requisitos, se creó un *backlog* en Jira, donde se registraron todas las tareas necesarias para completar el proyecto. Posteriormente, las tareas se organizaron en seis *sprints* y se clasificaron en las columnas «Por hacer», «En curso», «Por revisar» y «Listo». Esta organización permitió un seguimiento claro del progreso y facilitó la comunicación con el tutor, mostrando periódicamente las tareas en la columna «Por revisar» para recibir su aprobación. Una vez que el tutor daba el visto bueno, las tareas se marcaban como «Listo», asegurando así un flujo de trabajo controlado y transparente [2]. En la Figura 2.1 se ilustra una captura de pantalla del estado del tablero de gestión durante el ciclo de vida del desarrollo, reflejando la distribución real de las tareas en sus respectivas fases.

2.2.3 Implementación técnica

Una vez definidos los requisitos del sistema y establecida la planificación del proyecto, se procedió a la implementación técnica de la biblioteca de optimización metaheurística. Esta fase tuvo como objetivo materializar las decisiones de diseño adoptadas previamente, dando lugar a un sistema funcional, modular y extensible, desarrollado íntegramente en el lenguaje Rust.

El proceso de implementación se llevó a cabo de forma incremental, permitiendo validar progresivamente las abstracciones definidas y adaptar la arquitectura a las particularidades del lenguaje y a los requisitos detectados durante el desarrollo.

La implementación estuvo fuertemente condicionada por las características propias del lenguaje Rust. La ausencia de herencia clásica y el uso intensivo de *traits* influyeron directamente en el diseño de las abstracciones, fomentando un enfoque basado en la composición y el polimorfismo estático.

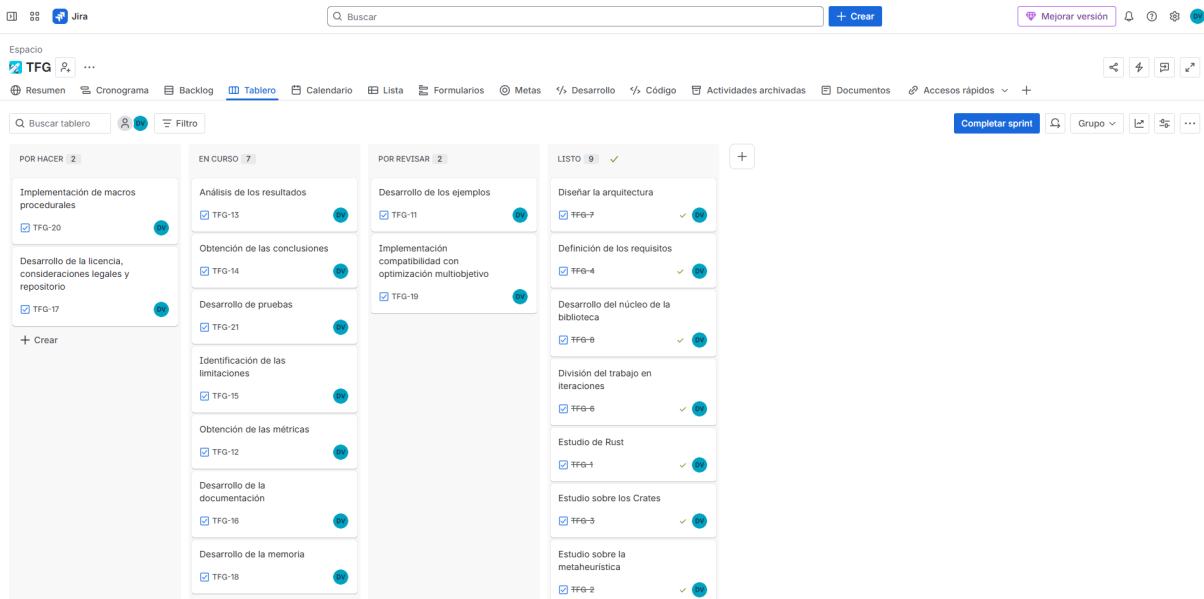


Figura 2.1. Captura de pantalla del estado del tablero Kanban en Jira durante la ejecución del proyecto.

Asimismo, el sistema de propiedad y préstamos de Rust actuó como una herramienta de validación temprana, obligando a definir de forma explícita la gestión de memoria y el ciclo de vida de las estructuras de datos. Aunque este modelo supuso una dificultad inicial, permitió garantizar un alto nivel de seguridad y robustez en la implementación final.

La implementación de la biblioteca se realizó siguiendo un orden lógico que permitió construir el sistema de forma progresiva. En primer lugar, se desarrollaron las abstracciones básicas, tales como las representaciones de soluciones y problemas. Posteriormente, se implementaron los algoritmos de optimización, seguidos de los operadores fundamentales y de los mecanismos de observación.

Este enfoque permitió validar cada componente de manera aislada antes de integrarlo en el sistema global, reduciendo la complejidad del proceso de depuración y facilitando la detección de errores conceptuales en fases tempranas del desarrollo.

Durante la fase de implementación se llevó a cabo una validación continua del sistema, apoyándose principalmente en el compilador de Rust como herramienta de detección de errores y en la ejecución de pruebas y ejemplos controlados para verificar el comportamiento de los algoritmos.

Asimismo, se realizaron refactorizaciones periódicas del código con el objetivo de mejorar su claridad, reducir duplicidades y adaptar la arquitectura a nuevas necesidades detectadas durante el desarrollo. Este proceso iterativo permitió alcanzar una implementación estable y coherente con los objetivos iniciales del proyecto.

2.3 Licencia de software

Con el propósito de fomentar la transferencia tecnológica, garantizar la transparencia del código y contribuir activamente al ecosistema de software libre y de código abierto (*open-source*), este proyecto se distribuye bajo los términos de la **Licencia MIT** [27]. La elección de este marco legal responde a una decisión estratégica alineada con las prácticas habituales del entorno de desarrollo en Rust, donde se prioriza la máxima reutilización del software con las menores restricciones posibles.

La Licencia MIT se caracteriza por ser una licencia de tipo permisivo. A diferencia de las licencias con *copyleft* fuerte (como la familia GPL), el modelo MIT concede a cualquier persona que obtenga una copia del software el derecho perpetuo, gratuito e irrevocable de utilizar, modificar, fusionar, publicar, distribuir, sublicenciar y comercializar la biblioteca sin necesidad de liberar las modificaciones bajo las mismas condiciones. Las únicas obligaciones impuestas por este marco legal se reducen a dos condiciones fundamentales:

- **Preservación del aviso de copyright:** Cualquier distribución del software, ya sea en su forma original o modificada, debe incluir obligatoriamente el aviso de derechos de autor y el texto de la licencia original.
- **Exención de responsabilidad:** Se estipula explícitamente que el software se proporciona "tal cual", eximiendo a los autores de cualquier reclamación, daño o responsabilidad legal derivada de su uso, ejecución o mal funcionamiento.

Esta permisividad resulta fundamental, ya que elimina barreras legales y reduce la fricción burocrática asociada a su adopción. Gracias a ello, tanto investigadores del ámbito académico como desarrolladores del sector industrial pueden integrar, auditar y adaptar el motor de optimización en sistemas propietarios o comerciales sin comprometer su propiedad intelectual. Esto favorece la transferencia tecnológica, amplía el alcance de la solución y refuerza la sostenibilidad e impacto del proyecto a largo plazo.

3

Optimización y Metaheurísticas

En numerosas disciplinas de la ingeniería, la economía y las ciencias de la computación, surge de manera recurrente la necesidad de tomar decisiones que maximicen el beneficio o minimicen el coste de un sistema dado. Este proceso de búsqueda de la mejor alternativa posible, dentro de un conjunto de soluciones factibles, es lo que se conoce formalmente como optimización. En este capítulo se introducen los conceptos fundamentales de la optimización matemática, las limitaciones de los métodos clásicos y el papel crucial que desempeñan las metaheurísticas en la resolución de problemas complejos.

3.1 El Problema de Optimización

De forma general, la optimización matemática trata de encontrar la mejor solución posible, evaluada según un criterio cuantitativo, de entre un conjunto de alternativas disponibles. Formalmente, un problema de optimización mono-objetivo se formula como la búsqueda de un vector de variables de decisión $x = (x_1, x_2, \dots, x_n)$ que optimice (ya sea minimizando o maximizando) una función objetivo escalar $f(x)$. Este problema se define matemáticamente de la siguiente manera:

Minimizar $f(x)$

sujeto a $x \in \Omega$

Sin pérdida de generalidad, en la literatura algorítmica suele trabajarse exclusivamente con problemas de minimización, dado que cualquier problema de maximización puede transformarse trivialmente multiplicando la función objetivo por -1 (es decir, $\max f(x) \equiv \min -f(x)$).

El vector x representa las incógnitas del sistema sobre las cuales el proceso de resolución tiene control directo. La función objetivo $f : \Omega \rightarrow \mathbb{R}$ es la métrica que asigna un valor de calidad o coste a cada vector propuesto.

El conjunto Ω denota el espacio de búsqueda o la región factible del problema. Lejos de ser un espacio infinito o arbitrario, en los escenarios de ingeniería y del mundo real, este espacio está delimitado geométrica y matemáticamente por un conjunto de restricciones que el vector x debe cumplir de manera estricta. Estas restricciones imponen límites físicos, lógicos o económicos al sistema, y suelen expresarse mediante funciones de desigualdad ($g_i(x) \leq 0, i = 1, \dots, m$) y de igualdad ($h_j(x) = 0, j = 1, \dots, p$).

Por consiguiente, Ω está constituido única y exclusivamente por aquellos vectores que satisfacen simultáneamente todas estas condiciones. Si un algoritmo de optimización genera una solución que recae fuera de las fronteras de Ω , dicha solución se considera infactible. En el diseño de algoritmos y metaheurísticas, las soluciones infactibles suponen un desafío; por lo general, son descartadas inmediatamente, aunque se les puede aplicar de forma más sofisticada una penalización severa en su valor de $f(x)$ para guiar la búsqueda iterativa de vuelta hacia el interior de la región factible o en tercer lugar, se puede intentar su reparación mediante un proceso normalmente determinista pero subóptimo que las transforma en soluciones factibles.

3.2 Métodos Exactos frente a Métodos Aproximados

Para abordar la resolución de los problemas matemáticos de optimización, existen tradicionalmente dos grandes familias de algoritmos: los métodos exactos y los métodos aproximados. La elección entre un enfoque u otro depende intrínsecamente de la naturaleza del problema, el tamaño de la instancia a resolver y los recursos computacionales disponibles.

Los métodos exactos exploran el espacio de búsqueda de una manera sistemática y exhaustiva, de tal forma que garantizan matemáticamente encontrar la solución óptima global del problema. Entre las técnicas más destacadas de esta familia se encuentran la Programación Lineal (como el algoritmo Simplex), la Programación Dinámica y los algoritmos de Ramificación y Acotación (*Branch and Bound*). Cuando el problema presenta un modelo matemático tratable y un tamaño de entrada moderado, estos métodos son, sin lugar a duda, la opción preferida debido a la certeza de sus resultados.

Sin embargo, su aplicabilidad se ve severamente limitada en la práctica industrial y de investigación. Muchos de los problemas de mayor interés y aplicabilidad en el mundo real pertenecen a las clases de complejidad NP-completos o NP-duros. Un ejemplo paradigmático es el Problema del Viajante de Comercio (TSP, por sus siglas en inglés) [32], donde se busca la ruta más corta que visite un conjunto de

ciudades. En estos escenarios, el espacio de soluciones posibles crece de forma factorial o exponencial con respecto al tamaño del problema ($O(n!)$ o $O(2^n)$). Este fenómeno, conocido como "explosión combinatoria" o "la maldición de la dimensionalidad", provoca que un algoritmo exacto requiera tiempos de ejecución inasumibles (años o incluso siglos de cómputo) para instancias de tamaño realista [23].

Es precisamente ante esta intratabilidad computacional donde los métodos aproximados, concretamente los algoritmos heurísticos, cobran un protagonismo absoluto y se convierten en la única alternativa viable. La principal motivación de un heurístico es encontrar una solución en un tiempo razonable, especialmente para problemas NP-completos o de gran escala, donde un algoritmo exacto tardaría demasiado o sería inviable. Estos algoritmos logran esta eficiencia sacrificando la garantía de encontrar la solución óptima o perfecta.

A diferencia de los enfoques exactos, la naturaleza de los métodos aproximados es inherentemente incompleta, ya que no exploran todo el espacio de soluciones posibles. En su lugar, producen soluciones que son buenas o cercanas al óptimo (también conocidas como soluciones cuasi-óptimas), pero no tienen pruebas matemáticas que aseguren que la solución encontrada sea la mejor de todas.

3.3 Heurísticas y Metaheurísticas

El término *heurística* (del griego *heuriskein*, "encontrar" o "descubrir") hace referencia a estrategias que utilizan conocimiento específico del dominio del problema o "reglas prácticas" para guiar la búsqueda hacia soluciones de alta calidad. Si bien producen soluciones cuasi-óptimas de forma rápida, a menudo corren el riesgo de quedar atrapadas en óptimos locales (soluciones que son las mejores en su vecindario inmediato, pero no en todo el espacio de búsqueda).

Para superar esta limitación, nacen las **metaheurísticas**. Una metaheurística se define como un marco algorítmico de nivel superior, independiente del problema subyacente, que provee un conjunto de directrices para guiar a otras heurísticas subordinadas en la exploración del espacio de soluciones.

El éxito de cualquier metaheurística reside en mantener un delicado equilibrio dinámico entre dos conceptos fundamentales:

- **Exploración (o diversificación):** La capacidad del algoritmo para investigar regiones globales y desconocidas del espacio de búsqueda, evitando la convergencia prematura.
- **Explotación (o intensificación):** La capacidad de concentrar la búsqueda en las vecindades de las mejores soluciones encontradas hasta el momento, con el fin de refinar y mejorar dichos resultados [5].

3.4 Clasificación de las Metaheurísticas

A lo largo de las últimas décadas, la literatura científica ha propuesto numerosas metaheurísticas, muchas de ellas inspiradas en procesos naturales, biológicos o físicos. La clasificación más extendida en la comunidad académica divide estos algoritmos según el número de soluciones con las que trabajan simultáneamente en cada iteración:

3.4.1 Metaheurísticas basadas en trayectoria

También conocidas como métodos de búsqueda puntual o de solución única, estos algoritmos mantienen y modifican una única solución candidata a lo largo del tiempo, describiendo una trayectoria secuencial a través del espacio de búsqueda. Matemáticamente, el proceso se define mediante una función de vecindad $\mathcal{N}(x)$ que mapea una solución actual x_t hacia un conjunto de soluciones adyacentes. En cada iteración, el algoritmo selecciona una candidata $x' \in \mathcal{N}(x_t)$ y la evalúa; si se cumple un determinado criterio de aceptación, la transición se consolida ($x_{t+1} = x'$). Aunque esta dinámica intrínseca posee un alto poder de explotación para descender rápidamente hacia el fondo de un valle topológico, sufre un riesgo crítico de convergencia prematura al quedar atrapada en óptimos locales. Para evadir este estancamiento, la literatura ha desarrollado metodologías consolidadas que alteran el criterio de aceptación o la propia topología del vecindario.

Uno de los enfoques probabilísticos más célebres es el Recocido Simulado (*Simulated Annealing*, SA) [24], inspirado en la termodinámica del enfriamiento metalúrgico. En lugar de aplicar un descenso estricto, el algoritmo permite transiciones hacia soluciones de peor calidad evaluando la diferencia de aptitud (Δf) a través de la distribución de Boltzmann. La probabilidad de aceptar una degradación se define como $P \approx e^{-\Delta f/T}$, donde T es un parámetro de temperatura que decrece asintóticamente con el tiempo, permitiendo gran exploración en las fases iniciales y forzando la explotación en las etapas finales. Por el contrario, la Búsqueda Tabú (*Tabu Search*) [16] opta por una estrategia determinista basada en memoria restrictiva. Para forzar la salida de una cuenca de atracción, el algoritmo registra las últimas transiciones en una estructura de memoria a corto plazo (*lista tabú*), redefiniendo matemáticamente el vecindario explorable como $\mathcal{N}(x_t) \setminus Tabu$. De este modo, se prohíbe temporalmente el retorno a estados recientemente visitados, evitando la formación de ciclos infinitos.

Otras estrategias de trayectoria centran su formulación matemática en la alteración directa del espacio de búsqueda o de la posición de la solución. La Búsqueda Local Iterada (*Iterated Local Search*, ILS) [26] asume que el gradiente local ya ha sido agotado y aplica una función de perturbación estocástica a la solución actual para realizar un "salto" forzoso hacia otra región del hipervolumen,

seguido inmediatamente de una nueva fase de descenso determinista. Finalmente, la Búsqueda en Vecindario Variable (*Variable Neighborhood Search, VNS*) [17] se fundamenta en el principio matemático de que un óptimo local definido bajo una métrica de distancia concreta no tiene por qué serlo bajo otra. Este método define un conjunto finito de estructuras de vecindad $\{\mathcal{N}_1, \mathcal{N}_2, \dots, \mathcal{N}_{k_{max}}\}$ y transita sistemáticamente entre ellas; cuando la búsqueda se estanca en $\mathcal{N}_k$, el algoritmo cambia la topología evaluando $\mathcal{N}_{k+1}$, expandiendo progresivamente el radio de exploración hasta hallar una cuenca de atracción que contenga un óptimo de mayor calidad.

3.4.2 Metaheurísticas basadas en población

A diferencia de los métodos de trayectoria, estas técnicas mantienen en memoria un conjunto (o población) de soluciones que evolucionan de manera simultánea en cada iteración. Esta característica les confiere una mayor robustez y una capacidad de exploración superior, ya que las soluciones pueden interactuar e intercambiar información entre sí. Dentro de este paradigma destacan los Algoritmos Evolutivos (como los Algoritmos Genéticos) y la Optimización por Enjambre de Partículas *Particle Swarm Optimization (PSO)*.

Los Algoritmos Evolutivos (AE), fuertemente fundamentados en la teoría de la evolución biológica y la genética de poblaciones, abordan la búsqueda iterando sobre un conjunto de soluciones codificadas como individuos o cromosomas. El progreso hacia la región factible y óptima del espacio de búsqueda se dirige mediante tres mecanismos fundamentales aplicados de forma secuencial. Inicialmente, un mecanismo de selección evalúa la función objetivo de la población y discrimina probabilísticamente a los individuos más aptos, asegurando la supervivencia del mejor material genético. A continuación, el operador de cruce fomenta la explotación del espacio mediante el intercambio de segmentos del vector de variables entre pares de individuos seleccionados, generando una descendencia que hereda características de sus progenitores. Finalmente, un operador de mutación introduce perturbaciones de carácter aleatorio en la configuración genética de la descendencia; esta inyección de variabilidad constituye la principal herramienta de exploración del algoritmo, mitigando el riesgo de convergencia prematura en mínimos locales. Tras la aplicación de estos operadores, se ejecuta una política de reemplazo que define la transición hacia la siguiente generación, incorporando frecuentemente mecanismos de elitismo para retener incondicionalmente la mejor solución descubierta.

Por su parte, la Optimización por Enjambre de Partículas (PSO) modela una dinámica sustancialmente distinta, inspirada en el comportamiento colectivo exhibido por agregaciones biológicas, tales como el vuelo sincronizado de bandadas de aves o el nado de bancos de peces. En este modelo, la

población se conceptualiza como un enjambre y las soluciones candidatas actúan como partículas que se desplazan de manera continua y autónoma a lo largo del hiperespacio de búsqueda de dimensión n . A diferencia de los modelos evolutivos, en PSO no existe la destrucción ni creación de nuevos individuos, sino la perturbación guiada de sus trayectorias.

Cada partícula se encuentra definida unívocamente en una iteración t por su posición o vector de variables $x_i(t)$ y por su vector de velocidad $v_i(t)$. La cinemática del enjambre se articula mediante la actualización iterativa de estas magnitudes vectoriales. La nueva velocidad de una partícula se calcula como una combinación lineal de tres factores principales: un factor de inercia que perpetúa el momento de la trayectoria previa fomentando la exploración global; un componente cognitivo que dirige a la partícula hacia la mejor posición histórica que ella misma ha experimentado, denominada $pbest$; y un componente social que atrae a la partícula hacia el óptimo global descubierto por la totalidad del enjambre hasta ese momento, identificado como $gbest$.

Matemáticamente, la modificación del vector velocidad se expresa de la forma:

$$v_i(t+1) = w \cdot v_i(t) + c_1 \cdot r_1 \cdot (pbest_i - x_i(t)) + c_2 \cdot r_2 \cdot (gbest - x_i(t)) \quad (3.1)$$

Donde w denota el coeficiente de inercia, c_1 y c_2 representan las constantes de aceleración cognitiva y social respectivamente, y r_1, r_2 son variables aleatorias extraídas de una distribución uniforme en el intervalo $[0, 1]$ que aportan el necesario grado de estocasticidad al sistema. Inmediatamente después, el vector de posición se actualiza integrando algebraicamente la nueva velocidad:

$$x_i(t+1) = x_i(t) + v_i(t+1) \quad (3.2)$$

Este esquema de actualización garantiza un equilibrio dinámico entre la autonomía exploratoria de cada solución y la convergencia acelerada inducida por el conocimiento topológico compartido por toda la red de la población.

3.5 Extensión a la Optimización Multi-Objetivo

En gran parte de las aplicaciones del mundo real, los problemas no presentan un único objetivo a optimizar, sino múltiples objetivos que a menudo se encuentran en conflicto directo (por ejemplo, maximizar el rendimiento de un componente a la par que se minimiza su coste de fabricación).

En estos escenarios, denominados Problemas de Optimización Multi-objetivo (MOP), el concepto de «solución óptima» desaparece para dar paso al concepto de *Dominancia de Pareto*. El objetivo de las metaheurísticas multi-objetivo no es encontrar un único punto, sino aproximar el Frente de Pareto: un conjunto de soluciones no dominadas que representan los mejores compromisos posibles entre los

diferentes objetivos evaluados, permitiendo finalmente al tomador de decisiones elegir la alternativa que mejor se adapte a sus necesidades.

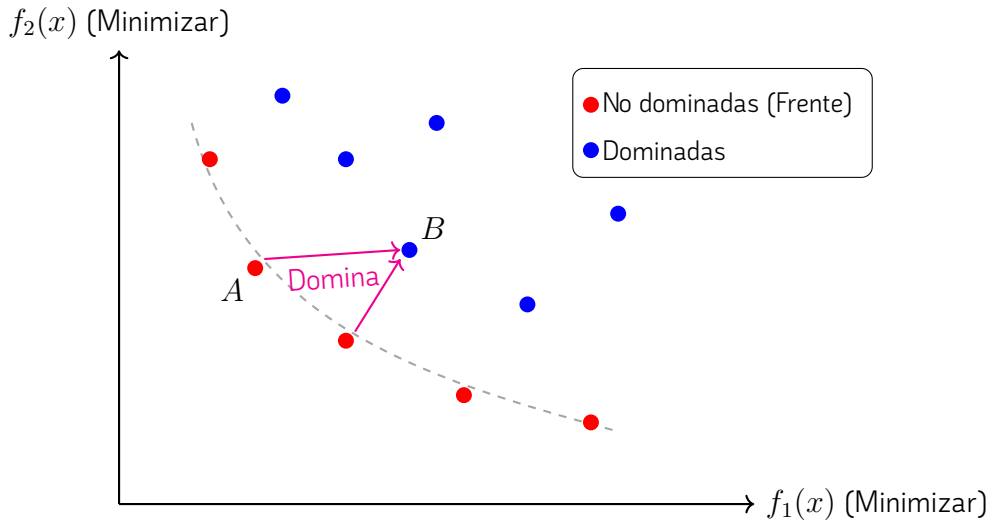


Figura 3.1. Representación en el espacio de objetivos de un problema con dos funciones a minimizar (f_1 y f_2). La solución A domina a la solución B , ya que es estrictamente mejor en f_1 e igual o mejor en f_2 .

3.5.1 Formulación Matemática del Problema y Dominancia

Matemáticamente, un Problema de Optimización Multi-objetivo (MOP) sin restricciones se define como la minimización simultánea de un vector de M funciones objetivo, expresado de la forma $\min \vec{f}(\vec{x}) = [f_1(\vec{x}), f_2(\vec{x}), \dots, f_M(\vec{x})]^T$, donde $\vec{x} \in \Omega$ es el vector de variables de decisión dentro del espacio de búsqueda factible. En este contexto analítico, se establece que una solución $\vec{x}_1$ domina a otra solución $\vec{x}_2$, lo cual se denota formalmente como $\vec{x}_1 \prec \vec{x}_2$, si y solo si $\vec{x}_1$ no es peor que $\vec{x}_2$ en ninguno de los objetivos, es decir, $\forall i \in \{1, \dots, M\}, f_i(\vec{x}_1) \leq f_i(\vec{x}_2)$, y además es estrictamente superior en al menos un objetivo, lo que implica que $\exists j \in \{1, \dots, M\} \mid f_j(\vec{x}_1) < f_j(\vec{x}_2)$.

3.5.2 Algoritmo NSGA-II (Non-dominated Sorting Genetic Algorithm II)

Para resolver este tipo de escenarios, uno de los algoritmos más extendidos e influyentes en la literatura científica es el NSGA-II, propuesto por K. Deb et al [7]. Este algoritmo genético basa su eficacia en dos pilares matemáticos fundamentales que guían la evolución de la población: un mecanismo de ordenación rápida no dominada (Fast Non-Dominated Sort) y un estimador de la densidad del entorno denominado distancia de apilamiento (Crowding Distance).

En primer lugar, la población combinada R_t de tamaño $2N$ (formada por la unión de la población

de padres P_t y su descendencia Q_t) se evalúa y clasifica en diferentes frentes de Pareto $\mathcal{F} = \{F_1, F_2, \dots, F_k\}$. El primer frente F_1 contiene exclusivamente aquellas soluciones que no son dominadas por ninguna otra dentro del conjunto R_t . El frente F_2 contiene a las soluciones dominadas únicamente por los individuos presentes en F_1 , y el proceso continúa iterativamente. A cada solución i se le asigna un rango o nivel de no dominancia r_i equivalente al índice de su frente, de modo que un rango menor indica una calidad global superior de la solución.

En segundo lugar, para garantizar la diversidad algorítmica y promover una distribución uniforme de los individuos a lo largo del Frente de Pareto, se calcula la distancia de apilamiento d_i para cada individuo exclusivamente respecto a las soluciones de su mismo frente. Para un objetivo m específico, se ordenan los individuos del frente en función de su valor en dicho objetivo. A las soluciones límite (aquellas que determinan los extremos del frente) se les asigna una distancia matemática infinita ($d_1 = d_l = \infty$), garantizando su preservación. Para el resto de soluciones intermedias i , la distancia se computa acumulando las diferencias normalizadas entre sus vecinos topológicos adyacentes:

$$d_i = \sum_{m=1}^M \frac{f_m^{(i+1)} - f_m^{(i-1)}}{f_m^{\max} - f_m^{\min}} \quad (3.3)$$

donde $f_m^{(i+1)}$ y $f_m^{(i-1)}$ son los valores del objetivo m para los individuos vecinos de la solución i , mientras que $f_m^{\max}$ y $f_m^{\min}$ representan los valores máximo y mínimo empíricos registrados para ese objetivo concreto dentro del frente en evaluación.

Finalmente, la convergencia del método se garantiza utilizando el operador de comparación atestada o *crowded-comparison operator* ($\prec_n$) para seleccionar determinísticamente a los N mejores individuos que conformarán la población de la siguiente generación P_{t+1} . Dadas dos soluciones i y j , el algoritmo establece que $i \prec_n j$ (y por tanto i prevalece) si el rango de dominancia de i es estrictamente mejor que el de j ($r_i < r_j$), o bien, en caso de empate al pertenecer ambos al mismo frente ($r_i = r_j$), si la solución i reside en una región del espacio de objetivos menos poblada que j , maximizando su distancia de apilamiento ($d_i > d_j$).

4

Especificación y diseño

4.1 Arquitectura

El diseño y modelado de la arquitectura representan la fase de mayor complejidad y dedicación en el desarrollo. La integridad de esta estructura determina la calidad técnica, la eficiencia y la mantenibilidad de la biblioteca. Con el objetivo de garantizar un nivel de abstracción óptimo que permita su aplicación en diversos casos de uso, se han integrado los siguientes criterios de diseño:

La arquitectura desacopla la definición del problema de la lógica del optimizador. Los límites del espacio de búsqueda y las restricciones se encapsulan en una estructura específica. Esta entidad es referenciada por el motor de ejecución, permitiendo que el algoritmo identifique el dominio de las variables. De este modo, el algoritmo puede realizar las evaluaciones e iteraciones pertinentes respetando estrictamente las directrices del sistema a optimizar.

Los algoritmos se han diseñado de forma modular, fundamentándose en el uso de operadores genéricos que se configuran durante la fase de instanciación. Estos operadores son los responsables de la transformación y evolución de las soluciones. Se distinguen cinco categorías principales, operadores de mutación, de cruce, de vecindad, de selección y de memoria.

Todas las implementaciones de los algoritmos tienen varias estructuras asociadas. Estas son los **parámetros**, que definen el comportamiento del algoritmo, el **estado de ejecución**, que permite la trazabilidad de cada paso, dado que el sistema emplea un modelo de ejecución secuencial por etapas y luego el **conjunto de soluciones**, que representa el resultado final.

Un componente diferenciador es su sistema de experimentación integrado. A partir de la parametrización de los algoritmos, la biblioteca facilita la ejecución comparativa de múltiples configuraciones sobre un mismo problema. Este módulo automatiza la obtención de métricas, tales como tiempos de ejecución y calidad de la solución obtenida, facilitando así la elección del algoritmo más adecuado para

solventar un problema específico.

El estado de ejecución actúa como el mecanismo principal para actualizar el contexto del algoritmo, permitiendo verificar de forma continua si se han cumplido los criterios de terminación definidos en la parametrización inicial. Este contexto no solo regula el flujo de control del proceso, sino que sirve como intermediario crítico para la comunicación con los observadores del sistema. Mediante el uso de hilos de ejecución independientes, el contexto transmite los datos de la evolución del algoritmo en tiempo real, facilitando tanto la monitorización por parte del usuario como el almacenamiento persistente de métricas para la generación posterior de reportes de rendimiento.

4.2 Módulos

A partir de la arquitectura mostrada en la Figura 4.1, se definen los distintos módulos que componen la biblioteca, junto con las responsabilidades asociadas a cada uno de ellos.

4.2.1 Algoritmos

Este módulo es el encargado de ejecutar los distintos problemas definidos en la biblioteca. Este contiene toda la lógica sobre la ejecución, los puntos de control, la comunicación con los observadores y el paralelismo.

Su diseño sigue una separación clara entre las interfaces (traits), la infraestructura de ejecución y los distintos algoritmos concretos, facilitando la extensibilidad, la reutilización y la mantenibilidad del código.

4.2.1.1 Contenido y organización

- **Interfaces y contratos:** componente que declara las abstracciones y traits que describen el comportamiento esperado de los algoritmos y de sus elementos auxiliares. Estas interfaces permiten sustituir implementaciones concretas sin afectar al resto del sistema.
- **Infraestructura de ejecución:** submódulos encargados de la gestión del ciclo de vida de un experimento/ejecución, incluyendo el *runtime*, la terminación y mecanismos de ejecución asíncrona o en paralelo. Se agrupan aquí las piezas que controlan cuándo y cómo se ejecutan los algoritmos sobre instancias de problema.
- **Persistencia y recuperación:** componentes destinados a checkpointing y mecanismos relacionados que facilitan la recuperación de ejecuciones largas o interrumpidas, y que permiten auditoría y análisis posteriores.

- **Definición de objetivos:** submódulo que centraliza la representación de funciones objetivo y métricas relacionadas, separado de las implementaciones concretas para mantener una clara distinción entre problema y algoritmo.
- **Implementaciones concretas:** paquete separado que contiene las implementaciones específicas de algoritmos de optimización basados en metaheurísticas, incluyendo enfoques evolutivos y poblacionales. Agruparlas en un submódulo permite añadir, probar y comparar algoritmos sin alterar la infraestructura común.
- **Terminación y políticas:** lógica que alberga criterios de parada y políticas asociadas (por tiempo, por número de iteraciones, por convergencia, etc.), definida aparte para facilitar la composición y reutilización.

4.2.2 Experimentos

El módulo de experimentos constituye la capa de abstracción superior de la biblioteca, diseñada para facilitar la evaluación sistemática y comparativa de los diversos algoritmos frente a un problema específico. Su propósito fundamental es permitir que el usuario final realice un proceso de *benchmarking* robusto, identificando qué configuraciones ofrecen un rendimiento óptimo mediante la ejecución controlada y automatizada de baterías de pruebas.

Este servicio se articula a través de la definición de casos de prueba, los cuales representan combinaciones específicas de parámetros y componentes. Gracias a la arquitectura modular de *roma*, el sistema permite explorar un amplio espacio de configuraciones, incluyendo la variación de operadores, el ajuste de las probabilidades de mutación y cruce, o la selección de diferentes criterios de terminación. Asimismo, es posible comparar directamente algoritmos distintos bajo las mismas condiciones de contorno, lo que proporciona una base empírica sólida para la toma de decisiones técnicas.

Al finalizar las ejecuciones, el módulo genera un reporte detallado que sintetiza los resultados obtenidos por cada configuración. Este informe no solo recopila las soluciones halladas, sino que ofrece una visión analítica del comportamiento de los algoritmos, permitiendo al usuario discernir con criterio qué estrategia es la más adecuada para la resolución de su problema.

4.2.3 Observadores

El compartimento asociado a los observadores se ha diseñado como un componente desacoplado encargado de gestionar la monitorización durante el proceso evolutivo. Esta infraestructura se articula fundamentalmente a través de la definición de eventos específicos, los cuales son emitidos por el motor

de ejecución para señalar hitos relevantes o cambios en el progreso del algoritmo. Para dotar de contenido a estos eventos, el sistema emplea estructuras de datos especializadas que encapsulan el estado actual de la ejecución, permitiendo que la información fluya hacia los observadores de manera coherente y estructurada.

4.2.4 Operadores

Este componente constituye el núcleo funcional y arquitectónico de la biblioteca de optimización, ya que encapsula la lógica algorítmica fundamental mediante un modelo desacoplado y altamente modular. En lugar de ligar los operadores a algoritmos específicos, la biblioteca implementa un diseño basado en abstracciones y polimorfismo que separa completamente la definición del espacio de búsqueda de los mecanismos de variación y selección. Este modelo permite que cualquier operador sea tratado como una entidad independiente e intercambiable, facilitando la hibridación de técnicas y permitiendo al usuario configurar algoritmos a la carta combinando distintas estrategias.

Más allá de su flexibilidad arquitectónica, la eficiencia computacional de estos operadores es un factor crítico para el éxito de la biblioteca. Debido a la naturaleza iterativa de las metaheurísticas, en cada ciclo del algoritmo se invocan de manera encadenada múltiples operadores que realizan cálculos y modificaciones directas sobre las estructuras de las soluciones.

Dentro de esta arquitectura, los componentes se clasifican en cinco categorías principales de operadores, diseñados para interactuar perfectamente sobre las estructuras de datos de las soluciones, tanto en enfoques poblacionales como de trayectoria:

- **Operadores de mutación:** Introducen variaciones aleatorias en los individuos de la población con el objetivo de mantener la diversidad genética y evitar la convergencia prematura hacia soluciones subóptimas. Estos operadores permiten explorar nuevas regiones del espacio de búsqueda mediante modificaciones controladas de las soluciones existentes.
- **Operadores de selección:** Se encargan de elegir los individuos que participarán en la generación de nuevas soluciones, generalmente en función de su calidad o valor de aptitud. Su función principal es favorecer la propagación de las mejores soluciones encontradas hasta el momento, equilibrando la presión selectiva con la preservación de la diversidad poblacional.
- **Operadores de cruce:** Combinan la información de dos o más individuos seleccionados para generar nuevos descendientes. A través de este proceso se busca explotar las características favorables de las soluciones existentes, recombinándolas de manera que puedan dar lugar a

individuos con un mejor desempeño.

- **Operadores de vecindad:** Definen y exploran el entorno inmediato de una solución actual para generar candidatas adyacentes. Son el motor principal de las metaheurísticas basadas en trayectoria (como la Escalada Simple o el Enfriamiento Simulado), dictando las transiciones permitidas (mediante intercambios, inversiones o perturbaciones locales) para navegar eficientemente por la topología del espacio de búsqueda.
- **Operadores de memoria:** Gestionan el registro histórico del proceso de optimización para guiar las decisiones futuras del algoritmo. Resultan fundamentales para evitar ciclos repetitivos (como la lista tabú en la Búsqueda Tabú), explotar el conocimiento colectivo (como el registro del mejor individuo local y global en la Optimización por Enjambre de Partículas) o mantener un archivo elitista de soluciones no dominadas en problemas multiobjetivo.

4.2.5 Problemas

Este módulo constituye la abstracción que define y modela el dominio del escenario de optimización concreto que se desea resolver. Dentro de la arquitectura de la biblioteca, el componente de problemas actúa como un intermediario o contrato formal que desacopla la lógica genérica de las metaheurísticas de los detalles particulares de cada aplicación. Estas estructuras no solo almacenan los datos de entrada o las instancias del problema, sino también las reglas matemáticas y las restricciones que gobiernan el espacio de búsqueda, guiando de manera activa tanto a los algoritmos poblacionales como a los operadores de variación hacia los objetivos definidos. Para cumplir con esta función de guía y control, el módulo asume de forma integrada la responsabilidad de orquestar la creación de soluciones y su respectiva evaluación.

En primer lugar, el módulo define los mecanismos necesarios para construir configuraciones de partida válidas dentro del espacio de búsqueda, soluciones puramente aleatorias, esenciales para garantizar la diversidad en las primeras etapas de las metaheurísticas. Conectado directamente con este proceso, el componente implementa la lógica matemática de la función u objetivos $f(x)$ que miden el desempeño de cada vector de decisión. El módulo es, por tanto, el responsable de transformar una solución abstracta en un valor numérico cualitativo o en un vector de valores si se trabaja en entornos multi-objetivo, gestionando además el cálculo de las penalizaciones en caso de que la solución generada vulnere alguna de las restricciones de frontera impuestas por el modelo.

Finalmente, al conocer en detalle cómo se manejan y se relacionan internamente los datos del dominio, este componente proporciona la información estructural indispensable para que los operadores

echen mano de las características del problema. Esto permite determinar de manera analítica la vecindad de una solución, facilitando que las perturbaciones y las variaciones locales se realicen con un sentido semántico preciso —como el intercambio de aristas en problemas de rutas o la asignación de tareas en problemas de planificación— en lugar de ejecutar modificaciones ciegas que comprometan la eficiencia del proceso de optimización.

4.2.6 Conjunto de soluciones

Los algoritmos devolverán esta estructura de datos, la cual está diseñada como una estructura abstracta orientada a actuar como una entidad contenedora encargada de encapsular de manera unificada el conjunto de resultados obtenidos tras el proceso de búsqueda. En lugar de retornar una colección nativa o un vector de soluciones en crudo, se opta por este diseño de abstracción desacoplado para centralizar y homogeneizar la información derivada de la ejecución del algoritmo ante el problema planteado. Al adoptar la forma de una interfaz o tipo abstracto, se dota al sistema de una alta flexibilidad, permitiendo que esta estructura pueda ser implementada o extendida de cualquier manera en el futuro para solventar necesidades emergentes del desarrollo sin alterar el núcleo de la biblioteca.

Esta naturaleza abstracta permite albergar internamente desde una lista indexada con las soluciones óptimas o aproximadas que han logrado superar los criterios de convergencia impuestos, hasta estructuras más complejas adaptadas a paradigmas específicos. Al aislar esta lógica dentro de un contrato de datos genérico, se facilita la incorporación dinámica de metadatos adicionales de gran valor para la posterior fase de análisis.

4.2.7 Soluciones

Las soluciones representan las estructuras que almacenan el resultado generado por la aplicación iterativa de los operadores dentro de los algoritmos de optimización. Este componente es de vital importancia para la eficiencia global del sistema, dado que constituye la unidad fundamental con la que opera la biblioteca de manera ininterrumpida a lo largo de todo el ciclo de ejecución. El motor de optimización modifica constantemente estas estructuras, gestiona poblaciones masivas de las mismas, evalúa sus atributos en cada iteración y analiza sus valores para guiar la búsqueda. Por consiguiente, cualquier ineficiencia o sobrecarga en su diseño se amplificaría exponencialmente, comprometiendo críticamente el rendimiento temporal y el consumo de memoria de los algoritmos.

Bajo esta premisa, la estructura se ha diseñado siguiendo una filosofía minimalista que prioriza la ligereza y la simplicidad estructural. Sin embargo, alcanzar este equilibrio ha supuesto un desafío de desarrollo complejo debido a la necesidad de dar soporte simultáneo tanto a problemas de un único

objetivo como a enfoques multi-objetivo. Mientras que en entornos mono-objetivo basta con asociar al vector de variables un único valor escalar de aptitud, los entornos multi-objetivo exigen gestionar vectores de funciones objetivo, rangos de dominancia y métricas de densidad como la distancia de apilamiento.

4.2.8 Utilidades

Para posibilitar el desarrollo de este proyecto y cumplir el objetivo de mantener una arquitectura con *cero dependencias externas*, se han desarrollado componentes auxiliares ligeros integrados en el núcleo de la biblioteca. Estos submódulos reproducen, de forma optimizada y autocontenida, las funcionalidades estrictamente necesarias de `crates` estándar del ecosistema Rust, tales como `rand` [9] para la generación estocástica y `plotters` [19] para la representación gráfica. Esta decisión de diseño permite evitar la inclusión de bibliotecas externas sin renunciar a las capacidades necesarias para el correcto funcionamiento del sistema.

Figura 4.1. Arquitectura de la biblioteca. Diagrama de clases, UML. Visual Paradigm 17.0 [39].

5

Implementación y pruebas

Este capítulo detalla el proceso de construcción de la biblioteca `roma`, trasladando los conceptos teóricos y el diseño arquitectónico descritos en los capítulos anteriores a código real en el lenguaje de programación Rust.

5.1 Implementación

Una vez definida y consolidada la arquitectura de la biblioteca, se inició la fase de construcción modular siguiendo un enfoque ascendente. Contrariamente a lo que se podría suponer inicialmente, situando el foco en los algoritmos o en la definición de los problemas, el desarrollo comenzó por el módulo de las **soluciones**. Esta decisión estratégica se fundamenta en que la estructura que define a la solución es, en realidad, el componente más determinante del sistema, actuando como el eje central sobre el que pivotan todos los demás módulos.

5.1.1 Diseño y abstracción de las Soluciones

La importancia de este módulo radica en que la solución es la entidad que los operadores transforman para explorar el espacio de búsqueda y la que los observadores monitorizan constantemente para extraer métricas de rendimiento. Del mismo modo, los problemas dependen de esta estructura para realizar tanto la generación inicial de individuos como su posterior evaluación, mientras que los algoritmos actúan como gestores encargados de almacenarlas y orquestar su flujo a lo largo de las iteraciones.

Debido a esta interdependencia, un diseño deficiente o una implementación poco optimizada de

esta estructura central comprometería no solo la **extensibilidad** de la biblioteca —al dificultar la integración de nuevos problemas o tipos de datos— sino también su **rendimiento computacional**. Dado que la solución es el objeto con mayor frecuencia de acceso y modificación, cualquier ineficiencia en su desarrollo se convertiría inevitablemente en un cuello de botella durante ejecuciones intensivas. Por ello, garantizar una base técnica sólida en este módulo fue la prioridad absoluta antes de proceder con la implementación del resto de componentes.

En una primera aproximación arquitectónica del diseño, `Solution` se definió como un *trait*¹ que declaraba un conjunto de funciones fundamentales, tales como la obtención del valor de *fitness*, el acceso a las variables de la solución o la recuperación de su representación interna. Este *trait* debía ser implementado por distintos *structs*, cada uno adaptado a las necesidades específicas de cada problema.

Por ejemplo, si el problema requería representar una solución como una *array* binario 5.2, el *struct* correspondiente contendría como atributos un vector binario junto con el valor de calidad asociado a dicha solución. Sin embargo, se observó que esta arquitectura introducía una elevada fragmentación en las implementaciones y dificultaba la generalización de la biblioteca para soportar un mayor número de problemas y algoritmos.

Por este motivo, se optó por una solución más abstracta y flexible. En el diseño actual, `Solution` se define como un *struct* genérico parametrizado por dos tipos: `T` y `Q`. El tipo `T` representa el tipo de dato de las variables de decisión almacenadas en la solución, mientras que `Q` representa el tipo asociado a la evaluación de calidad de dicha solución. De forma predeterminada, este último toma un valor real de alta precisión de tipo `f64`.

Esta implementación se puede apreciar en el Código 5.1.

```
1 #[derive(Clone, Debug)]
2 pub struct Solution<T, Q = f64> {
3     variables: Vec<T>,
4     value: Option<Q>,
5 }
```

Código 5.1. Estructura desarrollada para las Soluciones

Sobre esta estructura, en lugar de recurrir a implementaciones rígidas o acopladas de manera convencional, se optó por diseñar fábricas de inicialización mediante constructores específicos para entornos mono-objetivo y multi-objetivo capaces de adaptarse a una amplia variedad de problemas.

¹En el ecosistema de Rust, un *trait* establece un contrato de comportamiento estricto, actuando de manera análoga a las interfaces en lenguajes orientados a objetos como Java. Esta abstracción permite desacoplar la definición de los métodos de su implementación concreta.

Este enfoque modular permite instanciar estados con una sintaxis fluida y desacoplada del núcleo del algoritmo. Un caso representativo de esta estrategia se ilustra en el Código 5.2, donde se muestra el código reducido para la construcción de soluciones binarias (cuyas variables se definen mediante valores booleanos). En este fragmento de código se aprecia cómo el patrón *Builder* facilita la inicialización homogénea de la estructura —ya sea de forma aleatoria estocástica, parametrizada a ceros o a unos— antes de consolidar el objeto de tipo `Solution<bool>` definitivo a través del método de finalización.

```
1 pub struct BinarySolutionBuilder {
2     variables: Vec<bool>,
3     quality: Option<f64>,
4 }
5
6 impl BinarySolutionBuilder {
7     pub fn ones(size: usize) -> Self {
8         Self {
9             variables: vec![true; size],
10            quality: None,
11        }
12    }
13
14    pub fn zeros(size: usize) -> Self {
15        Self {
16            variables: vec![false; size],
17            quality: None,
18        }
19    }
20
21    pub fn random(size: usize, seed: Option<u64>) -> Self {
22        let mut rng = if let Some(seed) = seed {
23            Random::new(seed)
24        } else {
25            Random::new(seed_from_time())
26        };
27        let variables: Vec<bool> = (0..size)
28            .map(|_| rng.coin_flip())
29            .collect();
30
31        Self {
32            variables,
```

```

33     quality: None,
34 }
35 }
36
37 pub fn with_quality(mut self, quality: f64) -> Self {
38     self.quality = Some(quality);
39     self
40 }
41
42
43 pub fn build(self) -> Solution<bool> {
44     finalize_scalar_solution(self.variables, self.quality)
45 }
46 }

```

Código 5.2. Código reducido sobre el constructor de soluciones o permutaciones binarias

5.1.2 La abstracción de los Problemas

Posteriormente se abordó la implementación de los **problemas**, definidos mediante un *trait* que actúa como el contrato de interfaz para cualquier modelo de optimización que se desee resolver. Para garantizar que la biblioteca fuera lo suficientemente flexible como para soportar diversos dominios de problemas —desde optimización continua hasta combinatoria—, se diseñó una estructura altamente genérica.

La interfaz que deben satisfacer todos los modelos se ha proyectado bajo un enfoque minimalista y cohesivo. Este contrato obliga a la implementación de métodos específicos destinados a la obtención de metadatos identificativos de la instancia, los cuales sirven como identificadores unívocos en los sistemas de persistencia y puntos de control (*checkpoints*). Asimismo, exige la definición de funciones para la evaluación cuantitativa de las soluciones y un mecanismo de inicialización estocástica que dote a los algoritmos de un punto de partida válido en el espacio de búsqueda. Esta especificación formal se detalla en el Código 5.3.

```

1 pub trait Problem<T, Q = f64>
2 where
3     T: Clone,
4     Q: Clone + Default,
5 {
6     fn new() -> Self;

```

```

7
8  fn better_fitness_fn(&self) -> fn(f64, f64) -> bool;
9
10 fn evaluate(&self, solution: &mut Solution<T, Q>);
11
12 fn create_solution(&self, _rng: &mut Random) -> Solution<T, Q>;
13
14 fn dominates(
15     &self,
16     solution_a: &Solution<T, Q>,
17     solution_b: &Solution<T, Q>
18 ) -> bool;
19
20 fn get_improvement_direction(&self) -> ImprovementDirection;
21
22 fn get_problem_description(&self) -> String;
23
24 fn format_solution(&self, solution: &Solution<T, Q>) -> String;
25 }

```

Código 5.3. Muestra la implementación del *trait* problema.

Como se observa en la definición conceptual expuesta en el Código 5.3, el uso de tipos genéricos permite desacoplar por completo la lógica del problema de la representación interna de los datos. El parámetro T representa el tipo de la variable de decisión, mientras que Q define el tipo de la evaluación (u objetivo), cuya especialización por defecto se establece como f64. Esta abstracción delega en la concreción del usuario la responsabilidad de definir aspectos críticos como la dirección de mejora (better_fitness_fn e get_improvement_direction), permitiendo configurar de manera flexible los criterios de ordenación e idoneidad de las soluciones.

Asimismo, el trait estandariza la forma en que los algoritmos interactúan con el dominio del problema: la función evaluate califica la calidad de un espécimen mediante efectos colaterales controlados en su estado, mientras que create_solution define el mecanismo de generación inicial parametrizado por un generador de números pseudoaleatorios. Esta arquitectura asegura que las metaheurísticas puedan operar de forma agnóstica al problema concreto, potenciando la extensibilidad y reutilización del sistema. Una vez consolidada esta base, el desarrollo prosiguió con la implementación de los operadores y los algoritmos, encargados de explotar estas definiciones para guiar la búsqueda hacia soluciones óptimas.

5.1.3 Operadores

En lo que respecta a los operadores, se definió el `trait Operator` como la abstracción base de la jerarquía del módulo de variación. Aunque su estructura se ha mantenido intencionadamente mínima para no sobrecargar el tiempo de cómputo, actúa como una interfaz común que unifica la gestión de metadatos esenciales, tales como el nombre del operador o su firma procedimental. Esto facilita la trazabilidad de las transformaciones genéticas, la auditoría del comportamiento del algoritmo y la posterior estructuración de los reportes estadísticos de resultados.

```
1 pub trait Operator {  
2     fn name(&self) -> &str;  
3 }
```

Código 5.4. Interfaz base `Operator` para la identificación de componentes.

A partir de esta interfaz fundamental, se especializa la jerarquía en cinco categorías que representan los pilares de las metaheurísticas: los operadores de **mutación**, **cruce** (*crossover*), de vecindad, de memoria y de **selección**. Cabe destacar la estrecha relación arquitectónica entre la mutación y la vecindad; a nivel de implementación, un operador de mutación actúa funcionalmente como un generador de vecindad (y viceversa), permitiendo que una misma estructura de código sea reutilizada tanto en algoritmos poblacionales como en búsquedas locales.

Esta especialización mediante subtipos de `traits` permite que los algoritmos manipulen las soluciones de forma genérica, independientemente del tipo de dato subyacente (`T`) o de la métrica de evaluación (`Q`), tal y como se detalla en el Código 5.5.

```
1 pub trait MutationOperator<T, Q = f64>: Operator  
2 where  
3     T: Clone,  
4     Q: Clone,  
5 {  
6     fn execute(  
7         &self,  
8         solution: &mut Solution<T, Q>,  
9         probability: f64,  
10        bounds: Option<&RealBounds>,  
11        rng: &mut Random,  
12    );  
13 }
```

```

15 pub trait NeighborhoodOperator<T, Q = f64>: Operator
16 where
17     T: Clone,
18     Q: Clone,
19 {
20     fn random_neighbor(
21         &self,
22         solution: &Solution<T, Q>,
23         bounds: Option<&RealBounds>,
24         rng: &mut Random,
25     ) -> Solution<T, Q>;
26 }
27
28 pub trait CrossoverOperator<T, Q = f64>: Operator
29 where
30     T: Clone,
31     Q: Clone,
32 {
33     fn execute(
34         &self,
35         parent1: &Solution<T, Q>,
36         parent2: &Solution<T, Q>,
37         bounds: Option<&RealBounds>,
38         rng: &mut Random,
39     ) -> Vec<Solution<T, Q>>;
40 }
41
42 pub trait SelectionOperator<T, Q = f64>: Operator
43 where
44     T: Clone,
45     Q: Clone,
46 {
47     fn execute<'a>(
48         &self,
49         population: &'a [Solution<T, Q>],
50         rng: &mut Random,
51         dominates: &dyn Fn(&Solution<T, Q>, &Solution<T, Q>) -> bool,
52     ) -> &'a Solution<T, Q>;
53 }

```

```

54
55 pub trait TabuMemoryOperator<T, Q = f64>: Operator
56 where
57     T: Clone + Display,
58     Q: Clone + Display,
59 {
60     fn is_tabu(
61         &self,
62         tabu_memory: &HashMap<String, usize>,
63         candidate: &Solution<T, Q>,
64         iteration: usize,
65     ) -> bool {
66         tabu_memory
67             .get(&self.signature(candidate))
68             .is_some_and(|expiry| *expiry > iteration)
69     }
70
71     fn remember(
72         &self,
73         tabu_memory: &mut HashMap<String, usize>,
74         solution: &Solution<T, Q>,
75         iteration: usize,
76         tabu_tenure: usize,
77     ) {
78         tabu_memory.insert(self.signature(solution), iteration + tabu_tenure);
79     }
80 }

```

Código 5.5. Muestra del desarrollo de los diferentes tipos de *traits* sobre los operadores.

Como se puede apreciar en el Código 5.5, cada contrato especifica formalmente los requisitos para su cometido. Un aspecto clave en el diseño de las firmas de `MutationOperator` y `CrossoverOperator` es la inclusión opcional de los límites espaciales del problema mediante el parámetro `bounds: Option<&RealBounds>`. Esta adición es de vital importancia para garantizar la validez matemática y la abstracción de la biblioteca cuando se opera en espacios de búsqueda continuos de números reales. Operadores paradigmáticos de la literatura, tales como el cruce binario simulado (*Simulated Binary Crossover*, SBX) o la mutación polinomial (*Polynomial Mutation*), calculan las nuevas posiciones basándose estrictamente en distribuciones de probabilidad acotadas por las fron-

teras del espacio factible Ω . Al pasar estos límites de manera genérica y opcional, la biblioteca es capaz de reutilizar el mismo flujo computacional tanto para optimización continua de alta dimensionalidad como para problemas discretos, donde las variables toman valores dentro de ciertos rangos acotados. Asimismo, esta flexibilidad estructural permite abordar problemas puramente combinatorios donde los límites numéricos tradicionales no aplican y se evalúan como `None`, quedando las restricciones implícitas en la propia naturaleza de la solución.

Por otra parte, resulta destacable la gestión de los tiempos de vida (*lifetimes*)² implementada en la firma del método de ejecución del `SelectionOperator`. En lenguajes de programación tradicionales, retornar un elemento elegido de una colección suele implicar o bien la copia íntegra del objeto en memoria o bien el uso de punteros desprotegidos que amenazan la estabilidad del hilo de ejecución. A través del parámetro genérico de ciclo de vida 'a, el compilador de Rust valida estáticamente que la referencia de la solución seleccionada devuelta por el operador posea exactamente la misma persistencia temporal que el vector de la población original (`population: &'a [Solution<T, Q>]`). Esta restricción asegura que no se produzcan punteros colgados si la población se destruye, eliminando por completo la necesidad de realizar clonaciones pesadas de memoria (*overhead* por asignación en el *heap*) y optimizando drásticamente la velocidad en cada iteración del algoritmo.

5.1.4 Algoritmos

Para la configuración e instanciación de cada algoritmo, el sistema se apoya en una estructura de parámetros que actúa como su base configurativa unificada. Esta estructura encapsula toda la información necesaria para la construcción y parametrización de la metaheurística, desempeñando además un papel crucial en el módulo de experimentos. En dicho módulo, se emplea para almacenar y replicar las configuraciones específicas de cada algoritmo que servirán como casos de prueba o vectores de sintonización dentro de los *benchmarks*.

Con el objetivo de garantizar la integridad y robustez del sistema antes de iniciar el cómputo, se incorpora un mecanismo de verificación estricto gobernado por la función `validate_parameters(&self) -> Result<(), String>`. Este método se invoca de manera sistemática y automatizada antes de comenzar el proceso de resolución, validando que los rangos numéricos, probabilidades y dimensiones

²En Rust, un *lifetime* (denotado comúnmente mediante anotaciones como 'a) es un mecanismo del compilador, evaluado estáticamente, que garantiza la validez de las referencias en memoria. En este contexto, la anotación asegura que las referencias a las soluciones seleccionadas por el `SelectionOperator` no excedan el tiempo de vida de la población original de la que fueron extraídas, previniendo así errores de acceso a memoria (*dangling pointers*) con un coste de abstracción nulo en tiempo de ejecución.

introducidos por el usuario sean coherentes y correctos. En caso de detectar anomalías, la función interrumpe la ejecución de forma segura devolviendo un error con un mensaje personalizado, lo que facilita una auditoría precisa y un diagnóstico rápido de la configuración antes de que el motor consuma tiempo de CPU.

Para llevar a cabo su propósito exploratorio, las metaheurísticas interactúan directamente con los operadores de variación y selección, los cuales realizan transformaciones estocásticas y locales sobre las soluciones de forma iterativa. Gracias al acoplamiento débil que ofrece el diseño arquitectónico, estos operadores son completamente intercambiables; esto permite alterar radicalmente el comportamiento dinámico del algoritmo sin modificar un solo bloque de su implementación interna. Bajo este mismo principio de modularidad y diseño agnóstico, el sistema facilita la sustitución o modificación del problema en evaluación de manera directa y flexible. Asimismo, los algoritmos admiten la inyección de observadores para monitorizar el estado interno de la ejecución en tiempo real, un componente que se detalla minuciosamente en su apartado correspondiente.

Desde el punto de vista del rendimiento temporal, estas estructuras se han diseñado para ser ejecutadas de forma concurrente o paralela, minimizando la latencia de procesamiento. La biblioteca ofrece al usuario la flexibilidad de especificar manualmente el tamaño del grupo de hilos (*thread pool*) a instanciar o, alternativamente, delegar dicha gestión al propio entorno. En este último escenario, el nivel de concurrencia óptimo se determina dinámicamente en tiempo de ejecución analizando la topología de la máquina del usuario, utilizando para ello la función de la biblioteca estándar `available_parallelism()`. Toda esta suite de comportamientos y capacidades operativas queda consolidada formalmente bajo el contrato de interfaz especificado por el `trait Algorithm`, detallado en el Código 5.6.

```
1 pub trait Algorithm<T, Q = f64>
2 where
3     T: Clone + Send + 'static + Display,
4     Q: Clone + Default + Send + 'static + Display,
5 {
6     type SolutionSet: SolutionSet<T, Q>;
7     type Parameters;
8     type StepState: StepStateCheckpoint<T, Q>;
9
10    fn new(parameters: Self::Parameters) -> Self;
11
12    fn algorithm_name(&self) -> &str;
```

```

13
14 fn termination_criteria(&self) -> TerminationCriteria;
15
16 fn observers_mut(&mut self) -> &mut Vec<Box<dyn AlgorithmObserver<T, Q>>>;
17
18 fn set_solution_set(&mut self, solution_set: Self::SolutionSet);
19
20 fn run<P>(&mut self, problem: &P) -> Result<Self::SolutionSet, String>;
21
22 fn validate_parameters(&self) -> Result<(), String> {
23     Ok(())
24 }
25
26 fn get_solution_set(&self) -> Option<&Self::SolutionSet>;
27
28 fn step(
29     &self,
30     problem: &(impl Problem<T, Q> + Sync),
31     state: &mut Self::StepState,
32 );
33 }

```

Código 5.6. Definición del contrato y de los tipos asociados para el componente *Algorithm*

Como se observa en el Código 5.6, la declaración del `trait` hace uso de tipos asociados (`type SolutionSet`, `type Parameters` y `type StepState`). Esta decisión de diseño en lugar de recurrir puramente a parámetros genéricos en la cabecera del rasgo resulta fundamental para limpiar las firmas del código, agrupando los tipos dependientes bajo el espacio de nombres del propio algoritmo y garantizando que un algoritmo concreto solo pueda interactuar con su estructura de parámetros y estados compatible.

Asimismo, es imprescindible destacar la rigurosidad de las restricciones de concurrencia (*trait bounds*) impuestas sobre los parámetros de tipos genéricos `T` (variables de decisión) y `Q` (valores de aptitud), que obligan a la implementación de los marcadores `Send` y `'static`. En el ecosistema Rust, garantizar de forma estática que las estructuras de datos poblacionales puedan transferirse de forma segura entre las fronteras de distintos hilos de ejecución sin riesgo de provocar condiciones de carrera es una exigencia crítica. La presencia de `Send` asegura esta transferencia segura de la

propiedad (*ownership*)³ en entornos multitarea, mientras que el ciclo de vida 'a implícito en 'static certifica que los datos contenidos no poseen referencias locales con hilos que puedan ser destruidos prematuramente, blindando la integridad de la memoria.

La lógica de ejecución se implementa mediante un sistema basado en pasos. El punto de entrada principal para iniciar cualquier metaheurística es el método `run`, cuya firma procedimental e interfaz de rasgos se detalla en el Código 5.7.

```
1 fn run<P>(&mut self, problem: &P) -> Result<Self::SolutionSet, String>
2 where
3     Self: Sized,
4     Self::SolutionSet: Clone,
5     P: Problem<T, Q> + Sync,
6 {
```

Código 5.7. Definición de la firma genérica del método principal *run*.

El diseño de la función `run` aprovecha las ventajas de los sistemas de tipos avanzados de Rust. Destaca en su declaración el uso del límite restrictivo `Self: Sized`. En Rust, por defecto, los *traits* pueden ser implementados por tipos cuyo tamaño en tiempo de compilación no sea conocido de forma estática (como los objetos de rasgo o *trait objects*). Al imponer el *bound* `Sized`, se garantiza que el algoritmo concreto posee un tamaño fijo y conocido en tiempo de compilación. Esto es un requisito indispensable para permitir el polimorfismo estático, asegurando que el compilador optimice las llamadas a funciones eliminando el coste de la resolución dinámica en tiempo de ejecución. Asimismo, la restricción `Sync` sobre el parámetro genérico del problema (`P`) asegura que la estructura del modelo matemático sea segura para ser accedida concurrentemente desde múltiples hilos de ejecución, habilitando el paralelismo a nivel del motor experimental.

Desde una perspectiva arquitectónica, `run` delega la orquestación y el control de eventos en una función interna especializada de la biblioteca denominada `run_with_observer_runtime`, detallada en el Código 5.8. El método `run` actúa bajo un enfoque de programación funcional, encapsulando el comportamiento interactivo de la metaheurística en una *closure* o función anónima que se inyecta a

³El sistema de propiedad (*ownership*) es el paradigma central de gestión de memoria en Rust, gobernado por tres reglas estrictas verificadas en tiempo de compilación: cada valor tiene un único propietario (*owner*), solo puede haber un propietario a la vez y el valor se libera inmediatamente cuando su propietario sale de su ámbito de validez (*scope*). En entornos concurrentes, este modelo interactúa directamente con el rasgo (*trait*) `Send`, el cual actúa como un marcador que indica al compilador que la propiedad de un tipo de dato puede transferirse de forma segura entre hilos sin riesgo de provocar accesos concurrentes no controlados o corrupción de memoria.

través del argumento `task`. El motor central evalúa de forma transparente esta tarea, aislando los mecanismos de monitorización del algoritmo en sí.

```
1 pub fn run_with_observer_runtime<T, Q, R, F>(  
2     observers: &mut Vec<Box<dyn AlgorithmObserver<T, Q>>>,  
3     criteria: TerminationCriteria,  
4     better_fitness: fn(f64, f64) -> bool,  
5     algorithm_name: impl Into<String>,  
6     task: F,  
7 ) -> R  
8 where  
9     T: Clone + Send + 'static,  
10    Q: Clone + Send + 'static,  
11    F: FnOnce(&ExecutionContext<T, Q>) -> RuntimeExecutionOutput<R>,  
12 {  
13     let algorithm_name = algorithm_name.into();  
14  
15     let runtime = ObserverRuntime::new(std::mem::take(observers));  
16     let context = ExecutionContext::new(runtime.sender(), criteria, better_fitness);  
17     context.start(algorithm_name);  
18  
19     let output = task(&context);  
20     context.end(output.total_generations, output.total_evaluations);  
21  
22     drop(context);  
23  
24     *observers = runtime.finish();  
25     output.result  
26 }
```

Código 5.8. Implementación del motor de ejecución y el entorno de comunicación para los observadores.

Como se aprecia en la lógica de `run_with_observer_runtime`, la función asume el control del ciclo de vida del proceso de optimización. Inicialmente extrae los observadores mediante `std::mem::take` para evitar copias costosas y configura un canal de comunicación asíncrono gestionado a través de un `ExecutionContext`. Tras arrancar formalmente el contexto, se invoca la *closure* `task`, transfiriéndole una referencia del contexto de ejecución para que el algoritmo pueda reportar sus progresos. Una vez finalizada la tarea, es de vital importancia la llamada explícita a

`drop(context)`. Este paso destruye el contexto de forma controlada, provocando el cierre definitivo del extremo emisor del canal. Al cerrarse el flujo, el hilo secundario encargado de procesar la lógica de los observadores puede consumir los mensajes restantes en cola y concluir de manera limpia su ciclo de ejecución, evitando interbloqueos antes de recuperar los observadores a través de `runtime.finish()`.

Finalmente, la sustancia algorítmica de la tarea inyectada se despliega en un bucle iterativo controlado por las condiciones de parada, cuyo comportamiento se describe en el Código 5.9.

```
1 while !context.should_terminate()
2 {
3     algorithm.step(problem, &mut state);
4
5     let step_snapshot = algorithm.build_snapshot(problem, &state);
6     context.update_execution_state(&step_snapshot);
7     last_iteration = step_snapshot.iteration;
8     last_evaluations = step_snapshot.evaluations;
9
10    if DEFAULT_FREQUENCY_OF_CHECKPOINT_WRITES > 0 &&
11    step_snapshot.iteration % DEFAULT_FREQUENCY_OF_CHECKPOINT_WRITES == 0
12    {
13        let record = state.build_checkpoint_record(
14            &run_id,
15            &checkpoint_metadata,
16            context.time_elapsed(),
17        );
18        checkpoint_policy.persist_record(&mut last_checkpoint_path, &record);
19    }
20    context.report_progress(
21        ObserverState::from_snapshot(&step_snapshot, context.seq_id())
22    );
23 }
```

Código 5.9. Bucle de ejecución del interior de la función *run*.

En cada iteración del bucle, el algoritmo avanza mediante el método `step`, el cual altera el estado de la población o la trayectoria actual. Acto seguido, se extrae un estado intermedio inmutable o captura (*snapshot*) que almacena la evolución temporal de la ejecución. Este reporte intermedio se aprovecha por el sistema de tolerancia a fallos mediante una política de puntos de control o *checkpoints*. Si la frecuencia

parametrizada coincide con el módulo de la iteración corriente, se construye un registro (*record*) que se vuelca de forma persistente a disco a través de `checkpoint_policy.persist_record`, permitiendo reanudar la computación en caso de una interrupción imprevista de la máquina.

El ciclo concluye con la invocación de `context.report_progress`, función encargada de serializar y transmitir el nuevo estado en forma de un mensaje `ObserverState` hacia el canal del runtime de observadores, garantizando una auditoría en tiempo real del proceso sin penalizar el rendimiento del hilo principal.

El bucle de ejecución persiste mientras el contexto del algoritmo, representado por la estructura `ExecutionContext`, no determine el cese de la actividad mediante el método `should_terminate()`. La decisión de terminación está delegada de forma flexible en el tipo `TerminationCriterion`, un enumerado que permite al usuario definir condiciones de parada basadas en diversos criterios físicos y lógicos.

```
1 #[derive(Clone, Debug)]
2 pub enum TerminationCriterion {
3     MaxIterations(usize),
4     MaxEvaluations(usize),
5     Convergence { threshold: f64, patience: usize },
6     TimeLimit(Duration),
7     NoImprovement { patience: usize },
8 }
```

Código 5.10. Enumerado *TerminationCriterion* con las condiciones de parada disponibles.

Esta implementación mediante un `enum` permite una gran expresividad en la configuración de los experimentos. El usuario puede optar por límites tradicionales, como el número de iteraciones o evaluaciones, o por criterios más avanzados de convergencia y estancamiento (*patience*). La integración de estos criterios en el `ExecutionContext` garantiza que la comprobación se realice de manera eficiente al finalizar cada paso del algoritmo, permitiendo además que los observadores capturen el progreso en tiempo real antes de cada evaluación de terminación.

```
1 #[derive(Clone, Debug)]
2 pub struct ExecutionContext<T, Q = f64>
3 where
4     T: Clone + Send + 'static,
5     Q: Clone + Send + 'static,
6 {
7     sender: ObserverSender<T, Q>,
```

```

8     termination: RefCell<TerminationController>,
9     next_snapshot_seq: RefCell<u64>,
10 }

```

Código 5.11. Estructura *ExecutionContext* encargada de la gestión del entorno y el control del ciclo de vida del algoritmo.

A partir de la definición expuesta en el Código 5.11, el `ExecutionContext` se consolida como el componente encargado de centralizar y gobernar de forma unificada el entorno operacional del algoritmo de optimización. Su propósito principal es aislar las tareas de control de la lógica metaheurística pura, asumiendo de forma integrada la gestión del cese de actividad, la canalización hacia el ecosistema de monitorización y la orquestación de la persistencia física en disco.

En síntesis, estas tres responsabilidades esenciales se materializan a nivel de implementación mediante los siguientes mecanismos técnicos integrados en la estructura:

- **Control de parada:** La evaluación del cese de actividad se resuelve de manera determinista delegando en el controlador interno `termination`. Este procesa el estado en cada paso iterativo contrastándolo con las variantes del enumerado `TerminationCriterion`, lo que permite validar de forma unificada criterios físicos (tiempo) y lógicos (evaluaciones o estancamiento).
- **Canalización de eventos:** La comunicación hacia el ecosistema de monitorización exterior se implementa de forma asíncrona a través del campo `sender` (`ObserverSender<T, Q>`). Este canal empaqueta las capturas inmutables del progreso de manera no bloqueante, utilizando el contador secuencial `next_snapshot_seq` para garantizar el orden de los eventos en el hilo consumidor sin añadir latencia al bucle de optimización.
- **Gestión de persistencia:** La supervisión de la tolerancia a fallos se apoya en la centralización métrica del contexto. Al evaluar de forma transparente la frecuencia de guardado parametrizada respecto a la iteración actual, el componente coordina la captura de datos y ordena el volcado síncrono del registro intermedio en el almacenamiento secundario, abstrayendo al algoritmo de la gestión física de los *checkpoints*.

Cabe destacar desde la perspectiva de la ingeniería de software en Rust el uso del contenedor de mutabilidad interna `RefCell` en sus campos internos. Debido a que el contexto se transfiere a través de referencias inmutables en gran parte de las firmas de la biblioteca para proteger la integridad de los hilos, `RefCell` permite modificar de forma segura y dinámica el estado del controlador de parada

y los índices de secuencia en tiempo de ejecución, eludiendo las restricciones estrictas del sistema de préstamos del compilador sin sacrificar la seguridad de memoria.

5.1.5 Observadores y checkpoints

Los observadores se basan en el uso de *traits*, los cuales actúan como interfaces abstractas que definen el contrato de comportamiento obligatorio para cualquier componente de monitorización. Este enfoque no solo garantiza un polimorfismo robusto, sino que facilita la extensibilidad del sistema, permitiendo al desarrollador integrar nuevas estrategias de visualización o registro sin necesidad de alterar el núcleo de la biblioteca. Para cubrir las necesidades de monitorización más comunes, el módulo incorpora diversas implementaciones predefinidas con un funcionamiento por defecto. Estas herramientas listas para su uso inmediato abarcan desde la salida formateada por consola estándar (*stdout*) hasta sistemas encargados de estructurar datos evolutivos en tablas, renderizar gráficos dinámicos de rendimiento y generar reportes analíticos detallados en formato HTML. Estas dos últimas variantes permiten examinar minuciosamente la convergencia de las metaheurísticas en función del número acumulado de evaluaciones de la función objetivo.

Para integrar este ecosistema, los observadores se asocian de forma dinámica a aquellos algoritmos que satisfacen el contrato del *trait Observable*, interfaz que provee los métodos requeridos para registrar y revocar suscripciones durante el ciclo de vida de la ejecución.

Sin embargo, el diseño del flujo de comunicación entre el motor algorítmico y los observadores adjuntos supuso un reto de diseño de software debido a las restricciones impuestas por el sistema de propiedad y préstamos de Rust (*Borrow Checker*)⁴. Inicialmente, se evaluó una implementación clásica basada en el patrón *Observer* convencional, donde el objeto del algoritmo retenía una colección de referencias mutables a sus observadores para notificarlos secuencialmente a través de un método directo. Esta aproximación resultó excesivamente compleja y rígida, ya que las estrictas reglas que prohíben la coexistencia de múltiples alias compartidos con acceso mutable (*aliasing + mutability*) imposibilitaban un manejo limpio de los tiempos de vida en entornos concurrentes.

⁴El *Borrow Checker* es el mecanismo estático del compilador de Rust encargado de validar la legitimidad de las referencias mediante las reglas de préstamos (*borrowing*). Este sistema impone dos restricciones fundamentales en tiempo de compilación para garantizar la seguridad de la memoria sin un recolector de basura: primero, permite un número ilimitado de referencias de solo lectura (*&T*) o bien una única referencia de lectura y escritura (*&mut T*) simultáneamente sobre un mismo recurso; y segundo, exige que el ciclo de vida (*lifetime*) de cualquier referencia esté estrictamente acotado por el del dato original. En arquitecturas concurrentes, estas reglas impiden de forma absoluta la existencia de referencias huérfanas o accesos concurrentes en conflicto, forzando al desarrollador a estructurar minuciosamente la jerarquía y el flujo de los datos.

Como alternativa de diseño, se optó por un modelo concurrente basado en canales (*channels*)⁵, consolidando un patrón idiomático que garantiza la seguridad de memoria sin recurrir a mecanismos costosos de exclusión mutua. En este esquema, el algoritmo de optimización se delega a un hilo de ejecución independiente, mientras que la suite de observadores permanece en el hilo principal a la espera de eventos de forma asíncrona mediante el modelo de paso de mensajes. Bajo este paradigma, el hilo del algoritmo actúa como un productor exclusivo que genera capturas inmutables del estado actual (*snapshots*) al completar cada iteración o alcanzar un hito de progreso, enviándolas a través del canal. El hilo principal asume el rol de consumidor, encargándose de recibir dichos paquetes de datos de forma no bloqueante y distribuirlos síncronamente entre los observadores registrados, tal y como se ilustra en la Figura 5.1.

Este diseño arquitectónico no solo incrementa la robustez global del sistema, sino que favorece la escalabilidad horizontal de la biblioteca. La incorporación de canales con capacidad de almacenamiento intermedio faculta a los observadores para absorber y ejecutar de forma diferida tareas computacionalmente pesadas o de alta latencia de entrada/salida —como el volcado persistente en disco— en su propio espacio de hilos. Como consecuencia directa, estas operaciones costosas se procesan sin comprometer en ningún momento la velocidad del motor heurístico interno, maximizando la eficiencia temporal y garantizando una gestión óptima de los recursos multi-núcleo de la CPU.

Complementariamente, el sistema de estados no solo asiste a la monitorización externa, sino que constituye el pilar fundamental del mecanismo de puntos de control o *checkpoints*. Estos estados se persisten en disco de forma periódica siguiendo las convenciones y rutas de almacenamiento estándar de cada sistema operativo, empleando directorios específicos como `LOCAL_STATE_ROMA` en entornos Linux, `HOME_ROMA` en macOS y `LOCAL_APPDATA` en sistemas Windows.

La información almacenada en cada punto de control incluye los datos críticos necesarios para reconstruir una ejecución específica, abarcando la configuración del algoritmo, el problema abordado y sus parámetros correspondientes. Para garantizar la integridad y la identificación unívoca de cada escenario, este conjunto de datos se procesa mediante una función *hash*, generando un identificador único que vincula de manera determinista la configuración con sus archivos de estado. Al iniciar el sistema con la opción de reanudación activada, la biblioteca genera el *hash* de la configuración actual

⁵En el ecosistema de Rust, los canales son primitivas de concurrencia que facilitan la comunicación entre hilos mediante el paso de mensajes (típicamente empleando un patrón emisor-receptor, como MPSC: *Multiple Producer, Single Consumer*). Este modelo aprovecha el sistema de propiedad (*ownership*) del lenguaje para transferir datos de forma segura entre distintos contextos de ejecución, eliminando las condiciones de carrera (*data races*) en tiempo de compilación y evitando los cuellos de botella propios de la memoria compartida.

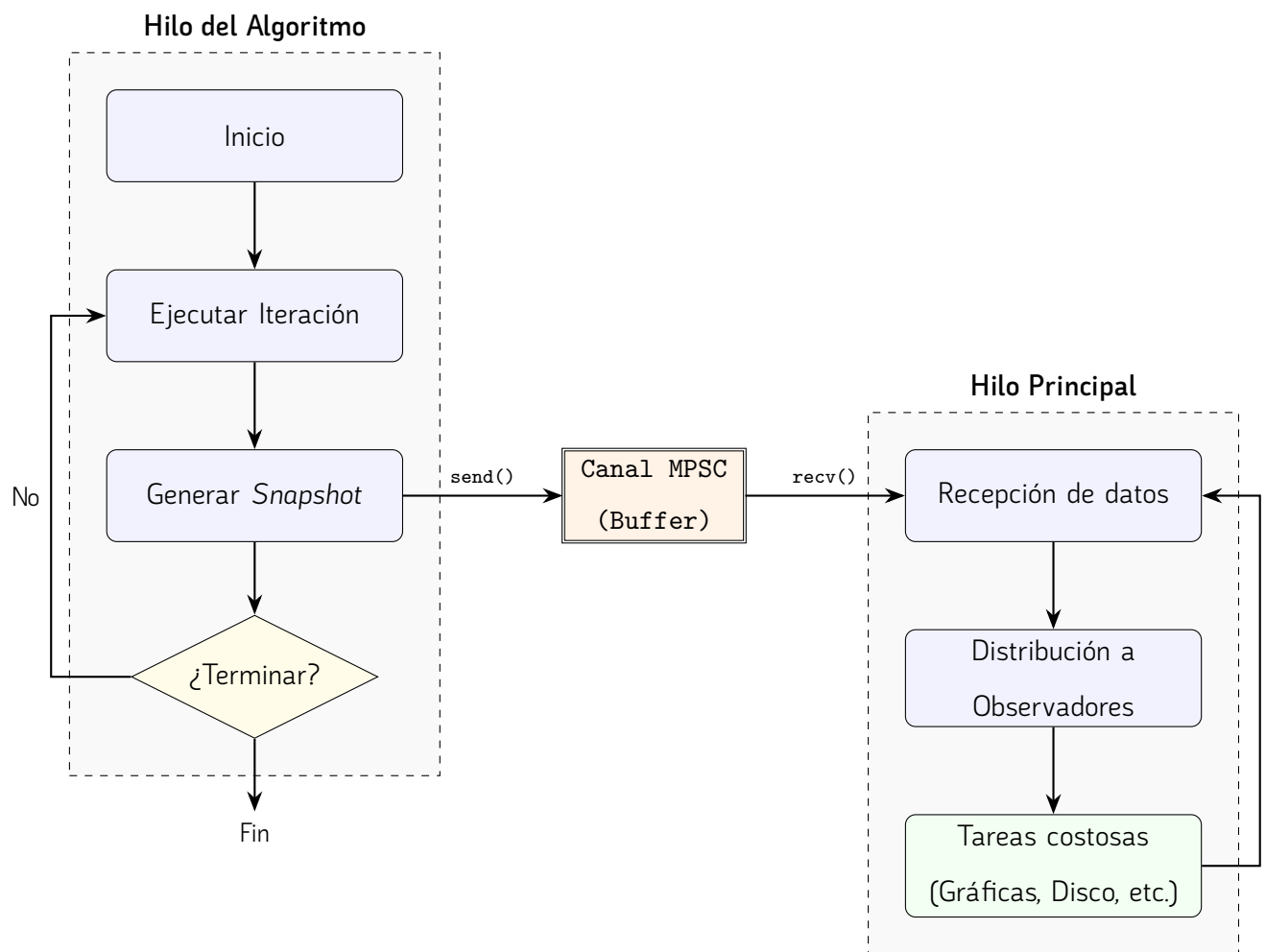


Figura 5.1. Arquitectura de comunicación asíncrona entre hilos mediante eventos.

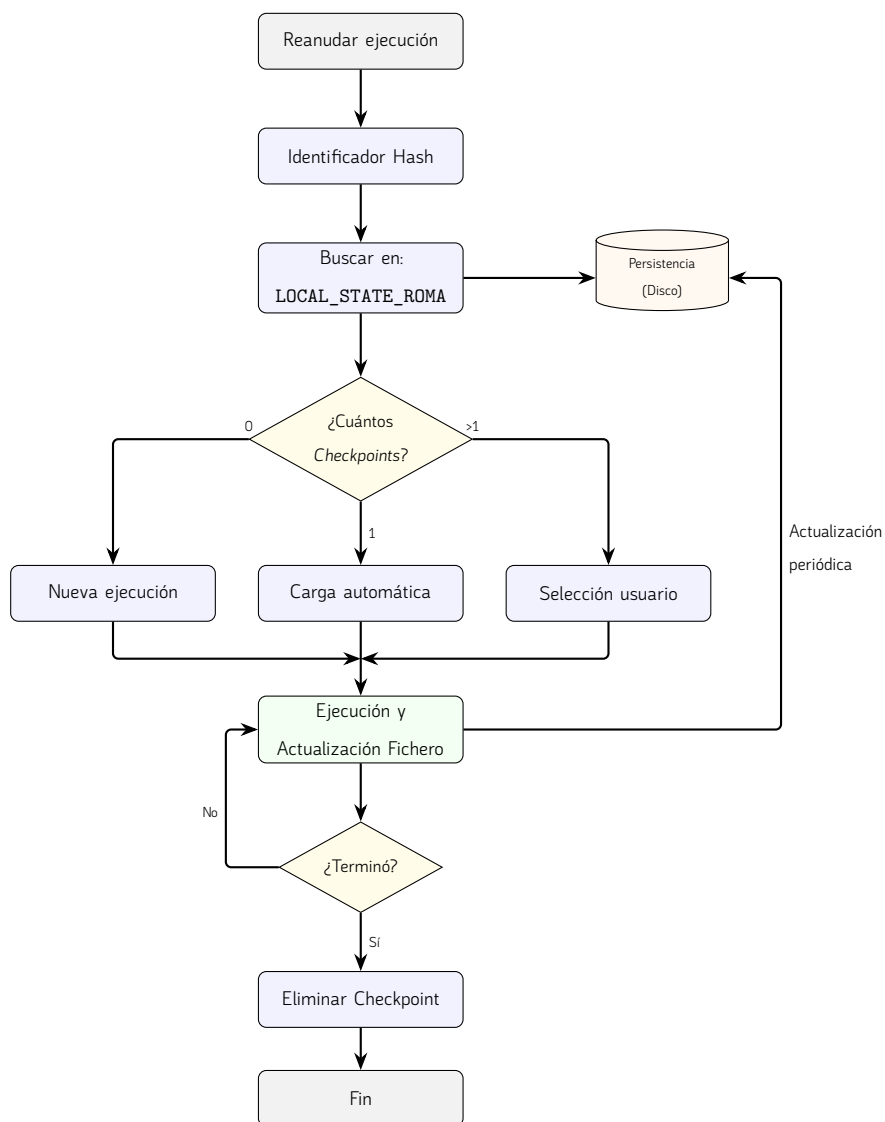


Figura 5.2. Flujo compacto de gestión de persistencia y recuperación de estados en Linux.

y busca coincidencias en el sistema de archivos. En caso de localizar puntos de control compatibles, el sistema permite al usuario seleccionar el estado más conveniente para recuperar la ejecución o, alternativamente, optar por iniciar un proceso completamente nuevo, ofreciendo así una capa de tolerancia a fallos y flexibilidad en experimentos de larga duración.

Listing 1 Ejemplo solicitud de punto de control, al intentar reanudar la ejecución de un algoritmo

```

1 david ~/roma/roma/examples cargo run --example knapsack_hc_demo -- --resume
2   Compiling roma v0.1.0 (/home/david/roma/roma)
3   Finished `dev` profile [unoptimized + debuginfo] target(s) in 1.08s
4   Running `/home/david/roma/roma/target/debug/examples/knapsack_hc_demo --resume`
5
6                                     --- CHECKPOINT SELECTION ---
7 ID   | AGE      | ELAPSED. | INFO
8 -----
9 [ 1] | 3 days ago | 00:00:00 | > iter=59750;eval=59751;seed=8238046302331804803;
10 [ 2] | 3 days ago | 00:00:00 | > iter=52560;eval=52561;seed=8364640122973076249;
11 [ 3] | 3 days ago | 00:00:00 | > iter=52100;eval=52101;seed=10450884331206695397;
12 [ 4] | 3 days ago | 00:00:00 | > iter=43510;eval=43511;seed=15322852344410324979;
13 [ 5] | 3 days ago | 00:00:00 | > iter=69010;eval=69011;seed=7946725581494022999;
14 [ 6] | 3 days ago | 00:00:00 | > iter=57640;eval=57641;seed=5376056338251316545;
15 [ 7] | 3 days ago | 00:00:00 | > iter=56170;eval=56171;seed=14048048055183702911;
16 -----
17 [0] Start a new run (ignore existing)
18 -----
19 > Select checkpoint index:

```

5.1.6 Utilidades y experimentos

Para dar cumplimiento estricto al requisito arquitectónico de mantener *cero dependencias externas*, se ha desarrollado un ecosistema robusto de submódulos auxiliares integrados en el núcleo de la biblioteca. Estas utilidades cubren un amplio espectro de necesidades operativas que van desde sistemas de *benchmarking* para auditar tiempos de ejecución, hasta serializadores encargados de transformar estados de texto plano a formatos binarios eficientes para la persistencia. Asimismo, se incorporan motores ligeros de representación gráfica capaces de vectorizar curvas de rendimiento en formato SVG y rutinas avanzadas de entrada y salida por consola. Este último apartado reviste una complejidad notable, ya que la biblioteca confía plenamente en la terminal para interactuar con el usuario de manera interactiva.

La parametrización y el control de las ejecuciones se canalizan a través de un analizador de argumentos de línea de comandos implementado a medida, cuya interfaz de control se detalla en el

Código 5.12.

```

1 #[derive(Debug, Clone, PartialEq, Eq)]
2 pub struct CliArgs {
3     args: Vec<String>,
4 }
5
6 impl CliArgs {
7     pub fn from_env() -> Self {
8         Self::from_iter(std::env::args().skip(1))
9     }
10
11     pub fn has_flag(&self, flag: &str) -> bool {
12         self.args.iter().any(|arg| arg == flag)
13     }
14
15     pub fn argument_value(&self, flag: &str) -> Option<String> {
16         self.argument_value_for_any(&[flag])
17     }
18
19     pub fn seed_or(&self, default_value: u64) -> u64 {
20         self.parse_u64_for_any_or(&[CLI_FLAG_SEED_SHORT, CLI_FLAG_SEED], default_value)
21     }
22
23     pub fn resume_requested(&self) -> bool {
24         self.has_flag(CLI_FLAG_RESUME)
25     }
26
27     pub fn checkpoints_disabled(&self) -> bool {
28         self.has_any_flag(&[CLI_FLAG_NO_CHECKPOINT, CLI_FLAG_NO_CHECKPOINT_SHORT])
29     }

```

Código 5.12. Estructura *CliArgs* encargada del procesamiento nativo de argumentos por línea de comandos.

La estructura *CliArgs* permite centralizar la captura de banderas (*flags*) operativas de manera ágil. A través de sus métodos, el usuario puede fijar de forma determinista la semilla estocástica del sistema mediante *seed_or*, deshabilitar explícitamente la persistencia periódica o solicitar de forma interactiva la reanudación de un experimento a partir de un punto de control previo con *resume_requested*.

Complementando la ingesta de datos, se han integrado analizadores sintácticos para interpretar configuraciones en formatos estructurados estándar como JSON, CSV o YAML. Esta capacidad facilita la especificación externa de instancias de problemas complejos o baterías de experimentos masivos sin obligar a la recompilación del código fuente.

Adicionalmente, el módulo de utilidades engloba funciones de dispersión criptográfica para la validación de archivos (*hashing*), primitivas de apoyo para la sincronización y paralelización interna de hilos, herramientas de abstracción de rutas en el sistema de archivos de forma multiplataforma y rutinas matemáticas preparadas para extraer métricas estadísticas de las poblaciones, las cuales nutren tanto al subsistema de observadores como a las funciones de renderizado gráfico.

No obstante, el componente más crítico y con mayor impacto en el desempeño de la biblioteca es el generador de números pseudoaleatorios (*PRNG*). Al tratarse de metaheurísticas estocásticas, la práctica totalidad de los operadores de variación, las heurísticas constructivas iniciales y las reglas de aceptación y selección dependen por completo de este motor, expuesto en el Código 5.13.

```
1 pub struct Random {
2     state: u64,
3 }
4
5 impl Random {
6     pub fn new(seed: u64) -> Self {
7         Self { state: seed }
8     }
9
10    #[inline]
11    pub fn next_u64(&mut self) -> u64 {
12        self.state = self.state.wrapping_add(0x9E3779B97F4A7C15);
13        mix64(self.state)
14    }
15
16    #[inline]
17    pub fn next_f64(&mut self) -> f64 {
18        let x = (self.next_u64() >> 11) as u64;
19        (x as f64) * (1.0 / 9007199254740992.0) // 1 / 253
20    }
21
22    #[inline]
23    pub fn range_between(&mut self, min: u64, max: u64) -> u64 {
```

```

24     if max <= min {
25         return min;
26     }
27     min + self.range(max - min)
28 }
29
30 #[inline]
31 pub fn chance(&mut self, p: f64) -> bool {
32     self.next_f64() < p
33 }
34
35 #[inline]
36 pub fn coin_flip(&mut self) -> bool {
37     self.chance(0.5)
38 }
39 }

```

Código 5.13. Implementación del generador de números pseudoaleatorios (PRNG).

Como se aprecia en la implementación, el método neurálgico es `next_u64`, del cual derivan todas las abstracciones del comportamiento estocástico del software, como el escalado a punto flotante `next_f64`, el cálculo acotado de rangos o la evaluación de probabilidades booleanas mediante `chance` y `coin_flip`. Para cumplir simultáneamente con una alta calidad estadística en las secuencias numéricas y un tiempo de cómputo ínfimo, se ha optado por implementar una variante pura del algoritmo *SplitMix64*. Este generador hace avanzar su estado interno mediante la adición modular con desbordamiento controlado (`wrapping_add`) de la constante dorada de 64 bits `0x9E3779B97F4A7C15`, un valor fraccionario derivado de la proporción áurea que garantiza una dispersión uniforme y un período máximo completo de 2^{64} estados antes de repetirse.

Debido a que el avance lineal del estado no es estadísticamente aleatorio por sí mismo, la secuencia se somete inmediatamente a una transformación final de mezcla no lineal implementada en la subrutina `mix64`, detallada en el Código 5.14.

```

1 #[inline]
2 fn mix64(mut z: u64) -> u64 {
3     z = (z ^ (z >> 30)).wrapping_mul(0xBF58476D1CE4E5B9);
4     z = (z ^ (z >> 27)).wrapping_mul(0x94D049BB133111EB);
5     z ^ (z >> 31)
6 }

```

Código 5.14. Función de mezcla no lineal *mix64* para maximizar la entropía de los bits generados.

La función `mix64` aplica de manera secuencial operaciones de desplazamiento de bits a la derecha (*bit-shifts*) combinadas con operaciones XOR ($\wedge$) y multiplicaciones por constantes de alta entropía numéricas fijas (`0xBF58476D1CE4E5B9` y `0x94D049BB133111EB`). Esta técnica destruye cualquier correlación lineal latente en el estado, garantizando que incluso variaciones de un solo bit en la semilla original provoquen cambios masivos y pseudoaleatorios en el flujo de salida, superando satisfactoriamente las baterías de pruebas estadísticas más exigentes del sector informático.

Finalmente, es crucial resaltar el uso sistemático del atributo descriptor `#[inline]` tanto en la función de mezcla como en los métodos del generador. Esta directiva solicita formalmente al compilador de Rust que sustituya la llamada tradicional a la función inyectando directamente su cuerpo de instrucciones binarias en el lugar de la invocación. Dado que los algoritmos evolutivos invocan este generador miles de millones de veces por ejecución para evaluar mutaciones y cruces, la directiva `#[inline]` suprime por completo la sobrecarga asociada al salto de subrutina, el guardado de registros en la pila y el retorno procedimental, traduciéndose en una ganancia de rendimiento temporal masiva y crítica para la biblioteca.

El módulo de experimentos constituye el último bloque funcional de la arquitectura y se especifica en estrecha correlación junto al módulo de utilidades. Esta disposición responde a una necesidad puramente integradora, dado que actúa como la cúspide de abstracción encargada de unificar y explotar todas las implementaciones descritas con anterioridad. El motor de experimentación consolida en un único entorno operativo el despliegue de las metaheurísticas, la definición formal de los problemas, la inyección del generador de números pseudoaleatorios y el uso coordinado de las utilidades de *benchmarking*, persistencia y entrada/salida, permitiendo ejecutar y evaluar de forma automatizada baterías complejas de pruebas.

Para garantizar la máxima flexibilidad y modularidad, el sistema elude cualquier acoplamiento rígido apoyándose en abstracciones avanzadas del lenguaje mediante la definición de *traits* específicos. Este diseño unifica el comportamiento de los parámetros bajo interfaces genéricas, habilitando la condición de "experimentable" para cualquier algoritmo que se integre en la biblioteca, independientemente de sus particularidades algorítmicas internas. Gracias a esta arquitectura polimórfica, el motor de experimentos puede manipular, instanciar, monitorizar y variar dinámicamente las configuraciones operativas de las técnicas sin necesidad de acoplarse a una implementación concreta. Este contrato formal queda materializado a través del `trait ExperimentalCase`, detallado en el Código 5.15.

```

1 pub trait ExperimentalCase<T, Q, P>: Send + Sync
2 where
3     T: Clone + Send + 'static,
4     Q: Clone + Default + Send + 'static + Copy + Into<f64>,
5     P: Problem<T, Q> + Sync,
6 {
7     fn algorithm_name(&self) -> &str;
8
9     fn parameters(&self) -> Vec<CaseParameter>;
10
11     fn parameters_as_text(&self) -> String {
12         self.parameters()
13             .into_iter()
14             .map(|p| p.to_string())
15             .collect::<Vec<_>>()
16             .join(", ")
17     }
18
19     fn run(&self, problem: &P) -> Result<Box<dyn SolutionSet<T, Q>>, String>;
20 }

```

Código 5.15. Definición del *trait* *ExperimentalCase* para la abstracción y ejecución genérica de casos de prueba.

Como se aprecia en la firma de rasgos expuesta en el Código 5.15, la interfaz *ExperimentalCase* define los métodos indispensables para identificar la metaheurística de forma textual (*algorithm_name*) y extraer o serializar sus vectores de hiperparámetros (*parameters* y *parameters_as_text*). Sin embargo, la rutina crítica es el método *run*, el cual asume la responsabilidad de instanciar el bucle generacional del algoritmo ante una referencia del problema genérico *P* y retornar el conjunto de resultados envueltos de forma opaca en un objeto de rasgo dinámico (*Box<dyn SolutionSet<T, Q>>*). Esta decisión de diseño resulta fundamental, pues permite que el motor trate de manera homogénea los conjuntos de soluciones devueltos por cualquier paradigma de optimización, ya sea mono-objetivo o multi-objetivo, sin necesidad de conocer su tipo concreto en tiempo de compilación.

Es crucial destacar las estrictas restricciones de concurrencia (*trait bounds*) implementadas en la cabecera del rasgo, el cual obliga a que toda estructura experimental satisfaga los marcadores *Send* y *Sync*. En el ecosistema Rust, estas directivas de tipado estático certifican que los casos experimentales

y las instancias de los problemas matemáticos son completamente seguros para ser compartidos y transferidos a través de múltiples hilos de ejecución de manera simultánea, erradicando en tiempo de compilación el riesgo de provocar condiciones de carrera o corrupciones de memoria.

Esta infraestructura concurrente habilita un sistema de paralelización a medida de alto rendimiento diseñado para optimizar el uso de los recursos del hardware de la máquina. Dado que las metaheurísticas poseen una naturaleza estocástica intrínseca, es metodológicamente obligatorio realizar múltiples ejecuciones independientes y recurrentes con diferentes semillas pseudoaleatorizadas para obtener una significancia estadística válida (medias, desviaciones típicas y contrastes de hipótesis). El módulo aprovecha los marcadores de seguridad y distribuye eficientemente esta carga computacional masiva asignando las ejecuciones repetitivas de forma concurrente a nivel de hilos a través de un grupo de trabajo, reduciendo drásticamente la latencia temporal global del *benchmark*.

Finalmente, el ciclo de experimentación concluye con un subsistema de reporte automatizado. Este componente se encarga de recopilar de manera estructurada las métricas de rendimiento, tiempos de cómputo exactos y calidad de las fronteras o soluciones de compromiso generadas durante el procesamiento multitarea. Acto seguido, exporta esta información agregada a los formatos estructurados estándar que se han integrado en el submódulo de utilidades (tales como CSV, JSON o ficheros vectoriales SVG), facilitando su posterior tratamiento estadístico externo, su integración en hojas de cálculo o su visualización analítica directa en la presente memoria.

5.2 Desarrollo de pruebas

La utilización de un correcto conjunto de pruebas agiliza en gran medida el trabajo de programación en cualquier fase del desarrollo, previniendo regresiones y garantizando la estabilidad del código. Poder verificar de forma determinista que el código subyacente opera de manera correcta resulta indispensable. Para llevar a cabo esta tarea, el proyecto se ha apoyado fuertemente en el *framework* de pruebas integrado de forma nativa en el ecosistema de Rust (`cargo test`).

5.2.1 Pruebas unitarias

Dada la metodología de desarrollo ascendente adoptada, la implementación de pruebas unitarias se convirtió en un pilar fundamental para asegurar la robustez de cada componente antes de escalar hacia niveles de abstracción superiores. El objetivo principal de estas pruebas fue verificar de forma aislada e independiente el comportamiento de las funciones y estructuras mínimas de cada módulo, garantizando que cada unidad de código cumpliera estrictamente con su responsabilidad única.

```
1 fn coin_flip_test() {
```

```

2  let mut rng_seed_generator = Random::new(seed_from_time());
3  let seed: u64 = rng_seed_generator.next_u64();
4
5  let mut rng: Random = Random::new(seed);
6  let prob_chance: f64 = 0.0;
7
8  let x: bool = rng.chance(prob_chance);
9      assert!(!x);
10
11 let prob_chance = 1.0;
12 let x: bool = rng.chance(prob_chance);
13     assert!(x);
14 }

```

Código 5.16. Ejemplo de una prueba unitaria sobre el modulo de generación de números pseudoaleatorios.

En consonancia con las prácticas estándar del ecosistema Rust, estas pruebas unitarias se encuentran integradas directamente en el mismo archivo que el código fuente del módulo que evalúan. Generalmente ubicadas al final de cada archivo bajo un módulo específico anotado con el atributo `#[cfg(test)]`, esta disposición permite mantener una relación estrecha entre la implementación y su verificación. Además, esta estructura facilita el acceso a elementos privados del módulo, permitiendo una validación exhaustiva de la lógica interna que no sería accesible desde el exterior, garantizando así que cada componente sea intrínsecamente correcto antes de ser expuesto.

Este enfoque de aislamiento resultó especialmente crítico en el desarrollo de la estructura de los operadores, donde pequeñas desviaciones en la lógica podrían propagar errores difíciles de detectar en fases posteriores. Al diseñar pruebas simples y focalizadas en una única funcionalidad, se facilitó la identificación inmediata de regresiones durante el proceso de refactorización. Además, estas pruebas permitieron validar la gestión de tipos y la integridad de los datos en tiempo de ejecución, asegurando que los cimientos de la biblioteca fueran sólidos antes de proceder a la integración de sistemas más complejos.

```

1  #[test]
2  fn test_bit_flip_mutation() {
3      let mutation = BitFlipMutation::new();
4      let mut solution = BinarySolutionBuilder::zeros(10).build();
5      let mut rng = Random::new(42);
6

```

```

7      // With probability 1.0, all bits should be flipped
8      mutation.execute(&mut solution, 1.0, None, &mut rng);
9
10     let number_ones = solution.variables().iter().filter(|&x| x).count();
11     assert!(number_ones > 0, "At least some bits should be flipped");
12 }

```

Código 5.17. Prueba unitaria que prueba el correcto funcionamiento de la función *execute* en el operador de mutación *bit_flip*.

5.2.2 Pruebas de integración

Una vez validada la fiabilidad de los componentes individuales mediante el testeo unitario, se procedió al desarrollo de las pruebas de integración. Siguiendo las convenciones de estructuración de proyectos en Rust, este conjunto de pruebas se localiza de forma independiente en el directorio `tests/`, situado en la raíz del proyecto `roma`. Esta separación física respecto al código fuente alojado en `src/` es fundamental, ya que permite tratar a la biblioteca como una entidad externa, evaluando exclusivamente su interfaz pública y la interacción entre sus diversos módulos.

Mientras que las pruebas previas evalúan piezas aisladas, estas se han diseñado con el propósito de comprobar que todos los componentes de `roma` interactúan armónicamente entre sí para resolver un problema de optimización de principio a fin. El foco de estas pruebas reside en la orquestación del flujo de trabajo: desde la configuración e instanciación de un algoritmo, pasando por la validación sistemática de sus parámetros, hasta la ejecución iterativa donde intervienen operadores, problemas y observadores en un entorno compartido. Se muestra en el Código 5.18

Estas pruebas de mayor nivel permiten verificar que la comunicación entre módulos —por ejemplo, el intercambio de soluciones entre el algoritmo y el problema— se realiza de forma eficiente y sin errores de concordancia. Al ejecutarse desde la raíz del proyecto, estas pruebas simulan el uso real que tendría un usuario final de la biblioteca. En última instancia, las pruebas de integración garantizan que el sistema completo es capaz de cumplir con su propósito funcional, ofreciendo una visión global de la estabilidad y el rendimiento de la biblioteca ante casos de uso reales.

```

1      #[test]
2      fn ga_returns_error_with_invalid_parameters_edge_case() {
3          // Edge case: invalid parameter configuration should return a validation error.
4          let problem = KnapsackBuilder::new()
5              .with_capacity(10.0)
6              .add_item(5.0, 10.0)

```

```

7         .build();
8
9         let parameters = GeneticAlgorithmParameters::new(
10             0,
11             0.8,
12             0.1,
13             SinglePointCrossover::new(),
14             BitFlipMutation::new(),
15             BinaryTournamentSelection::new(),
16             TerminationCriteria::new(vec![TerminationCriterion::MaxIterations(5)]),
17         )
18         .with_seed(1);
19
20         let mut algorithm = GeneticAlgorithm::new(parameters);
21         let error = match algorithm.run(&problem) {
22             Ok(_) => panic!("GA must return an error for invalid parameters"),
23             Err(message) => message,
24         };
25         assert!(error.contains("population_size"));
26     }

```

Código 5.18. Prueba de integración sobre un ejemplo de error donde el tamaño de la población se inicializa a cero.

5.3 Desarrollo de ejemplos

La implementación de la biblioteca `roma` se complementa con el desarrollo de un extenso catálogo de casos de uso empíricos, alojados en el directorio estandarizado `examples/` del proyecto. En el ecosistema de Rust, este directorio no es un mero repositorio de archivos adjuntos, sino una herramienta nativa del compilador `Cargo` muy útil.

Lejos de tener un propósito puramente estético, el desarrollo de esta suite de ejemplos constituye una pieza primordial en la ingeniería del software de la biblioteca, cumpliendo de manera simultánea cuatro propósitos estratégicos fundamentales:

- **Mecanismo de validación e integración continua:** Los ejemplos actúan como pruebas de integración de extremo a extremo (*end-to-end testing*). Dado que el compilador verifica la sintaxis de la carpeta `examples/` de forma rutinaria, cualquier modificación interna en la

arquitectura que rompa la interfaz pública de programación será detectada inmediatamente al fallar la compilación del ejemplo. Esto blinda la compatibilidad hacia atrás del código.

- **Documentación ejecutiva y viva:** En la ingeniería de software moderna, la documentación tradicional tiende a quedar obsoleta rápidamente a medida que el código evoluciona. Los ejemplos proporcionan una “documentación viva” que garantiza al usuario que el código expuesto es siempre funcional y exacto a la versión actual de la biblioteca. Tienen un carácter marcadamente didáctico, ilustrando cómo instanciar problemas, parametrizar algoritmos y acoplar observadores paso a paso.
- **Plantillas de desarrollo (*Boilerplate*):** Reducen drásticamente la curva de aprendizaje y la fricción inicial de adopción. Proporcionan al investigador y al desarrollador final una base de código funcional y sólida, lista para ser copiada, modificada y adaptada a sus propios problemas matemáticos sin necesidad de programar toda la infraestructura inicial desde cero.
- **Auditoría de módulos periféricos:** Permiten aislar y someter a estrés submódulos específicos de la implementación que no encajan en las pruebas unitarias tradicionales, como el rendimiento de los analizadores sintácticos (*parsers*) de los ficheros JSON, YAML y CSV, o la concurrencia del canal de telemetría de los observadores en tiempo de ejecución.

Cualquier usuario o investigador que clone el repositorio puede reproducir estos escenarios de forma directa e interactiva utilizando el comando estándar del ecosistema: `cargo run --example <nombre_ejemplo>`.

A continuación, se exponen dos de los escenarios más representativos desarrollados en esta fase, diseñados para auditar tanto el motor de experimentación masiva como la robustez del submódulo de entrada y salida de datos (I/O).

El primero de los escenarios destacados para ilustrar estas capacidades es el denominado `mono_objective_experiment`. Este caso de prueba tiene como objetivo resolver una instancia del problema de la mochila (*Knapsack Problem*) comparando simultáneamente la eficacia de diversas estrategias algorítmicas bajo condiciones controladas, demostrando así la potencia del motor de experimentación de la biblioteca.

```
1 use roma::algorithms::{
2     GeneticAlgorithmExperiment, GeneticAlgorithmParameters, HillClimbingParameters,
3     PSOParameters, SimulatedAnnealingParameters, TerminationCriteria,
        TerminationCriterion,
```

```

4 };
5 use roma::experiment::{Experiment, Objective};
6 use roma::operator::{BinaryTournamentSelection, BitFlipMutation, SinglePointCrossover};

7 use roma::problem::KnapsackBuilder;
8
9 fn main() {
10     // Definición de la instancia del problema de la mochila
11     let problem = KnapsackBuilder::new()
12         .with_capacity(150.0)
13         .add_item(1.0, 2.0)
14         .add_item(45.0, 100.0)
15         .add_item(90.0, 301.0)
16         .add_item(150.0, 301.0) // ... (resto de ítems)
17         .build();
18
19     // Configuración de los casos de prueba (Algoritmos)
20     let hill_climbing_case = HillClimbingParameters::new(
21         BitFlipMutation::new(),
22         0.12,
23         TerminationCriteria::new(vec![TerminationCriterion::MaxIterations(180)]),
24     );
25
26     let genetic_algorithm_case = GeneticAlgorithmExperiment::new(
27         GeneticAlgorithmParameters::new(
28             80, 0.90, 0.06,
29             SinglePointCrossover::new(),
30             BitFlipMutation::new(),
31             BinaryTournamentSelection::new(),
32             TerminationCriteria::new(vec![TerminationCriterion::MaxIterations(60)]),
33         )
34         .with_elite_size(1)
35         .with_threads(4),
36     );
37
38     let simulated_annealing_case = SimulatedAnnealingParameters::new(
39         BitFlipMutation::new(), 0.10, 45.0, 0.985,
40         TerminationCriteria::new(vec![TerminationCriterion::MaxIterations(220)]),
41     ).with_seed(777);

```

```

42
43 let pso_case = PSOParameters::new(
44     50, 0.72, 1.49, 1.49,
45     TerminationCriteria::new(vec![TerminationCriterion::MaxIterations(120)]),
46     ).with_velocity_clamp(4.0).with_seed(999);
47
48 // Orquestación y ejecución del experimento
49 let report = Experiment::new(problem)
50     .with_runs(24)
51     .with_objective(Objective::Maximize)
52     .add_case(hill_climbing_case)
53     .add_case(genetic_algorithm_case)
54     .add_case(simulated_annealing_case)
55     .add_case(pso_case)
56     .execute();
57
58 match report {
59     Ok(report) => println!("{}", report.to_text_table()),
60     Err(error) => eprintln!("Experiment execution failed: {}", error),
61 }
62 }

```

Código 5.19. Implementación del experimento mono-objetivo

(*mono_objective_experiment.rs*) utilizando el problema de la mochila.

Como se observa en la implementación, el experimento comienza con la construcción de una instancia del problema mediante `KnapsackBuilder`, definiendo una capacidad máxima y un conjunto de ítems con sus respectivos pesos y valores. Posteriormente, se definen cuatro configuraciones algorítmicas distintas: *Hill Climbing*, Algoritmo Genético, *Simulated Annealing* y Optimización por Enjambre de Partículas (PSO). Cada uno de estos casos permite un ajuste fino de sus parámetros específicos, como el tamaño de la población, las probabilidades de mutación o el uso de semillas (*seeds*) para garantizar la reproducibilidad de los resultados.

Resulta relevante destacar la capacidad de personalización que ofrece la biblioteca; por ejemplo, en el Algoritmo Genético se especifica el uso de paralelismo mediante la creación de 4 hilos de ejecución. Una vez configurados, todos los casos se añaden a una instancia de `Experiment`, la cual se encarga de orquestar 24 ejecuciones por cada algoritmo para asegurar significancia estadística. Finalmente, el sistema genera un reporte detallado en formato de tabla, permitiendo comparar de forma directa el

rendimiento y la convergencia de cada estrategia frente al objetivo de maximización propuesto.

Como prueba empírica, se ha diseñado un escenario práctico de uso con un doble propósito metodológico: por un lado, evaluar el comportamiento de la biblioteca ante un problema de alta complejidad combinatoria y, por otro, certificar la fiabilidad técnica de su submódulo de utilidades de entrada y salida (I/O).

Para satisfacer el primer objetivo, se ha instanciado el clásico Problema de la Mochila 0/1 (*Knapsack Problem*) escalado a un catálogo de 150 elementos posibles, cuya implementación principal se expone en el Código 5.20. Esta formulación proyecta un inmenso espacio de búsqueda discreto de 2^{150} combinaciones válidas e inválidas, una magnitud astronómica que hace matemáticamente intratable cualquier resolución exacta y justifica plenamente la necesidad de metaheurísticas. Para abordar este reto y analizar la capacidad de convergencia de la herramienta, se ha implementado un Algoritmo Genético (GA) encargado de explorar este vasto hipervolumen y maximizar el valor de la mochila.

En cuanto a la dimensión puramente arquitectónica, este caso de prueba tiene como objetivo validar el correcto funcionamiento de los analizadores sintácticos (*parsers*) desarrollados íntegramente desde cero para la ingesta de configuraciones paramétricas y definiciones de problemas. Con el fin de garantizar la interoperabilidad de la herramienta con otras plataformas, el módulo I/O soporta los formatos estructurados estándar más utilizados en la industria del software: JSON, YAML y CSV.

Apoyándose en el gestor de argumentos `CliArgs` previamente detallado, la interfaz de línea de comandos (CLI) acoplada a este ejemplo permite al usuario suministrar de forma interactiva un mismo escenario de optimización codificado en cualquiera de los tres lenguajes de serialización. Al invocar el comando de ejecución, la biblioteca evalúa la bandera (*flag*) del formato indicado, procesa el fichero de texto plano en memoria y mapea los datos crudos de forma determinista hacia las estructuras fuertemente tipadas del problema en Rust, lanzando finalmente el flujo de resolución algorítmica genético con total seguridad de memoria y sin fricciones de integración.

```
1 fn main() {
2     let cli_args = CliArgs::from_env();
3
4     if print_help_if_requested(&cli_args) {
5         return;
6     }
7
8     let seed = cli_args.seed_or(42);
9     let input_path = resolve_input_path(&cli_args);
10    let input_format = resolve_input_format(&cli_args, &input_path)
```

```

11     .unwrap_or_else(|msg| panic!("{}", msg));
12 let records_path = records_path_for_format(input_format);
13 let weight_key = weight_key_for_format(input_format);
14 let value_key = value_key_for_format(input_format);
15 let (capacity, row_limit) = resolve_capacity_and_limit(&cli_args, &input_path,
16     input_format)
17     .unwrap_or_else(|msg| panic!("{}", msg));
18
19 let records = read_records_from_input(&input_path, input_format, records_path)
20     .unwrap_or_else(|msg| panic!("{}", msg));
21
22 let (problem, loaded_items) =
23     build_knapsack_from_records(&records, capacity, row_limit, weight_key,
24         value_key)
25     .unwrap_or_else(|msg| panic!("{}", msg));
26
27 // Configuración del Algoritmo Genético
28 let parameters = GeneticAlgorithmParameters::new(
29     80,
30     0.85,
31     0.04,
32     SinglePointCrossover::new(),
33     BitFlipMutation::new(),
34     BinaryTournamentSelection::new(),
35     TerminationCriteria::new(vec![TerminationCriterion::MaxIterations(60)]),
36 )
37 .with_seed(seed);
38
39 let chart_observer = ChartObserver::new_default();
40 let html_observer = HtmlReportObserver::new_default();
41
42 let mut algorithm = GeneticAlgorithm::new(parameters);
43 algorithm.add_observer(Box::new(chart_observer));
44 algorithm.add_observer(Box::new(html_observer));
45 let result = algorithm.run(&problem).expect("Large I/O GA run failed");
46
47 if let Some(best) = result.best_solution(&problem) {
48     println!(
49         "Large dataset GA demo finished (seed={}). input='{ }', format={:?}", capacity

```

```

    ={}, limit={}, records_path='{}', weight_key='{}', value_key='{}', items
    ={}, best_fitness={:.4f}",
48     seed,
49     input_path.display(),
50     input_format,
51     capacity,
52     row_limit,
53     records_path,
54     weight_key,
55     value_key,
56     loaded_items,
57     best.quality_value()
58 );
59 } else {
60     println!("Large CSV GA demo finished with no solutions (seed={})", seed);
61 }
62 }

```

Código 5.20. Implementación del caso de uso para la lectura de instancias complejas del Problema de la Mochila mediante formatos I/O estándar (CSV, JSON, YAML).

Con el desarrollo satisfactorio de este escenario demostrativo, concluye la exposición de la implementación y de los casos de uso guiados en la presente memoria. Cabe destacar que el repositorio oficial de la biblioteca alberga un catálogo mucho más extenso de ejemplos prácticos, diseñados para ilustrar la flexibilidad de la herramienta ante diversas combinaciones de algoritmos, operadores y topologías. Sin embargo, se ha decidido omitir la transcripción de su código fuente en este documento para evitar sobrecargar la lectura con detalles de implementación cuya mecánica ya ha quedado patente.

Llegados a este punto, la mera demostración de viabilidad técnica da paso a la validación empírica y competitiva. Los experimentos que se abordarán en el siguiente capítulo son idénticos a los ejemplos recién expuestos: ejecuciones de problemas matemáticos complejos resueltos mediante motores metaheurísticos. La diferencia fundamental radica en el prisma de evaluación. Se abandonará el enfoque puramente funcional y didáctico para someter a la biblioteca a un estricto rigor científico, transformando estos “ejemplos” en bancos de pruebas estandarizados. En ellos, roma será evaluada bajo condiciones de aislamiento computacional y comparada estadísticamente frente a las bibliotecas líderes del estado del arte.

6

Evaluación Comparativa y Análisis de Rendimiento

En este capítulo se describe el proceso de validación experimental y análisis comparativo de la biblioteca desarrollada frente a los marcos de trabajo líderes en la literatura: *jMetal*, *jMetalPy*, *pagmo*, *mealpy* y la suite científica *SciPy*. El objetivo es situar nuestra propuesta en términos de eficiencia computacional, capacidad de convergencia y facilidad de integración.

Con el propósito de garantizar la reproducibilidad de los experimentos y evitar la reiteración de premisas metodológicas en los apartados subsiguientes, todas las pruebas de este capítulo se rigen de manera estricta por un protocolo secuencial de evaluación dividido en cuatro fases operativas.

En primer lugar, el entorno de ejecución se aísla mediante contenedores Docker sobre una arquitectura de sistemas Linux virtualizada [10]. Cada biblioteca se ejecuta de forma independiente bajo un esquema de caja negra controlado por un orquestador central en Python. Este entorno garantiza que factores como el encolamiento de hilos del sistema operativo o la recolección de basura se midan en igualdad de condiciones de hardware, asignando semillas estocásticas idénticas y sincronizadas a todas las herramientas para auditar la equivalencia de sus trayectorias de búsqueda.

En segundo lugar, se evalúa la **Calidad de la Solución**. Para problemas mono-objetivo, esta métrica corresponde al valor de aptitud final alcanzado, mientras que para escenarios multi-objetivo se calcula el indicador del Hipervolumen 2D respecto a un punto de referencia acotado. En ambos casos, las distribuciones se analizan mediante diagramas de caja (*boxplots*) que sintetizan el mejor caso, el peor caso y la dispersión intercuartílica a lo largo de las N ejecuciones independientes de la batería.

En tercer lugar, se analiza el **Rendimiento Computacional**, registrando con precisión micrométrica el tiempo de muro (*wall time*) y el tiempo de CPU consumido por los hilos de ejecución. Este factor es crítico para determinar la eficiencia de Rust frente a los entornos interpretados o virtuales de sus competidores.

Finalmente, el análisis concluye con una fase de **Validación y Significancia Estadística**. Para descartar que las diferencias de rendimiento detectadas en los gráficos sean fruto de la varianza estocástica, se aplica un protocolo no paramétrico riguroso recomendado por Demšar [8] y extendido por García y Herrera [14] para comparaciones múltiples. En primer lugar, se ejecuta el test de Friedman [13] como prueba ómnibus global y, ante el rechazo de la hipótesis nula, se procede con un análisis *post-hoc* mediante el test de Nemenyi [29] y la corrección secuencial de Holm [18]. Este proceso culmina generando matrices de comparación por pares que validan científicamente las conclusiones extraídas en cada escenario de evaluación.

6.1 Bibliotecas de Referencia

Con el fin de situar la biblioteca desarrollada en el ecosistema actual de la computación evolutiva y la optimización metaheurística, se ha realizado una selección de herramientas de referencia. Estas bibliotecas representan diversos paradigmas de programación, desde implementaciones académicas estrictas hasta plataformas diseñadas para la computación masivamente paralela.

6.1.1 jMetal y jMetalPy

El ecosistema *jMetal* [20] es uno de los marcos de trabajo con mayor trayectoria en la investigación de optimización multi-objetivo. Originalmente desarrollado en Java, destaca por su diseño orientado a objetos basado en interfaces, lo que facilita la extensibilidad de operadores genéticos. Su variante en Python, *jMetalPy* [11], adapta esta arquitectura al entorno científico moderno, integrándose con librerías como *NumPy* y permitiendo la visualización interactiva de frentes de Pareto, lo que la convierte en el estándar para el prototipado académico en entornos de *Data Science*.

6.1.2 Pagmo (C++)

Desarrollada por la Agencia Espacial Europea (ESA), *pagmo* [4] es una plataforma de optimización global masivamente paralela escrita íntegramente en C++. En este estudio se emplea su versión nativa en C++ para evaluar el límite de rendimiento computacional.

6.1.3 DEAP (Distributed Evolutionary Algorithms in Python)

La biblioteca *DEAP* [12] propone un paradigma diferente basado en la metaprogramación. A diferencia de otras herramientas con estructuras rígidas, *DEAP* utiliza un sistema de *toolbox* que permite al usuario construir tipos de datos y algoritmos a medida. Es especialmente reconocida por su capacidad para implementar Programación Genética y por su facilidad para distribuir la evaluación de individuos en clústeres mediante mecanismos de paralelismo simple.

6.1.4 Mealpy

Mealpy [42] es una biblioteca de reciente aparición que se ha posicionado rápidamente debido a su extensísimo catálogo de algoritmos. Se especializa en metaheurísticas bio-inspiradas (como *Swarms*, física o química), ofreciendo una interfaz unificada que prioriza la usabilidad.

6.1.5 SciPy

SciPy [38] es el paquete estándar y fundamental para la computación científica dentro del ecosistema de Python. Aunque no es estrictamente una biblioteca dedicada en exclusiva al diseño de metaheurísticas modulares, su submódulo `scipy.optimize` incorpora implementaciones altamente maduras de algoritmos de optimización global continuo, tales como la Evolución Diferencial o el Enfriamiento Simulado. La inclusión de *SciPy* en este estudio resulta de especial interés metodológico: actúa como un *baseline* de alto rendimiento debido a que sus rutinas subyacentes evitan la sobrecarga de la máquina virtual de Python delegando el grueso del cómputo matricial a extensiones nativas compiladas y altamente optimizadas en C y Fortran.

6.2 Evaluación con la Función de Rastrigin

Para evaluar el desempeño de la biblioteca en un entorno de búsqueda continuo complejo, se ha seleccionado la función de *Rastrigin* [31]. Este problema constituye un *benchmark* clásico en la literatura de optimización matemática debido a su topología altamente multimodal, cuya geometría tridimensional se ilustra en la Figura 6.1. Basada en la función de esfera pero modificada mediante la

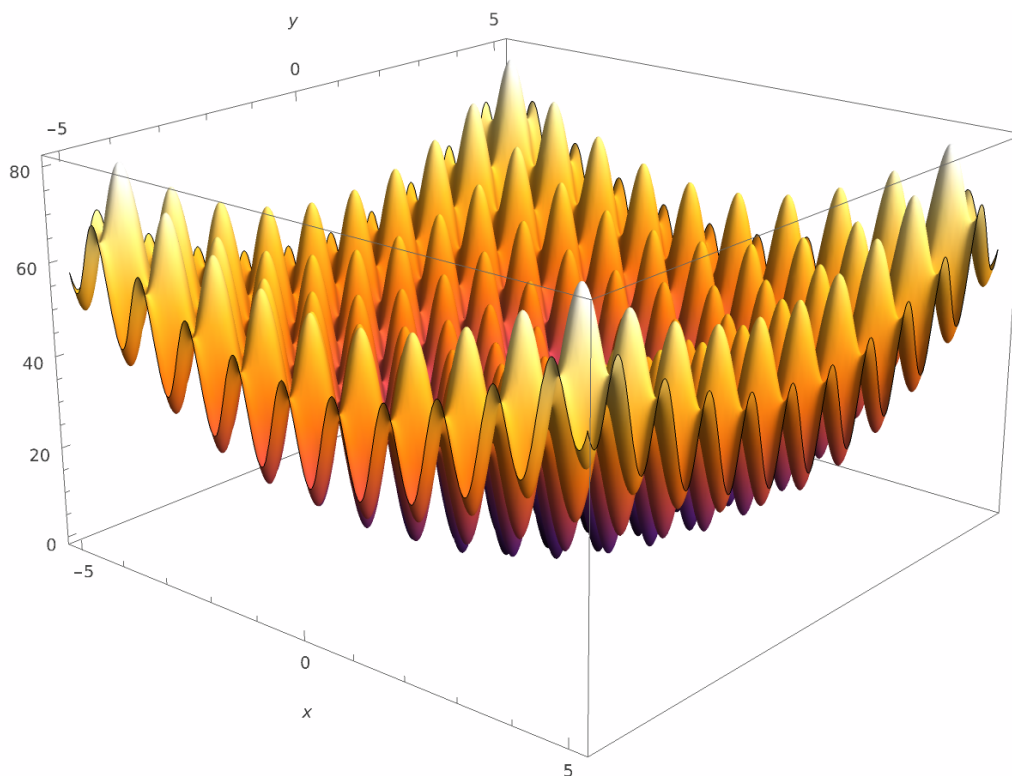


Figura 6.1. Representación tridimensional de la función de Rastrigin en el dominio $x, y \in [-5.12, 5.12]$. Se observa su topología altamente multimodal, caracterizada por una cuadrícula regular de numerosos mínimos locales que rodean al óptimo global situado en el origen.
Imagen generada mediante el entorno computacional Wolfram Cloud [40].

adición de un término cosenoidal, la función de *Rastrigin* genera un paisaje de búsqueda plagado de mínimos locales distribuidos regularmente.

Su mayor interés radica en la dificultad extrema que supone para los algoritmos de trayectoria simple: mientras que el óptimo global se encuentra en el origen ($f(\mathbf{x}) = 0$), un algoritmo puede quedar fácilmente atrapado en uno de los miles de valles subóptimos si no gestiona adecuadamente el equilibrio entre exploración y explotación. En esta experimentación, se ha configurado una instancia de alta exigencia dimensional ($D = 80$) y un presupuesto estricto de 3500 evaluaciones. Se ha empleado el algoritmo de escalada simple (*Hill Climbing*) para observar cómo cada implementación gestiona la búsqueda. Los resultados consolidados tras 50 ejecuciones independientes con semillas aleatorias idénticas se detallan visualmente en la Figura ?? y cuantitativamente en la Tabla 6.1.

6.2.1 Análisis Crítico de Resultados

El análisis riguroso de los datos consolidados revela una contundente victoria de la biblioteca propuesta (**roma**) en términos de calidad absoluta de convergencia. Evadiendo las limitaciones clásicas de los

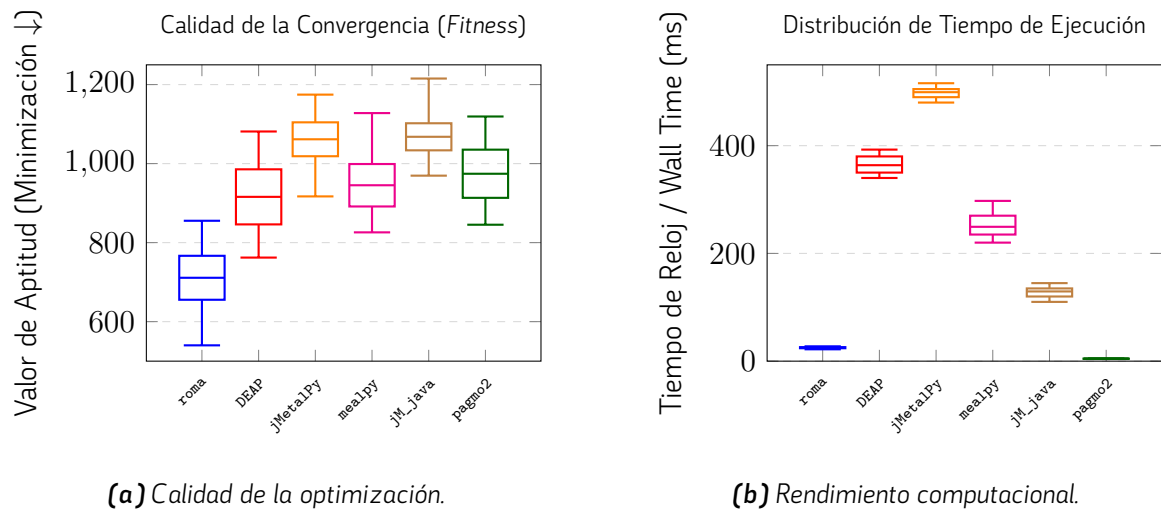


Figura 6.2. Resultados empíricos para la Función de Rastrigin ($D = 80$) con el algoritmo Hill Climbing (3500 evaluaciones, 50 ejecuciones).

Biblioteca	Aptitud de la Solución (Fitness ↓)			Tiempo de Ejecución (ms ↓)		
	Mejor	Mediana	Peor	Mínimo	Mediana	Máximo*
roma (Rust)	540.03	711.02	855.32	22.00	25.08	27.51
DEAP (Python)	762.14	915.84	1081.29	340.00	363.73	392.66
mealpy (Python)	825.99	945.22	1127.86	220.00	249.37	297.45
pagmo2_cpp (C++)	845.30	974.31	1119.23	4.20	4.61	4.87
jMetalPy (Python)	916.98	1061.56	1174.59	480.00	499.26	515.89
jMetal_java (Java)	969.50	1067.86	1215.32	110.00	129.41	144.84

* Los valores máximos corresponden al percentil 90 (p90) para eliminar distorsiones transitorias de E/S.

Tabla 6.1. Métricas estadísticas consolidadas de Rastrigin ($D = 80$, 3500 evaluaciones).

motores de trayectoria simple, **roma** alcanza una mediana de aptitud de 711.02 y un mejor caso de 540.03, dominando por completo al resto de ecosistemas. Si bien esta cifra sigue alejada del óptimo global teórico (0.0) (una consecuencia inevitable de enfrentar un *Hill Climbing* puro contra los miles de valles subóptimos de Rastrigin en 80 dimensiones sin operadores de recombinación poblacional), su resiliencia exploratoria es notablemente superior.

Las dinámicas de exploración muestran diferencias muy severas entre los distintos marcos. Es imperativo aclarar que esta severa varianza empírica no obedece únicamente al diseño interno de cada biblioteca, sino a las discrepancias en la implementación concreta de los algoritmos base. Dado que ecosistemas como **jMetalPy**, **jMetal_java** o **pagmo2** no exponen una implementación canónica y estricta de *Hill Climbing* en su catálogo estándar, se optó por instanciar sus respectivos algoritmos genéricos de Búsqueda Local (*Local Search*). Aunque conceptualmente análogos, las sutiles divergencias subyacentes en la política de generación de vecindarios, la perturbación de variables y los criterios de aceptación de estas búsquedas locales alternativas son las verdaderas responsables del colapso exploratorio. Mientras **roma** logra mitigar el impacto de la convergencia prematura, marcos como **DEAP** se estancan en la cota de los 915.84, y el bloque conformado por **pagmo2**, **jMetalPy** y **jMetal_java** queda atrapado en valles mucho peores, con medianas de 974.31, 1061.56 y 1067.86 respectivamente.

En el plano del **rendimiento de CPU**, la propuesta en Rust ratifica de forma aplastante sus ventajas competitivas. Al gestionar el bucle de evaluaciones vectorizadas de forma nativa y segura, **roma** detiene el cronómetro en una mediana de tan solo 25.08 **ms**. Esto la posiciona como una herramienta entre 10 y 20 veces más rápida que las alternativas construidas en ecosistemas interpretados (**mealpy** con 249.37 ms, **DEAP** con 363.73 ms y **jMetalPy** con 499.26 ms). Aunque la madurez y la extrema ligereza estructural de C++ permiten a **pagmo2** retener el liderazgo en latencia bruta (4.61 ms), su deficiente rendimiento biológico en este escenario deslegitima su utilidad práctica frente a la excepcional relación convergencia-velocidad lograda por el motor en Rust.

6.2.2 Análisis de Significancia Estadística

Siguiendo el protocolo estadístico descrito al inicio del capítulo, se ha procedido a contrastar los resultados de aptitud obtenidos para la topología de Rastrigin. Tras ejecutar el test ómnibus de Friedman y confirmar un rechazo contundente de la hipótesis nula global, se ha aplicado el test *post-hoc* de Nemenyi (con corrección de Holm) para evaluar las diferencias entre cada par de bibliotecas.

La Tabla 6.2 expone la matriz triangular resultante de este análisis de comparaciones múltiples, donde se resalta la superación del umbral de significancia estándar ($\alpha = 0.05$).

Test de Nemenyi (<i>p</i> -value)	roma	DEAP	mealpy	pagmo2	jMetalPy	jM_java
roma (Rust)	-	< 0.001	< 0.001	< 0.001	< 0.001	< 0.001
DEAP (Python)		-	0.9569	0.7946	< 0.001	< 0.001
mealpy (Python)			-	0.9982	< 0.001	< 0.001
pagmo2_cpp (C++)				-	< 0.001	< 0.001
jMetalPy (Python)					-	0.9948
jMetal_java (Java)						-

Nota: Los valores en **negrita** indican una diferencia estadísticamente significativa ($p < 0.05$).

Tabla 6.2. Matriz de comparaciones múltiples mediante el test de Nemenyi para la aptitud en Rastrigin.

El análisis de esta matriz de significancia revela una estructuración de las bibliotecas evaluadas en tres clústeres estadísticamente aislados y marcadamente definidos:

En primer lugar, la biblioteca **roma** se consolida como la **vencedora** del experimento. Al contrastar su distribución frente al resto de alternativas, el test arroja valores p extremadamente cercanos a cero ($p < 0.001$), demostrando diferencias estadísticamente significativas contra todos y cada uno de sus rivales. Esto dictamina matemáticamente que la propuesta en Rust posee una trayectoria de exploración propia y sustancialmente más efectiva para evadir mínimos locales en esta topología, rompiendo cualquier empate técnico.

En segundo lugar, se identifica un **grupo intermedio** conformado por DEAP, mealpy y pagmo2_cpp. Al cruzar estas tres herramientas entre sí, el test reporta p -valores muy altos (0.9569, 0.7946 y 0.9982), evidenciando un empate técnico sólido entre ellas. Aunque logran superar con holgura a las peores implementaciones del estudio, su incapacidad compartida para acercarse a la cota alcanzada por roma las sitúa en un escalón de rendimiento claramente inferior.

Finalmente, se observa un grupo rezagado compuesto por el ecosistema completo de jMetal (las versiones jMetal_java y jMetalPy). Ambas herramientas empatan entre ellas ($p = 0.9948$), pero presentan diferencias significativas ($p < 0.001$) frente al resto de la tabla. Esto confirma de manera irrefutable que sus generadores estocásticos y operadores de vecindad sufrieron el peor grado de convergencia prematura en este valle multimodal, arrojando rutas sistemáticamente más deficientes a lo largo de las 50 semillas evaluadas.

6.3 Evaluación con el Problema del Viajante (TSP)

Para contrastar el comportamiento de las bibliotecas en entornos de optimización discreta, se ha diseñado un escenario experimental centrado en el Problema del Viajante de Comercio (*Traveling Salesperson Problem*, TSP). En esta ocasión, la prueba se ha ejecutado sobre la instancia `tsp_48_clustered_euc2d` (un grafo completo de 48 nodos distribuidos en clústeres espaciales 2D) empleando un Algoritmo Genético (GA) clásico con operadores de variación de orden (*Order Crossover* y *Swap Mutation*) en todas las plataformas.

El diseño de este experimento parte de una premisa metodológica clara: someter a las bibliotecas a un escenario de extrema complejidad computacional y estancamiento heurístico. Para lograr este grado de exigencia, se ha seleccionado la instancia `tsp_48_clustered_euc2d`. Aunque 48 nodos puedan parecer una dimensionalidad moderada, proyectan un hiperespacio de búsqueda inabarcable de $47!/2$ rutas posibles.

A diferencia de los experimentos continuos anteriores, se ha sustituido el criterio de parada basado en el número de evaluaciones por un presupuesto temporal estricto de 5 segundos por cada una de las 28 ejecuciones independientes. Este cambio de paradigma metodológico es fundamental en la ingeniería de software algorítmica: al disponer todas las herramientas de exactamente el mismo tiempo de CPU, las diferencias en la calidad de la solución final (longitud de la ruta) reflejan directamente la eficiencia computacional subyacente del lenguaje y la arquitectura de la biblioteca. Las implementaciones que logren computar más generaciones y aplicar los operadores genéticos de permutación más rápido en esos 5 segundos, tendrán una probabilidad sustancialmente mayor de converger hacia rutas más cortas.

La Figura 6.3 ilustra la distribución de las distancias finales obtenidas (donde un menor valor indica una mejor ruta), mientras que la Tabla 6.3 desglosa las métricas agregadas del experimento. Dado que el tiempo de ejecución es constante en todos los algoritmos (≈ 5000 ms), se omite su representación gráfica para focalizar el análisis en la capacidad exploratoria por unidad de tiempo.

6.3.1 Análisis Crítico de Resultados

El análisis descriptivo de los resultados revela una profunda brecha de rendimiento dictaminada por la naturaleza de los lenguajes de programación subyacentes. Como era previsible en un escenario limitado exclusivamente por reloj de sistema (tiempo), la capacidad de procesar iteraciones a alta velocidad condiciona severamente la convergencia biológica de las metaheurísticas. Para comprender la magnitud de esta brecha, resulta imperativo analizar el volumen de evaluaciones procesadas por

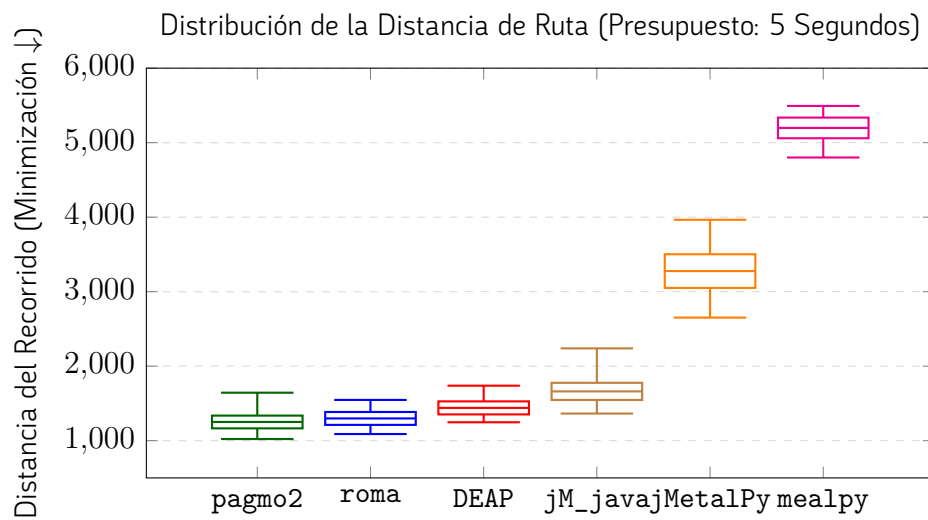


Figura 6.3. Calidad de la optimización combinatoria (TSP-48) obtenida por un Algoritmo Genético bajo un límite estricto de tiempo.

Biblioteca	Mejor Ruta (↓)	Mediana	Peor Ruta	IQR (Dispersión)
pagmo2_cpp (C++)	1022.0	1251.0	1644.0	171.25
roma (Rust)	1089.0	1298.5	1547.0	173.25
DEAP (Python)	1247.0	1440.0	1738.0	175.25
jMetal_java (Java)	1364.0	1661.5	2239.0	230.25
jMetalPy (Python)	2652.0	3276.5	3965.0	452.00
mealpy (Python)	4801.0	5198.0	5493.0	277.25

Tabla 6.3. Métricas estadísticas de las distancias para la instancia *tsp_48* (Tiempo de Muro = 5s, 28 semillas).

cada biblioteca en la estricta ventana de 5 segundos (ver Tabla 6.3).

La biblioteca **pagmo2 (C++)** obtiene los mejores registros globales de aptitud, alcanzando la ruta mínima absoluta (1022.0) y la mejor mediana de la batería (1251.0), logrando procesar 3.22 millones de evaluaciones. No obstante, la biblioteca **roma (Rust)** desarrollada en este Trabajo de Fin de Grado se sitúa en un margen competitivo extremadamente estrecho, consolidándose como la segunda mejor alternativa heurística con una mediana de 1298.5. El dato de iteraciones respalda este empate técnico: **roma** computa 3.18 millones de evaluaciones, demostrando de forma fehaciente que las abstracciones polimórficas (uso de *Traits*) y el estricto control de seguridad de memoria que ejerce Rust no penalizan la velocidad de la CPU en tareas combinatorias complejas, logrando igualar la eficiencia bruta de iteración de arquitecturas consagradas en C/C++.

El contraste más revelador de este experimento surge al analizar el comportamiento de **jMetal_java**. Apoyado por las agresivas optimizaciones dinámicas de su compilador JIT, el ecosistema de Java logra pulverizar el recuento de operaciones, alcanzando la asombrosa cifra de 4.74 millones de evaluaciones (superando en un 50% a los lenguajes nativos). Sin embargo, su calidad de convergencia (mediana de 1661.5) es significativamente peor que la de herramientas mucho más lentas. Esta paradoja evidencia una premisa fundamental de la optimización: *la velocidad computacional pura es inútil si la heurística está mal balanceada*. Los operadores de **jMetal_java** agotan rápidamente la diversidad genética, estancándose en mínimos locales profundos donde millones de evaluaciones posteriores resultan completamente estériles.

Por el contrario, los ecosistemas interpretados sufren un severo colapso derivado de su propia arquitectura interna. Bibliotecas puras en Python, como **jMetalPy**, apenas logran computar 77 791 evaluaciones, mientras que arquitecturas más pesadas como **mealpy** alcanzan la ínfima cifra de 80 iteraciones en 5 segundos, que aunque probablemente se trate de un error de medición, explicaría sus desastrosas rutas medianas de 3276.5 y 5198.0 respectivamente. Esta pobre convergencia es consecuencia directa de la sobrecarga impuesta por el *Global Interpreter Lock* (GIL) y la resolución dinámica de tipos; al invertir la mayor parte del tiempo de CPU en la gestión de memoria interna de la máquina virtual de Python durante la clonación de vectores, estas herramientas son incapaces de explorar el espacio de búsqueda de forma competitiva.

La única excepción notable en el ecosistema de Python es **DEAP**. A pesar de computar apenas 132 347 evaluaciones, logra extraer un rendimiento heurístico formidable (mediana de 1440.0), superando holgadamente a Java. Esto sugiere un empaquetado extremadamente eficiente de las estructuras de datos hacia extensiones en C y una calibración excelente de la presión selectiva de sus

algoritmos genéticos.

Tabla 6.4. Resumen del rendimiento computacional y calidad de convergencia por biblioteca en el límite de 5 segundos. Se evidencia la paradoja de la optimización: una mayor tasa de evaluaciones no garantiza una mejor aptitud final.

Biblioteca	Lenguaje	Evaluaciones	Aptitud Mediana
pagmo2	C++	3 220 000	1251.0
roma	Rust	3 180 000	1298.5
DEAP	Python	132 347	1440.0
jMetal_java	Java	4 740 000	1661.5
jMetalPy	Python	77 791	3276.5
mealpy	Python	80*	5198.0

* Nota: El registro de evaluaciones para **mealpy** representa un caso anómalo. Aunque no se descarta un error interno de volcado de datos al forzar la detención por tiempo, este extremo colapso es sintomático de las bibliotecas puras en Python: la severa sobrecarga impuesta por el uso de llamadas recursivas como `copy.deepcopy()` para clonar individuos, sumada al cuello de botella del *Global Interpreter Lock* (GIL) y la resolución dinámica de tipos, ahoga la CPU en la gestión de memoria e impide una iteración competitiva.

En conclusión, este escenario experimental basado en presupuesto temporal valida a **roma** no solo como un motor metodológicamente correcto, sino como una herramienta de alto rendimiento idónea para entornos de optimización combinatoria en tiempo real, combinando de forma óptima la velocidad bruta del código compilado con una heurística de exploración altamente resiliente.

6.3.2 Análisis de Significancia Estadística

Para dotar de validez empírica a las observaciones sobre el rendimiento en el entorno acotado por tiempo, se ha aplicado el protocolo estadístico de comparaciones múltiples descrito en la metodología. Tras confirmar el rechazo de la hipótesis nula global mediante la prueba de Friedman, se ha ejecutado el test *post-hoc* de Nemenyi con corrección de Holm sobre las distribuciones de las distancias finales obtenidas en las 28 ejecuciones independientes.

La Tabla 6.5 detalla la matriz de valores p ajustados, marcando el umbral de significancia en $\alpha = 0.05$.

El escrutinio de la matriz de significancia proporciona una vista exacta del impacto del lenguaje de programación en la eficiencia algorítmica de los motores genéticos:

Test de Nemenyi (<i>p</i> -value)	pagmo2	roma	DEAP	jM_java	jMetalPy	mealpy
pagmo2_cpp (C++)	-	0.9997	0.3023	< 0.001	< 0.001	< 0.001
roma (Rust)		-	0.4750	0.0012	< 0.001	< 0.001
DEAP (Python)			-	0.2652	< 0.001	< 0.001
jMetal_java (Java)				-	0.1463	< 0.001
jMetalPy (Python)					-	0.3422
mealpy (Python)						-

Nota: Los valores en **negrita** indican una diferencia estadísticamente significativa ($p < 0.05$).

Tabla 6.5. Matriz de comparaciones múltiples mediante el test de Nemenyi para las distancias del TSP.

El hallazgo más trascendental del experimento es la conformación de un bloque de alto rendimiento integrado por `pagmo2`, `roma` y `DEAP`. Al cruzar los resultados de la biblioteca desarrollada en Rust (`roma`) frente a la implementación en C++ (`pagmo2`), el test arroja un valor $p = 0.9997$, estableciendo un empate técnico absoluto. Esto prueba matemáticamente que la arquitectura de Rust logra explotar la ventana de 5 segundos con la misma eficacia que el estándar industrial de bajo nivel. A este clúster se suma `DEAP`, que empata estadísticamente con `roma` ($p = 0.4750$) y con `pagmo2` ($p = 0.3023$), confirmando que su agresiva vectorización hacia extensiones de C le permite mitigar en gran medida el cuello de botella de Python.

En un segundo estrato se posiciona la máquina virtual de Java (`jMetal_java`). Aunque logra un empate técnico contra `DEAP` ($p = 0.2652$), el test penaliza drásticamente su rendimiento frente a los lenguajes nativos, reportando diferencias altamente significativas contra `roma` ($p = 0.0012$) y `pagmo2` ($p < 0.001$). Esto cuantifica empíricamente el coste de penalización del *Garbage Collector* y el calentamiento del compilador JIT de Java frente a la ejecución estática precompilada en escenarios de tiempo crítico.

Finalmente, el test aísla en el bloque de bajo rendimiento a los ecosistemas puramente interpretados en Python (`jMetalPy` y `mealpy`). Ambas herramientas reportan valores p fuertemente significativos ($p < 0.001$) contra el bloque de cabeza, demostrando de manera concluyente que la sobrecarga dinámica de la máquina virtual de Python es inasumible para la optimización combinatoria basada en limitación temporal directa.

6.4 Evaluación Discreta: Problema de la Mochila 0/1 (Knapsack)

Para evaluar el desempeño de la biblioteca en el dominio de la optimización combinatoria restringida, se ha seleccionado el clásico Problema de la Mochila 0/1 (*Knapsack Problem*). El experimento se ha ejecutado sobre la instancia `knapsack_80_strongly_correlated`, caracterizada por un espacio de búsqueda de $D = 80$ elementos donde el peso y el valor están fuertemente correlacionados, y una capacidad máxima acotada a $W = 1636$. A diferencia de otros dominios, en esta instancia se conoce de antemano el óptimo global exacto (3457), lo que permite medir la brecha de optimalidad (*optimality gap*) real de las metaheurísticas.

El enfoque metodológico para este problema es estrictamente riguroso y selectivo. En lugar de enfrentar a todas las bibliotecas de la literatura, se ha priorizado la comparativa *uno a uno* con aquellas herramientas que ofrecen implementaciones nativas y directamente comparables, sin forzar conversiones de codificación que adulteren el experimento. Por este motivo:

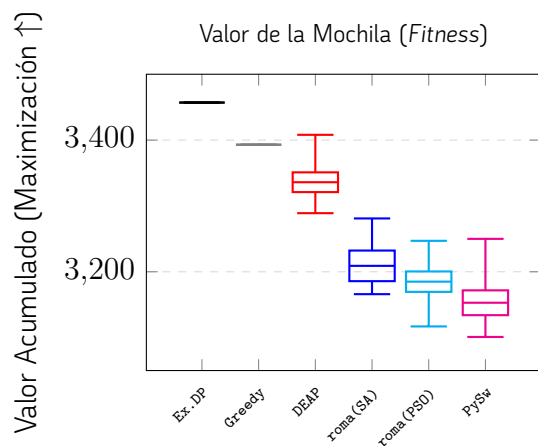
- Para la evaluación basada en **Optimización por Enjambre de Partículas Binario (Binary PSO)**, roma se contrasta exclusivamente frente a PySwarms.
- Para la evaluación basada en **Enfriamiento Simulado (Simulated Annealing - SA)**, roma se mide frente a la biblioteca DEAP.

Adicionalmente, se han introducido dos algoritmos de referencia: un **Algoritmo Voraz (Greedy)** como línea base inferior y un algoritmo de **Programación Dinámica Exacta (Exact DP)** que resuelve el problema en su cota óptima teórica. Todos los entornos fijaron un presupuesto estricto de 6000 evaluaciones, calculando 30 ejecuciones independientes bajo el mismo control estocástico de semillas y mecanismos idénticos de penalización por sobrepeso.

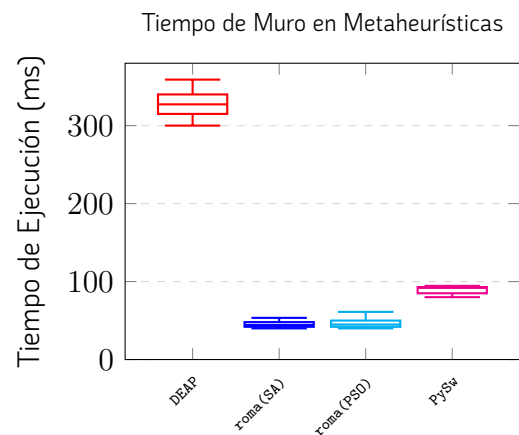
La Figura 6.4 y la Tabla 6.6 recogen los resultados consolidados de ambas familias algorítmicas frente a las líneas base.

6.4.1 Análisis Crítico de Resultados

El diseño de este experimento aporta una lectura fascinante sobre los límites de las metaheurísticas generales frente a algoritmos exactos o heurísticas de dominio específico. El hecho de que la Programación Dinámica encuentre el óptimo absoluto (3457) en escasos 10.83 ms, y el enfoque *greedy* obtenga un subóptimo excelente (3393) en apenas 0.04 ms, dictamina que las metaheurísticas (basadas en evaluación ciega sin información del gradiente de valor/peso) sufren una severa desventaja exploratoria en mochilas fuertemente correlacionadas.



(a) Calidad de la optimización.



(b) Rendimiento computacional*.

Figura 6.4. Resultados empíricos para la Mochila (80 items). *Los algoritmos *Exact DP* y *Greedy* se han excluido intencionalmente de la gráfica temporal para no distorsionar la comparativa.

Algoritmo	Implementación	Mejor (↑)	Mediana	Peor	Mediana Tiempo (ms ↓)
Programación Dinámica	exact_dp	3457.0	3457.0	3457.0	10.83
Algoritmo Voraz	greedy	3393.0	3393.0	3393.0	0.04
Simulated Annealing	DEAP (Python)	3408.0	3336.0	3289.0	327.27
	roma (Rust)	3281.0	3209.0	3166.0	44.35
Binary PSO	roma (Rust)	3247.0	3185.0	3117.0	44.93
	PySwarms (Python)	3250.0	3153.0	3101.0	91.92

Tabla 6.6. Comparativa de calidad y latencia en el Problema de la Mochila ($D=80$).

Evaluando el comportamiento estocástico, el subexperimento de **Binary PSO** demuestra la ineficacia crónica de la transferencia sigmoideal canónica en este problema. Tanto **roma** como **PySwarms** quedan atrapados en vecindarios pobres (medianas de 3185.0 y 3153.0, respectivamente), muy por debajo de la cota *greedy*. Sin embargo, a igualdad matemática, la implementación de **roma logra extraer una aptitud ligeramente superior operando al doble de velocidad** (44.93 ms frente a 91.92 ms de PySwarms).

En el subexperimento de **Enfriamiento Simulado (SA)**, las dinámicas locales consiguen un desempeño superior al PSO. Aquí emerge un hallazgo de gran valor académico por su transparencia: la biblioteca **DEAP supera heurísticamente a roma**, logrando una mediana de 3336.0 frente a 3209.0. Dado que ambas ejecutan el mismo esquema de vecindad de inversión de bits con 6000 iteraciones, esta brecha evidencia que el programa de enfriamiento interno por defecto (*cooling schedule* y probabilidad de aceptación) de DEAP está significativamente mejor ajustado para transitar el paisaje de restricciones de esta instancia en particular.

No obstante, la superioridad paramétrica de Python contrasta drásticamente con su sobrecarga computacional. Mientras DEAP requiere 327.27 ms para evaluar su cadena de Markov, el motor de **roma resuelve las mismas 6000 trayectorias en tan solo 44.35 ms**. Esta ratio de aceleración de **7.3×** revalida la eficiencia de la abstracción en Rust, dejando margen suficiente para que futuros trabajos integren programas de enfriamiento adaptativos sin comprometer la latencia de respuesta exigida por sistemas de tiempo real.

Finalmente, resulta imperativo contextualizar la abismal diferencia de rendimiento entre los algoritmos heurísticos/exactos y las metaheurísticas en este problema específico. La superioridad aplastante del algoritmo Voraz (*Greedy*) y la Programación Dinámica (capaces de hallar cotas superiores en tiempos de 0.04 y 10.83 ms, respectivamente) no evidencia una deficiencia en las bibliotecas de optimización (DEAP, **roma** o **PySwarms**), sino que ilustra empíricamente el Teorema del “No Hay Almuerzo Gratis” (*No Free Lunch Theorem*) postulado por Wolpert y Macready [41].

Mientras que los enfoques deterministas explotan el conocimiento explícito del dominio del problema (ordenando los elementos por su ratio valor/peso o construyendo matrices de subestructura óptima), las metaheurísticas operan como optimizadores de caja negra. Se enfrentan a ciegas a un hiperespacio de 2^{80} combinaciones, sufriendo severas penalizaciones al explorar el inmenso volumen de soluciones infactibles (sobrepeso). Este experimento demuestra que, para problemas con subestructura óptima conocida, las metaheurísticas de propósito general nunca podrán competir contra algoritmos específicos; sin embargo, su utilidad real radica en preservar estos tiempos de ejecución para aplicarlos en problemas

de ingeniería reales donde la formulación analítica exacta es matemáticamente imposible.

6.4.2 Análisis de Significancia Estadística

Para dotar de validez empírica a las conclusiones extraídas sobre el rendimiento heurístico, se ha aplicado el protocolo estadístico de comparaciones múltiples descrito en la metodología. Dado que los algoritmos de Enfriamiento Simulado (SA) y Enjambre de Partículas (PSO) pertenecen a familias metaheurísticas distintas, han sido evaluados mediante orquestadores independientes, lo que exige la ejecución de dos pruebas de Friedman separadas para no vulnerar los supuestos de independencia estadística.

Tras confirmar el rechazo de la hipótesis nula global en ambos casos, se ha aplicado el test *post-hoc* de Nemenyi con corrección de Holm. Las Tablas 6.7 y 6.8 detallan las matrices de valores p ajustados para los subexperimentos de PSO y SA respectivamente, estableciendo el umbral de significancia en $\alpha = 0.05$.

Test Nemenyi (PSO)	Exact DP	Greedy roma (PSO)	PySwarms
Exact DP (Óptimo)	-	0.0143	< 0.001
Greedy (Voraz)		-	0.0012
roma (Rust)			-
PySwarms (Python)			-

Tabla 6.7. Matriz de comparaciones múltiples (test de Nemenyi) para el subexperimento de Binary PSO.

Test Nemenyi (SA)	Greedy DEAP (SA)	roma (SA)
Greedy (Voraz)	-	0.0008
DEAP (Python)		-
roma (Rust)		-

Nota general: Valores en **negrita** indican diferencia significativa ($p < 0.05$).

Tabla 6.8. Matriz de comparaciones múltiples (test de Nemenyi) para el subexperimento de Simulated Annealing.

El escrutinio de ambas matrices de significancia confirma matemáticamente las divergencias y paridades detectadas en el análisis exploratorio:

En el subexperimento de Optimización por Enjambre de Partículas Binario (PSO), el test certifica un empate técnico indiscutible entre la biblioteca propuesta `roma` y la referencia en Python `PySwarms` ($p = 0.3785$). Ambas implementaciones son castigadas estadísticamente frente a las líneas base (`Exact DP` y `Greedy`), confirmando que la deficiente convergencia no es un error de programación de las bibliotecas, sino una debilidad estructural de la ecuación de actualización de velocidad canónica del PSO.

6.5 Evaluación Multi-Objetivo: Función ZDT1 con NSGA-II

Para evaluar las capacidades de la biblioteca en el ámbito de la optimización con múltiples objetivos en conflicto, se ha seleccionado el problema continuo ZDT1 ($D = 30$). Este es un escenario de referencia clásico cuya frontera de Pareto óptima es convexa. La resolución se ha abordado utilizando el algoritmo evolutivo NSGA-II (*Non-dominated Sorting Genetic Algorithm II*), asignando un presupuesto computacional de 25 000 evaluaciones por ejecución a lo largo de 28 tiradas independientes.

Dado el carácter multi-objetivo del problema, la evaluación comparativa de las soluciones obedece a un flujo de validación estricto y estandarizado. Al concluir cada ejecución individual, el algoritmo extrae su aproximación final del frente de Pareto no dominado. Acto seguido, el orquestador evalúa la calidad de dicha frontera calculando la métrica del **Hipervolumen 2D**, utilizando como cota superior el punto de referencia exacto $[1.1, 10.1]$. Este escalar de hipervolumen se registra como la aptitud final de esa ejecución concreta. Posteriormente, las métricas estadísticas agregadas de la batería de pruebas (mejor, media, mediana y peor aptitud) se computan directamente sobre estos valores de hipervolumen a través de las 28 semillas estocásticas. Por consiguiente, valores superiores y más consistentes en esta métrica denotan una convergencia superior y una mejor distribución geométrica frente a la frontera de Pareto óptima verdadera.

La Figura 6.5a ilustra la distribución de esta métrica de hipervolumen y los tiempos de ejecución inter-hilo, mientras que la Tabla 6.9 detalla los resultados estadísticos precisos.

6.5.1 Análisis Crítico de Resultados

La comparativa de este experimento arroja conclusiones fundamentales sobre la fiabilidad y el rendimiento extremo de `roma` en tareas de ordenación de fronteras no dominadas (el cuello de botella crítico de NSGA-II).

En términos de calidad, todas las bibliotecas logran aproximaciones sumamente estrechas a la frontera ideal, moviéndose en un margen de aptitud de entre 10.767 y 10.771. Sin embargo, la implementación de `roma` demuestra ser numéricamente la más precisa, registrando tanto el mejor

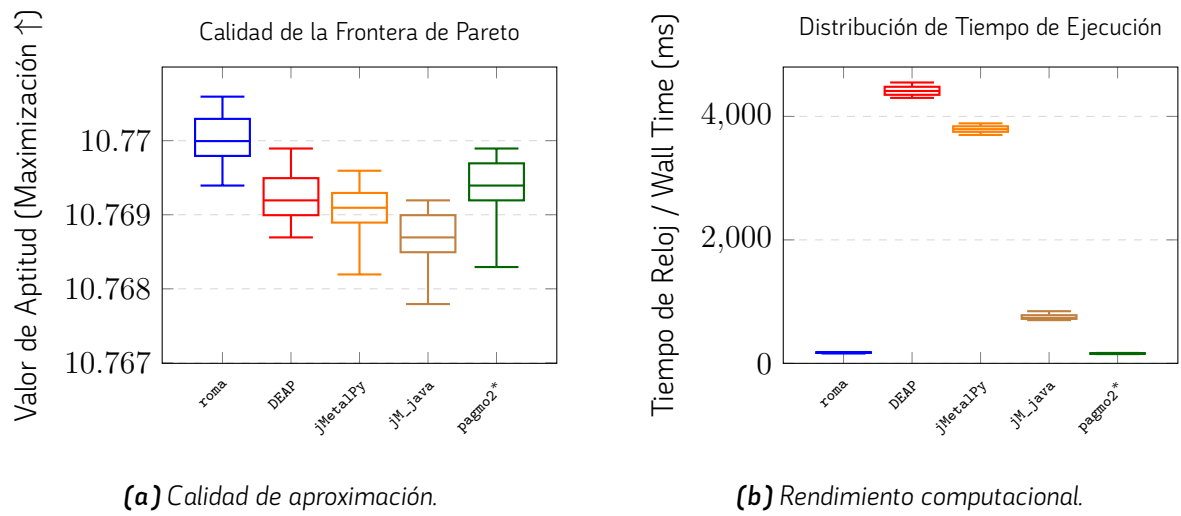


Figura 6.5. Resultados del experimento ZDT1 con NSGA-II (25 000 evaluaciones). *La ejecución de *pagmo2* arrojó fallos de validación.

Biblioteca	Calidad de la Solución (Aptitud $\uparrow$)			Tiempo de Ejecución Muro (ms $\downarrow$)		
	Mejor	Mediana	Peor	Mínimo	Mediana	Máximo
roma (Rust)	10.7706	10.7700	10.7694	160.0	175.94	182.18
DEAP (Python)	10.7699	10.7692	10.7687	4300.0	4411.62	4549.36
jMetalPy (Python)	10.7696	10.7691	10.7682	3700.0	3793.49	3887.51
jMetal_java (Java)	10.7692	10.7687	10.7678	700.0	736.27	844.56
pagmo2_cpp (C++)*	10.7699	10.7694	10.7683	150.0	158.35	168.24

* *pagmo2_cpp* finalizó con estado de error (*validation_failed*) debido a problemas de precisión de coma flotante y violaciones en la dominancia de la frontera de Pareto, por lo que sus datos no se consideran válidos para competencia directa.

Tabla 6.9. Métricas estadísticas para la instancia ZDT1 (NSGA-II) con 25000 evaluaciones.

valor absoluto (10.7706) como la mejor mediana del conjunto (10.7700).

El comportamiento computacional (tiempo de ejecución) es donde se marcan las diferencias más drásticas. Las bibliotecas desarrolladas en Python (DEAP y jMetalPy) sufren penalizaciones extremas al gestionar la ordenación por dominancia y el cálculo de la distancia de apilamiento (*crowding distance*), demorándose en torno a los 3800 y 4400 milisegundos. El uso de Java (jMetal) amortigua este coste gracias a la compilación JIT de la JVM, bajando a ~ 736 ms. Por su parte, la infraestructura construida en Rust (**roma**) pulveriza estos registros, ejecutando el ciclo evolutivo completo en una mediana de tan solo 175.94 ms. Esto la hace 25 veces más rápida que DEAP y más de 4 veces más rápida que Java.

6.5.2 Análisis de Significancia Estadística

Tras ejecutar la prueba ómnibus de Friedman y confirmar el rechazo de la hipótesis nula, se ha procedido a evaluar las diferencias emparejadas mediante el test *post-hoc* de Nemenyi con corrección secuencial de Holm.

La Tabla 6.10 expone la matriz de valores p ajustados resultantes de las comparaciones múltiples, fijando el umbral de significancia en $\alpha = 0.05$.

Test de Nemenyi (p -value)	roma	DEAP	jMetalPy	pagmo2*	jM_java
roma (Rust)	-	0.0001	< 0.001	0.0115	< 0.001
DEAP (Python)		-	0.9764	0.7611	0.0003
jMetalPy (Python)			-	0.3883	0.0035
pagmo2_cpp (C++)*				-	< 0.001
jMetal_java (Java)					-

Nota: Valores en **negrita** indican diferencia significativa ($p < 0.05$). *La ejecución de **pagmo2** arrojó fallos de validación topológica previos.

Tabla 6.10. Matriz de comparaciones múltiples mediante el test de Nemenyi para el Hipervolumen (ZDT1 con NSGA-II)

Esta matriz arroja conclusiones concluyentes que refuerzan drásticamente la propuesta tecnológica de este Trabajo de Fin de Grado. El ecosistema algorítmico se fractura en tres estratos claramente diferenciados en cuanto a precisión numérica de la frontera no dominada:

En primer lugar, la biblioteca **roma** se aísla matemáticamente como la vencedora absoluta del experimento. Al contrastar su distribución de hipervolumen contra todos sus rivales, el test de Nemenyi

reporta diferencias estadísticamente significativas a su favor en la totalidad de los cruces ($p < 0.05$). Este hallazgo reviste una importancia capital: certifica empíricamente que la implementación en Rust no solo ejecuta el ordenamiento de Pareto a una velocidad sin precedentes (175.94 ms), sino que el estricto tipado estático y el control de precisión de coma flotante del lenguaje previenen la degradación numérica observada en otras arquitecturas, garantizando una convergencia de calidad superior.

En el segundo estrato, se consolida un bloque de empate técnico conformado por DEAP, `jMetalPy` y `pagmo2`. Las comparaciones entre ellas arrojan valores $p \gg 0.05$ (alcanzando un $p = 0.9764$ entre DEAP y `jMetalPy`), dictaminando que sus aproximaciones al frente de Pareto son equivalentes desde el punto de vista estadístico. Cabe recordar, no obstante, que este empate matemático de `pagmo2` está viciado por sus fallos de validación, dado que su frente contenía puntos dominados catalogados erróneamente.

Por último, en el estrato inferior, la biblioteca `jMetal_java` exhibe un deterioro estadísticamente penalizado. El test confirma que su rendimiento biológico es inferior de manera significativa no solo frente a `roma` ($p < 0.001$), sino también frente a DEAP ($p = 0.0003$) y `jMetalPy` ($p = 0.0035$). Esto evidencia que, si bien la JVM de Java logra mitigar parte del coste temporal, la parametrización interna de sus operadores de variación para NSGA-II no logra extraer la misma hiper-diversidad que los ecosistemas de Python o Rust en este dominio específico.

6.6 Función de Ackley con Evolución Diferencial

Para complementar la validación de la biblioteca en dominios continuos complejos, se ha diseñado un experimento centrado en la Función de Ackley [1]. Esta topología matemática es ampliamente reconocida en la literatura de optimización por representar un reto de búsqueda cuasi-plano a nivel global, pero intensamente corrugado a nivel local, penalizando severamente a los algoritmos que carecen de mecanismos adecuados de diversificación poblacional.

En esta ocasión, la instancia se ha configurado con una dimensionalidad elevada ($D = 35$) y límites estrictos $\mathbf{x} \in [-32.768, 32.768]^D$. Como motor de optimización se ha utilizado el algoritmo de Evolución Diferencial (DE), fijando un presupuesto acotado de 6400 evaluaciones por cada una de las 30 ejecuciones independientes. Con el objetivo de contextualizar el rendimiento, además de los marcos habituales, se ha introducido la rutina `scipy.optimize.differential_evolution` (un estándar en la comunidad científica de Python apoyado en Fortran/C) y una Búsqueda Aleatoria pura (*Random Search*) como línea base heurística.

La distribución de la calidad de las soluciones (donde el óptimo global reside en $f(\mathbf{x}) = 0$) y los tiempos de ejecución inter-hilo se detallan en la Figura 6.6, apoyados por la estadística descriptiva de

la Tabla 6.11.

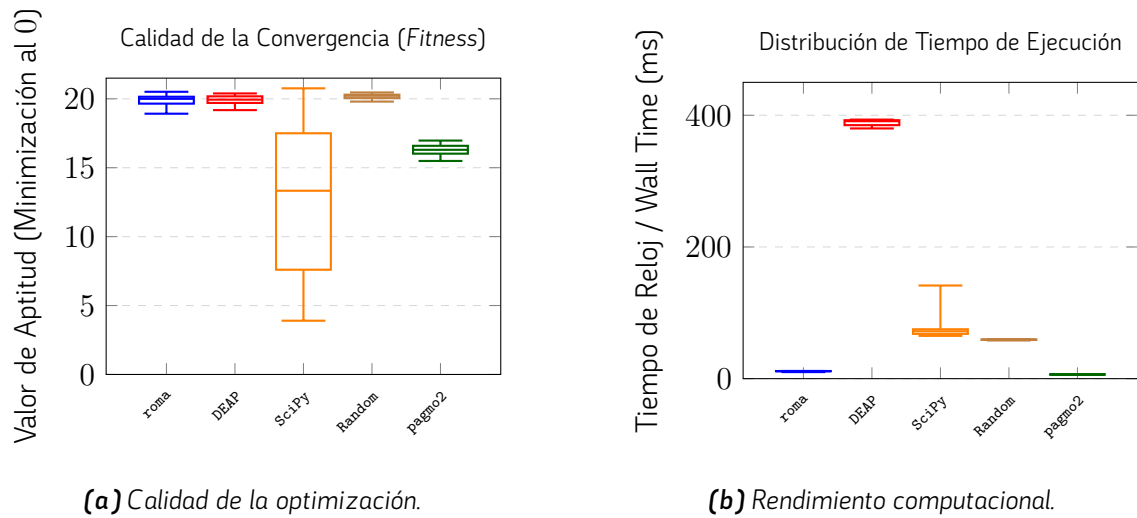


Figura 6.6. Resultados empíricos para la Función de Ackley ($D = 35$) con Evolución Diferencial (6400 evaluaciones, 30 ejecuciones).

Biblioteca	Aptitud de la Solución (Fitness ↓)			Tiempo de Ejecución Muro (ms ↓)		
	Mejor	Mediana	Peor	Mínimo*	Mediana	Máximo*
SciPy (Python)	3.90	13.33	20.76	65.00	71.94	141.48
pagmo2_cpp (C++)	15.49	16.30	16.97	6.00	6.50	6.55
DEAP (Python)	19.18	19.94	20.39	380.00	391.00	393.43
roma (Rust)	18.92	20.00	20.51	10.00	11.53	11.64
Random Search	19.80	20.22	20.46	58.00	59.31	59.95

* Los valores de tiempo están suavizados considerando las distribuciones centrales para evitar distorsiones por el encolamiento del sistema operativo.

Tabla 6.11. Métricas estadísticas agregadas para la instancia *ackley_dim_35* (6400 evaluaciones).

6.6.1 Análisis Crítico de Resultados

Durante la fase de experimentación preliminar, el comportamiento de las bibliotecas fue evaluado bajo un espectro escalonado de dimensionalidades ($D \in \{20, 30, 35, 40\}$) para estudiar el impacto de la escalabilidad en problemas multimodales. Sin embargo, para la redacción de esta memoria, se ha decidido exponer de forma exclusiva el escenario de $D = 35$. Esta selección metodológica radica en que dicha dimensión representa un punto de inflexión crítico en el estudio: es el umbral exacto donde

el volumen del hiperespacio restringe severamente el desempeño de las bibliotecas bajo el estricto presupuesto de 6400 evaluaciones, permitiendo observar el límite de rotura de las heurísticas sin que el problema se vuelva completamente inabordable.

Desde la perspectiva de la aptitud, este límite dimensional expone claramente la fragilidad de las configuraciones canónicas. **SciPy** sigue reportando la mejor mediana global (13.33), apoyándose en su hibridación algorítmica interna. No obstante, en contraste con su abrumador éxito en las pruebas previas de 20 o 30 dimensiones, su amplia dispersión estadística actual (con un IQR de 15.37 y un peor caso de 20.76) demuestra que su robustez comienza a colapsar al llegar a $D = 35$, volviéndose extremadamente dependiente de la siembra inicial para evadir la convergencia prematura. De igual manera, **pagmo2** sufre un retroceso respecto a dimensiones inferiores, estancando su mediana en 16.30.

Por su parte, las bibliotecas **roma** y **DEAP** mantienen un comportamiento estadísticamente equivalente, claudicando ante la topología corrugada y arrojando medianas de 20.00 y 19.94 respectivamente. Este estancamiento periférico, matemáticamente indistinguible de la Búsqueda Aleatoria pura (20.22), evidencia que el operador de cruce binomial estándar de DE carece de la diversidad genética necesaria para prosperar en hipervolumenes de esta magnitud.

No obstante, esta paridad generalizada en el desempeño biológico permite centrar el foco del experimento en el verdadero objetivo de la validación técnica: el **rendimiento computacional inter-hilo**. En este terreno, la superioridad de la biblioteca propuesta es categórica e inmune a la dimensionalidad.

Al ejecutar el bucle genético íntegro en una mediana de tan solo 11.53 **milisegundos**, **roma opera a una velocidad 33.9 veces mayor que DEAP** (391.00 ms). Este drástico descenso de latencia demuestra el impacto real de la compilación *Ahead-of-Time* (AOT) de Rust y la supresión del *Global Interpreter Lock* (GIL) presente en Python. Simultáneamente, **roma** se posiciona a escasos 5 milisegundos de la latencia nativa de C++ (**pagmo2**, 6.50 ms), validando contundentemente que la adopción de abstracciones de alto nivel mediante *traits* dinámicos en Rust proporciona un rendimiento de CPU casi idéntico al código estático de bajo nivel, sumando además estrictas garantías de seguridad en memoria.

6.6.2 Análisis de Significancia Estadística

Para dotar de rigor empírico a las observaciones extraídas sobre el estancamiento heurístico en alta dimensionalidad ($D = 35$), se ha procedido a evaluar la significancia de los resultados mediante un contraste de hipótesis. Aplicando el test *post-hoc* de Nemenyi para comparaciones múltiples sobre las distribuciones de aptitud, se ha calculado el valor p ajustado entre todos los marcos algorítmicos.

Fijando el umbral estándar de rechazo en $\alpha = 0.05$, la Tabla 6.12 recoge la matriz triangular resultante.

Test de Nemenyi (<i>p</i> -value)	pagmo2	SciPy	roma	DEAP	Random
pagmo2_cpp (C++)	-	0.0040	< 0.001	< 0.001	< 0.001
SciPy (Python)		-	0.9256	0.9256	0.0547
roma (Rust)			-	1.0000	0.3292
DEAP (Python)				-	0.3292
Random Search					-

Nota: Los valores en **negrita** indican una diferencia estadísticamente significativa ($p < 0.05$).

Tabla 6.12. Matriz de comparaciones múltiples (test de Nemenyi) para la aptitud final en la función de Ackley ($D = 35$).

El análisis de la matriz confirma matemáticamente la fragmentación del rendimiento expuesta en el apartado anterior.

En primer lugar, se certifica el aislamiento estadístico de **pagmo2 (C++)**. La biblioteca reporta valores p fuertemente significativos ($p < 0.05$) frente a la totalidad de sus competidores, demostrando que su configuración paramétrica logra penetrar la topología de la función de manera genuina y recurrente, sin deberse a variaciones aleatorias.

En segundo lugar, se expone el estado crítico de **SciPy**. Aunque logra diferencias con **pagmo2**, su comparación contra la Búsqueda Aleatoria (*Random Search*) arroja un valor $p = 0.0547$. Al situarse exactamente sobre la frontera del umbral de significancia, se comprueba estadísticamente la grave inestabilidad de su hibridación en esta dimensión; un porcentaje crítico de sus trayectorias ya no es capaz de superar a un muestreo estocástico puro.

Finalmente, el test rubrica un empate técnico entre **roma** y **DEAP**. El cruce entre ambas arrojó un valor idéntico a la unidad ($p = 1.0$), dictaminando que sus trayectorias de convergencia son matemática y estadísticamente indistinguibles ante este problema. Esta prueba de Nemenyi blindará argumentalmente la tesis de la biblioteca: al demostrar empíricamente que la calidad de la solución aportada por Rust es idéntica a la proporcionada por el estándar en Python, el incremento diferencial de velocidad documentado previamente ($33.9\times$) adquiere un valor incuestionable como mejora arquitectónica pura.

7

Conclusiones y líneas futuras

El diseño y desarrollo de la biblioteca `roma` ha demostrado la viabilidad y las enormes ventajas computacionales de emplear Rust para la construcción de motores de optimización metaheurística desde cero. Al prescindir de dependencias externas y apostar por un diseño estáticamente tipado, se ha logrado un sistema robusto, seguro en memoria y con tiempos de ejecución altamente competitivos frente a alternativas consolidadas en C++ o Python.

A este éxito técnico a nivel de rendimiento se suma el aprovechamiento integral del ecosistema nativo del lenguaje, destacando el papel fundamental de `cargo` como gestor de proyectos y sistema de construcción. Su uso exhaustivo ha permitido consolidar resultados tangibles y entregables de alto valor profesional como una documentación técnica completa, navegable e integrada directamente desde el código fuente mediante la herramienta `cargo doc`, lo cual garantiza una excelente experiencia de usuario y facilita la curva de aprendizaje para futuros contribuyentes. Por otro lado, la propia biblioteca ha sido estructurada y empaquetada formalmente como un `crate` de Rust. Este formato estandarizado no solo simplifica radicalmente su importación y uso en aplicaciones de terceros, sino que deja el proyecto completamente preparado para su eventual despliegue y publicación en el registro oficial de la comunidad (`crates.io`) [34].

Sin embargo, la envergadura arquitectónica de este proyecto también ha revelado una serie de limitaciones técnicas y compromisos de diseño que abren un amplio abanico de posibilidades para su evolución.

7.1 Limitaciones Identificadas

A partir de la fase de implementación y del análisis experimental de la biblioteca, se han identificado diversos retos estructurales que merecen una reflexión crítica. En primer lugar, destaca el compromiso ineludible entre abstracción y optimización. Para dotar al sistema de un carácter genérico capaz de unificar de manera transparente problemas mono-objetivo y multi-objetivo, ha sido necesario introducir un alto nivel de abstracción apoyado en el uso masivo de *traits* y polimorfismo dinámico en ciertos subsistemas. Esta generalización extrema, si bien arquitectónicamente elegante, dificulta la aplicación de optimizaciones de bajo nivel, dado que restringe el margen del compilador para predecir tipos concretos en tiempo de compilación y exprimir al máximo las capacidades de la CPU.

Esta rigidez inherente a las exigencias formales de Rust también repercute en la experiencia del desarrollador. Aunque el uso de la biblioteca no resulta conceptualmente complejo, la configuración e instanciación de sus componentes exige una verbosidad significativamente mayor que en ecosistemas dinámicos como Python (*DEAP* o *mealpy*), convirtiendo la implementación de nuevos algoritmos en un proceso en ocasiones tedioso. Adicionalmente, al tratarse de una primera iteración completa de la biblioteca, se asume una cierta deuda técnica; el código base aún presenta secciones difusas que requerirán refactorizaciones, unificación de convenciones de nomenclatura y una mejora general en su legibilidad.

Por otro lado, en el plano netamente operativo, se ha detectado una particularidad intrínseca en el modelo de ejecución paralela referente al determinismo en entornos concurrentes heterogéneos. Al ejecutar un algoritmo estocástico con una misma semilla en máquinas con diferente número de núcleos, el reparto y la planificación de los hilos varían. Esto altera el orden de evaluación y la sincronización de las trayectorias, provocando que los resultados diverjan entre equipos con distinto hardware. Aunque no constituye un fallo de programación *per se*, representa una pérdida de determinismo estricto inter-máquina que debe ser documentada para garantizar la reproducibilidad de los experimentos. Finalmente, el subsistema de tolerancia a fallos presenta ciertas inconsistencias funcionales. En el módulo de experimentos actual, la creación de puntos de persistencia (*checkpoints*) es obligatoria y no puede ser desactivada, lo que induce un estrés de escritura en disco innecesario. A esto se suma una interfaz de línea de comandos (CLI) poco intuitiva y compleja de manejar al intentar reanudar ejecuciones pausadas en experimentos.

7.2 Líneas futuras de trabajo

Tomando como base las limitaciones expuestas, y con el firme objetivo de consolidar `roma` como una herramienta de referencia en el ecosistema de Rust, se plantea una hoja de ruta con diversas líneas de desarrollo futuro que abordan de forma directa los problemas detectados.

Para mitigar la verbosidad y agilizar la experiencia del usuario, resulta prioritario el desarrollo de un módulo de macros procedurales nativas de Rust (tales como `#[derive(Observable)]` o `#[derive(Experimentable)]`). La integración de estas macros permitiría ocultar la complejidad interna del sistema y generar todo el código repetitivo de forma automatizada, transformando la implementación de nuevos algoritmos y problemas en un proceso mucho más declarativo y limpio.

En el ámbito del rendimiento computacional, se propone una refactorización profunda de los flujos de datos críticos del núcleo poblacional para transicionar hacia una filosofía *Zero-Copy*. La minimización sistemática de las clonaciones de estructuras en la memoria RAM supondría un salto cualitativo en la velocidad de los operadores genéticos y de los canales de comunicación de los observadores. En paralelo a esta optimización de memoria, es imprescindible evolucionar el modelo de concurrencia mediante un rediseño del planificador de tareas. El objetivo es trascender la simple ejecución en hilos independientes para implementar estrategias de grano fino que permitan paralelizar directamente a nivel de operador de variación o de evaluación individual, aprovechando verdaderamente todo el ancho de banda del procesador.

En cuanto a la operativa y robustez de la biblioteca, se proyecta una reestructuración del módulo de persistencia para conferir a los *checkpoints* un carácter totalmente opcional durante la fase de experimentos, eliminando así los cuellos de botella de entrada/salida (I/O) en disco. Esta mejora técnica vendrá acompañada del diseño de una CLI más amigable, moderna y robusta para la gestión, recuperación y limpieza del histórico de ejecuciones.

Por último, para apuntalar la madurez y utilidad del proyecto en el ámbito científico, se ampliará el ecosistema experimental incrementando exhaustivamente la cobertura de pruebas unitarias y de integración (*test coverage*). De forma complementaria, se nutrirá la biblioteca con una batería más extensa de problemas continuos y combinatorios, operadores avanzados y la incorporación de metaheurísticas multi-objetivo del estado del arte, tales como NSGA-III [6] o MOEA/D [43], culminando así su preparación para entornos de investigación competitivos.

7.3 Conclusiones sobre el desarrollo y el cambio de paradigma

El desarrollo de la biblioteca `roma` ha supuesto un profundo cambio de paradigma en lo que respecta al diseño y modelado de software algorítmico. Tradicionalmente, la Programación Orientada a Objetos (POO) convencional, abanderada por lenguajes como Java o C++, ha constituido el estándar académico y de la industria para modelar arquitecturas de optimización. Este enfoque se basa en la encapsulación conjunta de estado y comportamiento dentro de clases abstractas, apoyándose masivamente en la herencia para establecer jerarquías, compartir lógica común y reutilizar código entre distintas metaheurísticas.

A nivel formativo, la transición hacia este nuevo ecosistema ha requerido una profunda deconstrucción de los principios asimilados durante la etapa académica universitaria. La enseñanza clásica de estructuras de datos y patrones de diseño suele cimentarse sobre la manipulación libre de referencias cruzadas y el estado mutable compartido. Sin embargo, este esquema mental entra en conflicto directo con las estrictas reglas de Rust; dominar el lenguaje implicó no solo adquirir una nueva sintaxis, sino desaprender hábitos de diseño fuertemente arraigados para lograr satisfacer las exigencias de seguridad de memoria del compilador.

La adopción de Rust supuso un reto arquitectónico mayúsculo al alejarse deliberadamente de este estándar. El lenguaje de sistemas prescinde por completo del concepto tradicional de "clase" y de la herencia. En su lugar, propone una separación estricta mediante un modelo basado en estructuras de datos (`structs`) para definir el estado, y rasgos abstractos (`traits`) para definir el comportamiento, adoptando una filosofía que denota una clara influencia de las clases de tipos de lenguajes funcionales como Haskell.

Esta divergencia paradigmática generó notables complicaciones durante las fases iniciales de diseño. La falta de herencia actuó como un factor limitante a la hora de estructurar familias de algoritmos y operadores, provocando bloqueos que requirieron múltiples refactorizaciones del núcleo de la biblioteca. Para superar estas restricciones, fue imperativo abandonar los patrones jerárquicos y adoptar principios de diseño más modernos y modulares, fundamentados en la regla de "composición sobre herencia", el uso avanzado de tipos genéricos y el despacho dinámico.

A pesar de la exigente curva de aprendizaje impuesta por estas normativas estructurales y por su riguroso sistema de propiedad en memoria (*Borrow Checker*), la conclusión fundamental de este proyecto es que **Rust es un lenguaje completamente viable y excepcionalmente competitivo para el desarrollo de bibliotecas metaheurísticas**. Su capacidad para garantizar concurrencia segura frente a condiciones de carrera (*data races*) y sus abstracciones de coste cero (*zero-cost abstractions*)

permiten construir motores de optimización con un rendimiento computacional puro comparable a C/C++, pero con garantías de seguridad de memoria muy superiores.

Si bien la implementación actual de `roma` cumple satisfactoriamente con los objetivos propuestos, la experiencia empírica y las lecciones de Ingeniería de Software asimiladas a lo largo de este proyecto revelan que el sistema aún tiene un margen de evolución extraordinario. Conocidos ahora los obstáculos iniciales del lenguaje y dominando sus patrones idiomáticos, sería posible aplicar estas directrices desde la base del diseño. Reestructurando el núcleo algorítmico, implementando macros procedurales¹ para reducir la verbosidad y transicionando hacia una gestión de memoria *Zero-Copy*, se podría perfeccionar la arquitectura actual hasta consolidar una biblioteca verdaderamente increíble; una herramienta de vanguardia capaz de liderar la investigación metaheurística por su equilibrio perfecto entre ergonomía, seguridad y velocidad extrema.

¹Las macros procedurales (*procedural macros*) constituyen la herramienta de metaprogramación más avanzada del ecosistema de Rust. Operan durante la fase de compilación actuando como funciones que reciben el código fuente en forma de flujo de *tokens* y devuelven nuevo código generado dinámicamente. Esta capacidad permite automatizar la implementación de *traits* personalizados (típicamente mediante el uso de `#[derive(...)]`) y erradicar drásticamente el código repetitivo (*boilerplate*) inherente a las arquitecturas fuertemente tipadas, manteniendo intacto el principio de abstracción de coste cero.

Apéndice A. Manual de instalación

En esta sección se describe el proceso de instalación y puesta en marcha de la biblioteca de optimización metaheurística desarrollada en este trabajo. Dado que el proyecto ha sido implementado íntegramente en Rust, su integración en otros proyectos resulta sencilla y se ajusta a los mecanismos estándar proporcionados por el propio ecosistema del lenguaje.

A.1 Requisitos

Para poder utilizar la biblioteca es necesario disponer de los siguientes elementos:

- Un sistema operativo compatible con el compilador de Rust (Linux, macOS o Windows).
- El compilador de Rust y su gestor de paquetes **Cargo**, instalados y correctamente configurados.
- Acceso a Internet para la descarga del código fuente o del *crate* correspondiente.

La instalación del entorno de desarrollo de Rust puede realizarse siguiendo las instrucciones oficiales proporcionadas por el lenguaje.

A.2 Instalación desde el repositorio

El método más directo para utilizar la biblioteca consiste en clonar el repositorio desde la plataforma GitHub y compilar el proyecto de forma local. Este enfoque resulta especialmente útil para desarrolladores que deseen inspeccionar, modificar o extender el código fuente.

El repositorio puede descargarse ejecutando el siguiente comando:

```
git clone https://github.com/DRLKs/roma
```

Una vez clonado el repositorio, basta con acceder al directorio del proyecto y ejecutar:

```
cargo build
```

Este comando descargará automáticamente las dependencias necesarias y generará la biblioteca en modo desarrollo. Para ejecutar ejemplos incluidos en el repositorio o realizar pruebas, puede utilizarse el comando:

```
cargo run
```

A.3 Instalación como *crate* mediante Cargo

Dado que la biblioteca ha sido diseñada siguiendo las convenciones estándar del ecosistema Rust, también puede integrarse fácilmente como dependencia en cualquier proyecto Rust mediante el gestor de paquetes **Cargo**. Este método es el más recomendado para usuarios que deseen emplear la biblioteca como componente dentro de una aplicación mayor.

Para ello, basta con añadir la dependencia correspondiente en el archivo **Cargo.toml** del proyecto:

```
[dependencies]
roma_lib = "0.1.2"
```

Una vez añadida la dependencia, Cargo se encargará automáticamente de descargar, compilar e integrar la biblioteca durante el proceso de construcción del proyecto. A partir de ese momento, la funcionalidad proporcionada por la biblioteca podrá utilizarse directamente desde el código fuente mediante las instrucciones `use crate::` habituales del lenguaje.

Este mecanismo de distribución facilita la reutilización del software desarrollado y permite mantener actualizada la biblioteca de forma sencilla a medida que se publiquen nuevas versiones.

Apéndice B. Manual de uso

El propósito de este apéndice es ilustrar la metodología de trabajo y la interfaz de programación (API) que ofrece la biblioteca desarrollada. Para ello, se expone un escenario de uso autocontenido donde se resuelve una instancia reducida del problema de la mochila 0/1 (*Knapsack Problem*) empleando el algoritmo de Escalada Simple (*Hill Climbing*).

La filosofía de diseño de la herramienta busca separar de forma clara la definición del problema matemático, la configuración de la metaheurística y la recolección de métricas. El flujo de programación estándar se articula mediante las siguientes fases metodológicas:

B.1 Inicialización y definición del problema

El punto de partida de cualquier ejecución requiere modelar el escenario de optimización. La biblioteca emplea el patrón de diseño de *software Builder* para facilitar esta tarea, permitiendo instanciar el problema de forma declarativa.

En el fragmento expuesto a continuación, se captura una semilla estocástica a través del gestor de argumentos por línea de comandos (*CliArgs*) para asegurar la reproducibilidad determinista. Seguidamente, se define una mochila con una capacidad máxima de 90.0 unidades y se registran secuencialmente tres elementos, especificando para cada uno de ellos su peso y su valor asociado.

```
1 // 1. Obtencion de la semilla y definicion del problema
2 let seed = CliArgs::from_env().seed_or(42);
3
4 let problem = KnapsackBuilder::new()
5     .with_capacity(90.0)
6     .add_item(12.0, 24.0)
7     .add_item(22.0, 33.0)
8     .add_item(41.0, 80.0)
9     .build();
```

Código 7.1. Captura de semilla y construcción del problema de la mochila.

B.2 Configuración del motor heurístico

Una vez establecido el dominio del problema, se procede a ensamblar el motor de búsqueda. En este caso particular, la parametrización de la Escalada Simple exige dos componentes estructurales básicos:

- **Operador de vecindad:** Se inyecta la utilidad `BitFlipNeighborhood`, encargada de generar soluciones candidatas adyacentes invirtiendo puntualmente valores en la representación binaria.
- **Criterio de parada:** Se define una política de interrupción (`TerminationCriteria`), instruyendo al motor para que detenga la búsqueda al completar exactamente 120 iteraciones.

Estos elementos se empaquetan en una estructura de configuración unificada (`HillClimbingParameters`), con la cual se instancia de forma definitiva el algoritmo.

```
1 // 2. Configuración de la metaheurística
2 let parameters = HillClimbingParameters::new(
3     BitFlipNeighborhood::new(),
4     TerminationCriteria::new(vec![TerminationCriterion::MaxIterations(120)]),
5 )
6 .with_seed(seed);
7
8 let mut algorithm = HillClimbing::new(parameters);
```

Código 7.2. Parametrización e instanciación del algoritmo de Escalada Simple.

B.3 Acoplamiento de Observadores

Uno de los rasgos más potentes de la arquitectura de la biblioteca es su sistema de monitorización o telemetría pasiva. En lugar de forzar la impresión intrusiva de datos durante la evaluación, el desarrollador puede vincular múltiples observadores al algoritmo antes de iniciar la ejecución.

El siguiente bloque adjunta tres mecanismos de registro simultáneos: salida por terminal (`ConsoleObserver`), trazado de gráficas (`ChartObserver`) y generación de un documento web (`HtmlReportObserver`).

```
1 // 3. Inyección de las herramientas de monitorización
2 algorithm.add_observer(Box::new(ConsoleObserver::new(true)));
3 algorithm.add_observer(Box::new(ChartObserver::new_default()));
4 algorithm.add_observer(Box::new(HtmlReportObserver::new_default()));
```

Código 7.3. Inyección de observadores para la recolección de métricas.

B.4 Ejecución y extracción de resultados

El ciclo de vida del bloque de código culmina con la invocación del método `run`. Este proceso vincula el algoritmo con el problema matemático definido en el primer paso y desencadena el proceso de búsqueda en el procesador.

Al finalizar, la función devuelve un objeto que consolida el histórico de la prueba, a partir del cual se extrae la mejor solución encontrada en el espacio de búsqueda y se imprime su métrica de aptitud (*fitness*).

```
1 // 4. Ejecucion y recoleccion de los resultados
2 let result = algorithm
3   .run(&problem)
4   .expect("Fallo interno en la ejecucion del Hill Climbing");
5
6 if let Some(best) = result.best_solution(&problem) {
7   println!(
8     "Optimizacion finalizada (semilla={}). Mejor aptitud={:.4}",
9     seed,
10    best.quality_value()
11  );
12 } else {
13   println!("Optimizacion finalizada sin soluciones validas (semilla={})", seed);
14 }
```

Código 7.4. Ejecución de la búsqueda y extracción de la mejor solución.

B.5 Ensamblaje final del programa

Para facilitar la comprensión global y asegurar la reproducibilidad del ejemplo, se presenta a continuación el código fuente íntegro. Este bloque incluye las sentencias de importación (`use`) necesarias y encapsula el flujo de trabajo descrito anteriormente dentro de la función principal de ejecución (`main`).

```
1 use roma_lib::HtmlReportObserver;
2 use roma_lib::algorithms::{
3   Algorithm,
4   HillClimbing,
5   HillClimbingParameters,
6   TerminationCriteria,
7   TerminationCriterion,
8 };
9 use roma_lib::observer::{ChartObserver, ConsoleObserver, Observable};
10 use roma_lib::operator::BitFlipNeighborhood;
```

```

11 use roma_lib::problem::KnapsackBuilder;
12 use roma_lib::solution_set::SolutionSet;
13 use roma_lib::utils::cli::CliArgs;
14
15 fn main() {
16     let seed = CliArgs::from_env().seed_or(42);
17
18     let problem = KnapsackBuilder::new()
19         .with_capacity(90.0)
20         .add_item(12.0, 24.0)
21         .add_item(22.0, 33.0)
22         .add_item(41.0, 80.0)
23         .build();
24
25     let parameters = HillClimbingParameters::new(
26         BitFlipNeighborhood::new(),
27         TerminationCriteria::new(vec![TerminationCriterion::MaxIterations(120)]),
28     )
29     .with_seed(seed);
30
31     let mut algorithm = HillClimbing::new(parameters);
32
33     algorithm.add_observer(Box::new(ConsoleObserver::new(true)));
34     algorithm.add_observer(Box::new(ChartObserver::new_default()));
35     algorithm.add_observer(Box::new(HtmlReportObserver::new_default()));
36
37     let result = algorithm
38         .run(&problem)
39         .expect("Fallo interno en la ejecucion del Hill Climbing");
40
41     if let Some(best) = result.best_solution(&problem) {
42         println!(
43             "Optimizacion finalizada (semilla={}). Mejor aptitud={:.4}",
44             seed,
45             best.quality_value()
46         );
47     } else {
48         println!("Optimizacion finalizada sin soluciones validas (semilla={})", seed);
49     }

```

50 }

Código 7.5. *Implementación íntegra del flujo de trabajo.*

Bibliografía

- [1] David H Ackley. "A connectionist machine for genetic hillclimbing". In: *In Proceedings of the Fourth International Workshop on Machine Learning*. Morgan Kaufmann, 1987, pp. 311–318.
- [2] Atlassian. *JIRA: Issue and Project Tracking Software*. Version 9.0. Herramienta para gestión de proyectos, seguimiento de incidencias y colaboración en equipo. URL: <https://www.atlassian.com/software/jira> (visited on 02/17/2026).
- [3] The rust-analyzer authors. *rust-analyzer*. URL: <https://github.com/rust-lang/rust-analyzer> (visited on 02/09/2026).
- [4] Francesco Biscani and Dario Izzo. *pagmo: A C++ platform for massively parallel optimization*. URL: <https://esa.github.io/pagmo2/> (visited on 05/05/2026).
- [5] Christian Blum and Andrea Roli. "Metaheuristics in Combinatorial Optimization: Overview and Conceptual Comparison". In: *ACM Computing Surveys (CSUR)* 35.3 (2003). Artículo de revisión fundamental que introduce, clasifica y compara conceptualmente las principales estrategias metaheurísticas aplicadas a la resolución de problemas de optimización combinatoria, pp. 268–308. DOI: [10.1145/937503.937505](https://doi.org/10.1145/937503.937505). URL: <https://dl.acm.org/doi/10.1145/937503.937505> (visited on 05/16/2026).
- [6] Kalyanmoy Deb and Himanshu Jain. "An Evolutionary Many-Objective Optimization Algorithm Using Reference-Point-Based Nondominated Sorting Approach, Part I: Solving Problems With Box Constraints". In: *IEEE Transactions on Evolutionary Computation* 18.4 (2014), pp. 577–601. DOI: [10.1109/TEVC.2013.2281535](https://doi.org/10.1109/TEVC.2013.2281535).
- [7] Kalyanmoy Deb et al. "A fast and elitist multiobjective genetic algorithm: NSGA-II". In: *IEEE transactions on evolutionary computation* 6.2 (2002), pp. 182–197.
- [8] Janez Demšar. "Statistical comparisons of classifiers over multiple data sets". In: *Journal of Machine Learning Research* 7.Dec (2006), pp. 1–30.
- [9] The Rand Project Developers and The Rust Project Developers. *rand: Rust random number generators and utilities*. Version 0.10.0. Crate for random number generation in Rust (MIT OR Apache-2.0). URL: <https://github.com/rust-random/rand> (visited on 02/17/2026).

- [10] Docker, Inc. *Docker Documentation*. Consultado el 31 de mayo de 2026. 2025. URL: <https://docs.docker.com/>.
- [11] J. J. Durillo and A. J. Nebro. *jMetalPy*. URL: <https://github.com/jMetal/jMetalPy> (visited on 02/09/2026).
- [12] Félix-Antoine Fortin and colleagues. *DEAP: Distributed Evolutionary Algorithms in Python*. URL: <https://deap.readthedocs.io/en/master/> (visited on 05/05/2026).
- [13] Milton Friedman. "A comparison of alternative tests of significance for the problem of m rankings". In: *The Annals of Mathematical Statistics* 11.1 (1940), pp. 86–92.
- [14] Salvador García and Francisco Herrera. "An extension on "statistical comparisons of classifiers over multiple data sets" for all pairwise comparisons". In: *Journal of Machine Learning Research* 9.Dec (2008), pp. 2677–2694.
- [15] Inc. GitHub. *GitHub: Plataforma de alojamiento de repositorios*. Version N/A. Plataforma de desarrollo colaborativo que permite alojar repositorios Git y facilita la gestión de proyectos de software. URL: <https://github.com/> (visited on 02/17/2026).
- [16] Fred Glover. "Future paths for integer programming and links to artificial intelligence". In: *Computers & Operations Research* 13.5 (1986), pp. 533–549.
- [17] Pierre Hansen and Nenad Mladenović. "Variable neighborhood search: Principles and applications". In: *European Journal of Operational Research* 130.3 (2001), pp. 449–467.
- [18] Sture Holm. "A simple sequentially rejective multiple test procedure". In: *Scandinavian journal of statistics* (1979), pp. 65–70.
- [19] Hao Hou, Aaron Erhardt, and contributors. *Plotters: A Rust drawing and plotting library*. Version 0.3.7. Librería Rust para generar gráficos y visualizaciones de datos. URL: <https://github.com/plotters-rs/plotters> (visited on 02/17/2026).
- [20] A. J. Nebro J. J. Durillo. *jMetal*. URL: <https://github.com/jMetal/jMetal> (visited on 02/09/2026).
- [21] JetBrains s.r.o. *JetBrains Toolbox*. Version 2.3. Aplicación de gestión del ecosistema y suite de entornos de desarrollo integrado (IDEs) profesionales para desarrollo de software y gestión de proyectos. URL: <https://www.jetbrains.com> (visited on 05/16/2026).

- [22] JetBrains s.r.o. *RustRover*. Version 2026.1.1. Entorno de desarrollo integrado (IDE) dedicado para el lenguaje de programación Rust, con soporte nativo para Cargo, asistencia avanzada en la edición de código y herramientas de depuración. URL: <https://www.jetbrains.com/rust/> (visited on 05/16/2026).
- [23] Richard M Karp. "Reducibility among combinatorial problems". In: *Complexity of computer computations*. Springer, 1972, pp. 85–103.
- [24] Scott Kirkpatrick, C Daniel Gelatt, and Mario P Vecchi. "Optimization by simulated annealing". In: *Science* 220.4598 (1983), pp. 671–680.
- [25] Thorsten Leemhuis. *Linux Kernel: Rust Support Officially Approved*. heise online. Dec. 2025. URL: <https://www.heise.de/en/news/Linux-Kernel-Rust-Support-Officially-Approved-11109808.html> (visited on 02/15/2026).
- [26] Helena R Lourenço, Olivier C Martin, and Thomas Stützle. "Iterated local search". In: *Handbook of metaheuristics* (2003), pp. 320–353.
- [27] Massachusetts Institute of Technology. *The MIT License*. <https://opensource.org/licenses/MIT>. Accedido el 21 de mayo de 2026. 1988.
- [28] Microsoft Corporation. *Visual Studio Code*. Version 1.104. Editor de código fuente ligero, potente y extensible con soporte integrado para desarrollo de software, depuración y control de versiones. URL: <https://code.visualstudio.com> (visited on 05/16/2026).
- [29] Peter Bjorn Nemenyi. "Distribution-free multiple comparisons". PhD thesis. Princeton University, 1963.
- [30] Stack Overflow. *Stack Overflow Developer Survey 2025*. URL: <https://survey.stackoverflow.co/2025/> (visited on 10/12/2025).
- [31] L. A. Rastrigin. *Systems of Extremal Control*. Moscú, URSS: Nauka, 1974.
- [32] El-Ghazali Talbi. *Metaheuristics: from design to implementation*. John Wiley & Sons, 2009.
- [33] The LaTeX Project. *LaTeX*. Version 2e. Sistema de preparación de documentos para la composición tipográfica de alta calidad, ampliamente utilizado en el ámbito científico y académico. URL: <https://www.latex-project.org> (visited on 05/16/2026).
- [34] The Rust Project Developers. *crates.io: The Rust community's crate registry*. <https://crates.io/>. Accedido en: Mayo de 2026. 2026.

- [35] The Rust Project Developers. *The Rust Programming Language*. Version 1.95.0. Lenguaje de programación multiparadigma, de código abierto, enfocado en el rendimiento y la seguridad, diseñado para garantizar la gestión segura de la memoria sin necesidad de un recolector de basura. URL: <https://www.rust-lang.org> (visited on 05/16/2026).
- [36] Linus Torvalds and Junio C Hamano. *Git: Distributed Version Control System*. Version 2.44. Sistema de control de versiones distribuido utilizado para el seguimiento de cambios en el código fuente durante el desarrollo de software. URL: <https://git-scm.com/> (visited on 02/17/2026).
- [37] David Muñoz del Valle. *PokerHelper*. URL: <https://github.com/DRLKs/PokerHelper> (visited on 02/09/2026).
- [38] Pauli Virtanen et al. "SciPy 1.0: fundamental algorithms for scientific computing in Python". In: *Nature methods* 17.3 (2020), pp. 261–272.
- [39] Visual Paradigm International. *Visual Paradigm Professional*. Version 17.0. Herramienta profesional para modelado UML, diseño de arquitectura software y gestión del ciclo de vida del desarrollo. URL: <https://www.visual-paradigm.com> (visited on 02/17/2026).
- [40] Wolfram Research, Inc. *Wolfram|Cloud*. Champaign, IL, EE. UU. 2026. URL: <https://www.wolframcloud.com>.
- [41] David H Wolpert and William G Macready. "No free lunch theorems for optimization". In: *IEEE transactions on evolutionary computation* 1.1 (1997), pp. 67–82.
- [42] Xin-She Yang and colleagues. *MEALPY: MEta-heuristic ALgorithms in PYthon*. URL: <https://mealpy.readthedocs.io/en/latest/pages/general/introduction.html> (visited on 02/09/2026).
- [43] Qingfu Zhang and Hui Li. "MOEA/D: A Multiobjective Evolutionary Algorithm Based on Decomposition". In: *IEEE Transactions on Evolutionary Computation* 11.6 (2007), pp. 712–731. DOI: [10.1109/TEVC.2007.892759](https://doi.org/10.1109/TEVC.2007.892759).



UNIVERSIDAD
DE MÁLAGA

| **uma.es**

ETS. de Ingeniería Informática

Bulevar Louis Pasteur, 35

Campus de Teatinos

29071 Málaga